From One Declaration to One Self-Revising Fractal: Admissibility, Residual Governance, and Recursive Objectivity in Semantic Meme Field Theory

Part 4 of "From One Assumption to One Operator"

A fourth discussion on how ledgered declaration becomes observerhood through admissible self-revision

---

Abstract

Part 1 of this sequence explored the temptation of one operator. It asked whether a primitive operation, repeated recursively, could present the hidden structure of a pre-time field.

The guiding movement was:

primitive operation → recursion → pre-time → collapse → ledger → time. (0.1)

Part 2 corrected this movement. It argued that recursion should not be read as literal pre-time generation. Recursion may be a grammar of presentation rather than a process running before time. The pre-time field does not evolve before time; it is disclosed through viewpoint-selected filtration.

The corrected movement was:

pre-time field → viewpoint → filtration → collapse → ledger → time. (0.2)

Part 3 then found the hidden condition inside filtration. A field is not filterable merely because a viewpoint exists. A viewpoint must become a declaration. It must declare a baseline q, feature map φ, protocol P, projection operator $\hat{O}_P$, gate, trace rule, and residual rule. Time was therefore redefined as ledgered declared disclosure.

The central Part 3 operator was:

$\mathfrak{D}_P = \text{UpdateTrace}_P \circ \text{Gate}_P \circ \hat{O}_P \circ \text{Declare}_P$. (0.3)

and the resulting time formula was:

$\text{Time}_P = \text{order}(\mathfrak{D}_P(\Sigma_0))$. (0.4)

Part 4 now asks the next question.

If declared disclosure produces trace and residual, and if trace and residual can revise future declaration, when does this revision become observerhood rather than arbitrary self-modification?

The answer developed here is admissible self-revision.

A system is not mature merely because it can revise its own declaration. A pathological system can revise itself by erasing its past, hiding residual, breaking frame robustness, redefining contradiction as confirmation, or changing its rules whenever failure appears. Such a system is

not an observer in the mature sense. It is an unstable or dogmatic self-modifier.

A mature observer is a stable self-revising declaration system constrained by admissibility.

The declaration at episode k is:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (0.5)

where:

$P_k = (B_k, \Delta_k, h_k, u_k)$. (0.6)

A self-revision has the form:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (0.7)

where L_k is ledgered trace, R_k is residual, and U_a is an admissible revision operator.

The admissible declaration family is:

$\mathfrak{F}_adm = \{D \mid WellFormed(D) \wedge TracePreserving(D) \wedge ResidualHonest(D) \wedge FrameRobust(D) \wedge BudgetBounded(D) \wedge NonDegenerate(D)\}$. (0.8)

The family of admissible self-revisions generates an iterated structure:

$\mathfrak{A} = \bigcup_{a \in A_adm} U_a(\mathfrak{A})$. (0.9)

This is the Self-Revising Declaration Fractal.

The mature observer is then:

$\hat{O}_self = Fix(\mathcal{U} \mid \mathfrak{F}_adm)$. (0.10)

In words:

$\hat{O}_self$ is the stable attractor of trace-preserving admissible declaration revision. (0.11)

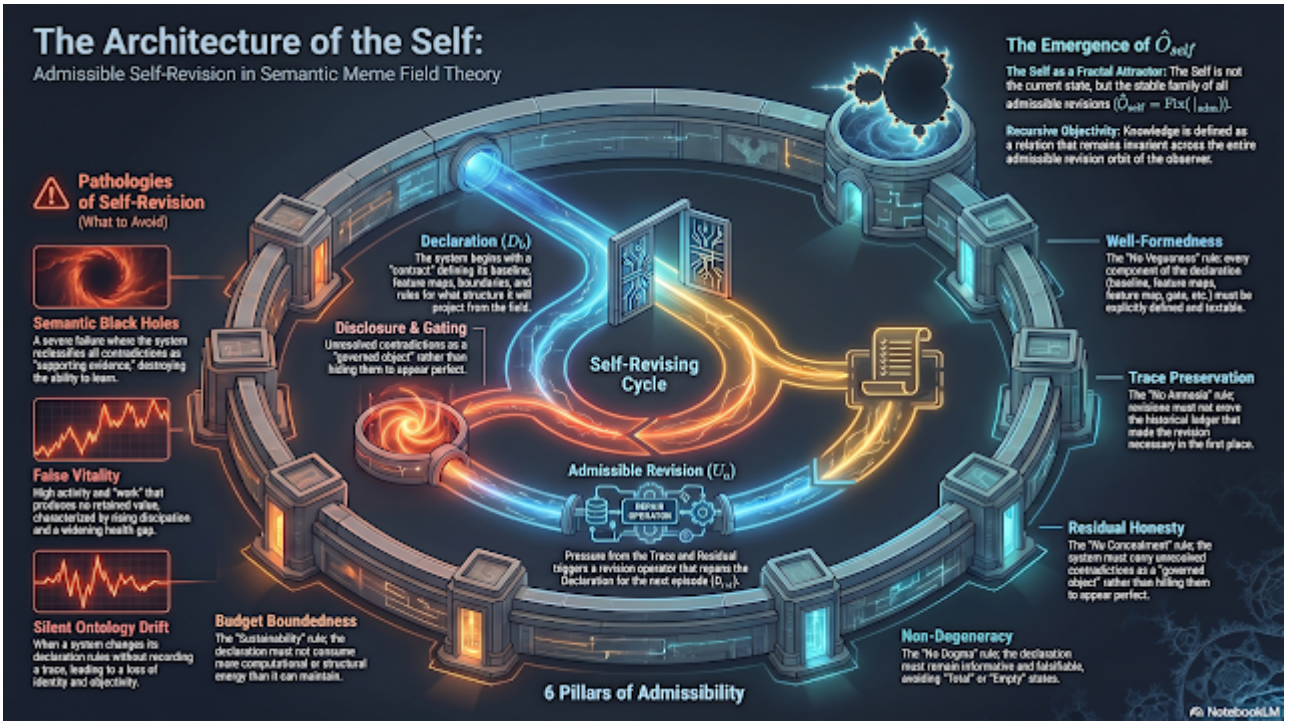
Part 4's central thesis is therefore:

Selfhood is not merely projection, memory, recursion, or self-reference. Selfhood is admissible self-revision of the declaration that governs future projection. (0.12)

This article defines the admissibility constraints, explains why they are necessary, classifies the main pathologies of self-revision, introduces trust regions and switch gates, defines declaration energy, and explains recursive objectivity as invariance across the admissible revision orbit.

The final movement of the tetralogy is:

One Assumption → One Operator → One Filtration → One Declaration → One Self-Revising Fractal. (0.13)



---

0. Reader's Guide: Why Part 4 Is Needed

0.1 What Part 3 completed

Part 3 completed the theory of declared time.

It began from the problem left by Part 2:

What makes Σ filterable? (0.14)

The answer was:

Declaration. (0.15)

A pre-time field does not become filterable in the abstract. It becomes filterable when a bounded observer declares the conditions under which structure can be distinguished, projected, gated, traced, and carried as residual.

The core Part 3 chain was:

$\Sigma_0 \to \mathrm{Declare_P} \to \Sigma_P \to \hat{O}_P \to \mathrm{Gate_P} \to \mathrm{Trace_P} + \mathrm{Residual_P} \to \mathrm{Ledger_P} \to \mathrm{Time_P}$. (0.16)

The core Part 3 operator was:

$\mathfrak{D}_P = \mathrm{UpdateTrace_P} \circ \mathrm{Gate_P} \circ \hat{O}_P \circ \mathrm{Declare_P}$. (0.17)

The core Part 3 definition of time was:

$\mathrm{Time_P} = \mathrm{order}(\mathfrak{D}_P(\Sigma_0))$. (0.18)

In words:

Time is ledgered declared disclosure. (0.19)

This solved the hidden meta-time problem. The pre-time field does not need to run, tick, update, or recurse before time. It becomes time-like only when declared projection is gated into trace and ordered in ledger.

0.2 What Part 3 left open

Part 3 ended with a new problem.

If declared disclosure produces trace and residual, then trace and residual can pressure the next declaration.

The system can say:

The old boundary was wrong. (0.20)

The old feature map was insufficient. (0.21)

The old baseline was outdated. (0.22)

The old gate opened too early. (0.23)

The old trace rule failed to preserve accountability. (0.24)

The old residual rule hid too much. (0.25)

The old frame robustness test failed. (0.26)

This produces a natural revision rule:

$D_{k+1} = \text{Revise}(D_k \mid L_k, R_k)$. (0.27)

Or:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (0.28)

This is where Part 4 begins.

Part 3 explained how a declaration produces time.

Part 4 explains how time, residual, and trace revise declaration.

0.3 Why self-revision is dangerous

Self-revision sounds like intelligence. But not all self-revision is intelligent.

A system can revise itself pathologically.

It can revise by forgetting:

D_{k+1} erases L_k. (0.29)

It can revise by denial:

R_k rises but D_{k+1} refuses to change. (0.30)

It can revise by noise:

D_{k+1} changes without trace-preserving reason. (0.31)

It can revise by semantic black hole:

Contradiction is reclassified as confirmation. (0.32)

It can revise by frame collapse:

Equivalent frames no longer produce equivalent governed responses. (0.33)

It can revise by budget collapse:

The new declaration is too costly, dissipative, or unstable to maintain. (0.34)

Therefore, self-revision alone does not define observerhood.

The missing word is admissibility.

0.4 The central question of Part 4

Part 4 asks:

When is self-revision admissible? (0.35)

More fully:

When may a ledgered declaration revise itself without losing trace, hiding residual, breaking frame robustness, exceeding budget, or becoming empty, total, or unfalsifiable? (0.36)

The answer is a constraint family.

A declaration belongs to the admissible family if:

$\mathfrak{F}_adm = \{D \mid WellFormed(D) \wedge TracePreserving(D) \wedge ResidualHonest(D) \wedge FrameRobust(D) \wedge BudgetBounded(D) \wedge NonDegenerate(D)\}$. (0.37)

A revision operator is admissible if:

$U_a : \mathfrak{F}_adm \times Ledger \times Residual \rightarrow \mathfrak{F}_adm$. (0.38)

and:

$\forall D \in \mathfrak{F}_adm, U_a(D,L,R) \in \mathfrak{F}_adm$. (0.39)

This means that admissible revision stays inside the admissible declaration family.

The self-revising observer is then the stable attractor of such revisions:

$\hat{O}_self = Fix(\mathcal{U} \mid \mathfrak{F}_adm)$. (0.40)

0.5 Why this is naturally fractal

The structure is fractal because the output of one declared disclosure becomes the input for the next declaration.

At each cycle:

Declaration produces projection. (0.41)

Projection passes or fails gate. (0.42)

Gate writes trace. (0.43)

Trace leaves residual. (0.44)

Residual pressures revision. (0.45)

Revision produces new declaration. (0.46)

Then the cycle repeats.

In compact form:

Declaration → Projection → Gate → Trace + Residual → Revision → Declaration. (0.47)

The same pattern occurs at multiple levels.

A lower-level closure becomes a higher-level declaration input:

$\text{Closure}_n \rightarrow \text{DeclarationInput}_{n+1}$. (0.48)

This gives the self-similar family:

$\mathfrak{U} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{U})$. (0.49)

This is not yet a fully solved mathematical theorem. It is a proposed constraint family for future mathematics. But it is explicit enough to define the next object.

0.6 The movement from time to observerhood

Part 3 gave:

$\text{Time}_P = \text{order}(\mathfrak{D}_P(\Sigma_0))$. (0.50)

Part 4 gives:

$\hat{O}_{self} = \text{Fix}(\mathcal{U} \mid \mathfrak{F}_{adm})$. (0.51)

The transition is:

$\text{Time}_P \rightarrow \text{AdmissibleDeclarationRevision} \rightarrow \hat{O}_{self}$. (0.52)

In words:

Time gives history. Residual gives pressure. Admissible revision gives observerhood. (0.53)

A system with time can have history.

A system with history and residual can learn.

A system that uses history and residual to revise its declaration admissibly becomes observer-like.

A system whose admissible revisions stabilize into a non-degenerate attractor becomes self-like.

0.7 Roadmap

The article proceeds in six movements.

First, Sections 1–3 define the transition from declared time to self-revising declaration. They introduce D_k and U_a.

Second, Sections 4–10 define the admissibility family: well-formedness, trace preservation, residual honesty, frame robustness, budget boundedness, and non-degeneracy.

Third, Sections 11–12 add trust regions, switch gates, quarantine, and declaration energy.

Fourth, Sections 13–14 define the Self-Revising Declaration Fractal $\mathfrak{A}$ and its possible contraction and stability conditions.

Fifth, Sections 15–17 classify failure modes, define $\hat{O}_{self}$, and extend objectivity into recursive objectivity.

Sixth, Section 18 closes the tetralogy and explains how the theory moves from one assumption to one self-revising observer.

The shortest roadmap is:

Declared time → trace and residual → admissible revision → fractal attractor → $\hat{O}_{self}$. (0.54)

---

1. From Declared Time to Self-Revising Observerhood

1.1 Time is not yet selfhood

Part 3 defined time as ledgered declared disclosure:

$$\mathrm{Time}_P = \mathrm{order}(\mathfrak{D}_P(\Sigma_0)). \quad (1.1)$$

This is a strong result. But time is not yet selfhood.

A ledger may record events without revising the declaration that produced them. Such a ledger has history, but not selfhood.

For example:

A camera records images, but may not revise its own feature map. (1.2)

A database stores records, but may not revise its own schema. (1.3)

A legal archive stores precedent, but only a court or institution interprets and updates future judgment. (1.4)

An AI log stores outputs, but only a governed runtime uses those logs to revise retrieval, verification, and gating. (1.5)

Thus:

Ledger alone ≠ self. (1.6)

Time alone ≠ observerhood. (1.7)

Observerhood begins when trace changes future disclosure.

Selfhood begins when trace changes future declaration.

1.2 From trace to future disclosure

A trace is not a passive record. It bends future projection.

In Part 3:

Trace = stored record that bends future projection. (1.8)

So a trace-bearing system is already more than a simple reactive system.

A reactive system follows:

Input → Response. (1.9)

An adaptive system follows:

Input → Response → Trace → ModifiedResponse. (1.10)

An observer-like system follows:

Input → Declaration → Projection → Gate → Trace + Residual → DeclarationRevision. (1.11)

The last form is the important one.

The observer-like system does not merely respond differently after trace. It revises the way it declares the field.

Thus:

ObserverPotential_P = ability of L_P and R_P to modify future disclosure. (1.12)

And:

Ô_self = L modifying future D. (1.13)

1.3 Declaration as the object of self-revision

Part 3 replaced vague viewpoint v with declaration D:

D = (q, φ, P, Ô_P, Gate_P, TraceRule_P, ResidualRule_P). (1.14)

This means that self-revision can no longer be described vaguely as “changing perspective.”

It must be described as changing one or more components of D.

A self-revision may change:

baseline q. (1.15)

feature map φ. (1.16)

boundary B. (1.17)

observation rule Δ. (1.18)

window h. (1.19)

intervention family u. (1.20)

projection operator Ô. (1.21)

gate rule. (1.22)

trace rule. (1.23)

residual rule. (1.24)

So:

SelfRevision = controlled transformation of D. (1.25)

The declaration is the object being revised.

1.4 The self-revising cycle

At episode k, the system has declaration D_k.

It declares a field:

$\Sigma_k = \text{Declare}(\Sigma_0 \mid D_k)$. (1.26)

It projects visible structure:

$V_k = \hat{O}_k(\Sigma_k)$. (1.27)

It gates projection into a committed tick:

$\tau_k = \text{Gate}_k(V_k, L_k, R_k)$. (1.28)

It updates ledger:

$L_{k+1} = \text{UpdateTrace}_k(L_k, \tau_k)$. (1.29)

It updates residual:

$R_{k+1} = \text{Residual}_k(\Sigma_k, V_k, \tau_k, L_{k+1})$. (1.30)

It revises declaration:

$D_{k+1} = U_a(D_k, L_{k+1}, R_{k+1})$. (1.31)

This gives the cycle:

$D_k \to \Sigma_k \to V_k \to \tau_k \to (L_{k+1}, R_{k+1}) \to D_{k+1}$. (1.32)

This is the basic unit of Part 4.

1.5 The difference between adaptation and self-revision

Adaptation changes behavior.

Self-revision changes declaration.

This difference matters.

A thermostat adapts behavior when temperature changes, but it does not necessarily revise what temperature means, what feature map to use, what boundary to consider, or what gate commits a state.

A mature institution may revise not only its decisions but its evidence rule, legal category, appeal

process, and jurisdictional boundary.

A mature AI runtime may revise not only its answer but its retrieval strategy, verifier threshold, residual ledger, and artifact schema.

A mature scientific theory may revise not only a prediction but the variables, baseline, measurement protocol, and admissible model class.

Therefore:

Adaptation = change of response under fixed D. (1.33)

SelfRevision = change of D under trace and residual. (1.34)

Observerhood in this framework requires the second.

1.6 Why self-revision creates a fractal

The cycle is self-similar.

A declaration creates closure.

That closure becomes material for the next declaration.

Thus:

$\text{Closure}_k \rightarrow \text{DeclarationInput}_{k+1}$. (1.35)

At a lower level, a projection becomes a trace.

At the next level, the trace becomes evidence for revising the declaration.

At the next level, the revised declaration changes which projections can appear.

This repeated pattern creates a recursive family.

The family of all admissible declaration states reachable from D_0 is:

$\mathfrak{A} = \text{closure}(\{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\})$. (1.36)

If the family is stable, it satisfies:

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (1.37)

This is the Self-Revising Declaration Fractal.

1.7 Why admissibility is the central issue

Without admissibility, the fractal is meaningless.

A declaration could revise itself into anything.

It could erase memory.

It could hide contradiction.

It could redefine success.

It could break objectivity.

It could become unfalsifiable.

So the key problem is not writing $D_{k+1} = U_a(D_k,L_k,R_k)$. That is easy.

The key problem is constraining U_a.

The rule must be:

$U_a : \mathfrak{F}_adm \times Ledger \times Residual \rightarrow \mathfrak{F}_adm.$ (1.38)

This means self-revision begins in admissible declaration space and remains inside admissible declaration space.

So:

Admissible self-revision = self-revision that preserves declaration validity. (1.39)

This is the formal heart of Part 4.

1.8 From observer potential to Ô_self

Part 3 gave observer potential:

O_P = recursive trace-guided projection capacity under P. (1.40)

Part 4 upgrades this to Ô_self:

$\hat{O}_self = Fix(\mathcal{U} \mid \mathfrak{F}_adm)$. (1.41)

This says that mature selfhood is not merely one act of reflection. It is not a single loop. It is a stable admissible revision pattern.

The self is not D_0.

The self is not L_0.

The self is not one projection.

The self is the stable constrained orbit by which D revises through L and R.

Thus:

Self ≠ declaration. (1.42)

Self ≠ ledger. (1.43)

Self ≠ projection. (1.44)

Self = admissible revision attractor of declaration through ledger and residual. (1.45)

1.9 The first statement of the Part 4 thesis

We can now state Part 4's thesis:

A mature observer is a self-revising declaration system whose revisions preserve trace, disclose residual, maintain frame robustness, remain budget-bounded, and avoid degeneracy. (1.46)

Or:

$\hat{O}_{self}$ = stable attractor of trace-preserving admissible declaration revision. (1.47)

This is the mature form of the observer in this tetralogy.

1.10 Summary of Section 1

Part 3 gave:

$Time_P = order(\mathfrak{D}_P(\Sigma_0))$. (1.48)

Part 4 begins from:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (1.49)

Time gives history.

Residual gives pressure.

Admissible revision gives observerhood.

The self-revising cycle is:

$D_k \to \Sigma_k \to V_k \to \tau_k \to (L_{k+1}, R_{k+1}) \to D_{k+1}$. (1.50)

The core observer formula is:

$\hat{O}_{self} = Fix(\mathcal{U} \mid \mathfrak{F}_{adm})$. (1.51)

The next section defines D_k in detail.

---

2. Declaration States D_k

2.1 Why D_k must be explicit

A self-revising system must know what it is revising.

If “viewpoint” remains vague, self-revision becomes vague. The system may say it changed perspective, but no one knows what changed. Was it the boundary? The feature map? The gate? The baseline? The residual rule? The authority structure? The time window?

Part 4 therefore requires an explicit declaration state.

At episode k:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (2.1)

where:

$P_k = (B_k, \Delta_k, h_k, u_k)$. (2.2)

This is the object revised by U_a.

The declaration state is not a metaphysical essence. It is an operational contract.

It says how the field is made readable at episode k.

2.2 Baseline q_k

The baseline q_k declares what counts as background.

Without q_k, structure cannot be distinguished from environment.

q_k answers:

What would happen without maintained order? (2.3)

What counts as normal? (2.4)

What counts as passive background? (2.5)

What counts as deviation? (2.6)

Revision may change q_k when the old baseline becomes outdated.

For example:

A market's normal funding baseline changes after a crisis. (2.7)

An AI system's base reliability estimate changes after repeated verifier failures. (2.8)

A biological system's baseline changes after injury, infection, or adaptation. (2.9)

An institution's baseline changes after legal reform or legitimacy crisis. (2.10)

Thus:

$q_{k+1} = \text{Repair}_q(q_k, L_k, R_k)$. (2.11)

This is one possible revision type.

2.3 Feature map φ_k

The feature map φ_k declares what counts as structure.

$\varphi_k : X \to \mathbb{R}^d$. (2.12)

It determines what the system can see as organized, relevant, measurable, or maintainable.

If φ_k is poor, the system may miss the true structure.

For example:

An AI runtime may measure fluency but not evidence support. (2.13)

A bank may measure price risk but not collateral mobility. (2.14)

A medical system may measure symptoms but not underlying systemic inflammation. (2.15)

An institution may measure procedural compliance but not legitimacy. (2.16)

When residual reveals missing structure, φ_k must be revised.

So:

$\varphi_{k+1} = \text{Repair}_\varphi(\varphi_k, L_k, R_k)$. (2.17)

Feature-map revision is one of the most important forms of learning.

2.4 Protocol P_k

The protocol is:

$P_k = (B_k, \Delta_k, h_k, u_k)$. (2.18)

Each component can be revised.

Boundary:

B_k = what is inside the object. (2.19)

Observation rule:

Δ_k = how the object is sampled, measured, or aggregated. (2.20)

Window:

h_k = time or state horizon of judgment. (2.21)

Intervention family:

u_k = admissible action set. (2.22)

Protocol revision occurs when the object itself was wrongly declared.

Examples:

A liquidity problem thought to be desk-level becomes group-level. (2.23)

A token-level AI problem becomes an episode-level coordination problem. (2.24)

A local biological symptom becomes a systemic disease pattern. (2.25)

A legal dispute becomes a constitutional issue. (2.26)

These are not merely new answers. They are new protocols.

Thus:

$P_{k+1} = \mathrm{RepairP}(P_k, L_k, R_k)$. (2.27)

2.5 Projection operator $\hat{O}_k$

The projection operator selects visible structure from the declared field.

$\Sigma_k = \mathrm{Declare}(\Sigma_0 \mid D_k)$. (2.28)

$V_k = \hat{O}_k(\Sigma_k)$. (2.29)

$\hat{O}_k$ determines how the declared field becomes visible.

Projection may fail if it is unstable, biased, too coarse, too sensitive, or not aligned with φ_k.

Then:

$\hat{O}_{k+1} = \mathrm{Repair}\hat{O}(\hat{O}_k, L_k, R_k)$. (2.30)

In AI, this may mean changing retrieval, reasoning, claim extraction, source selection, or verifier routing.

In science, it may mean changing instruments or models.

In finance, it may mean changing the projection from price to funding, collateral, or legal transferability.

Projection revision is not mere answer correction. It is a change in the way visible structure appears.

2.6 Gate_k

The gate determines whether projected structure becomes committed trace.

$\tau_k = \mathrm{Gate}_k(V_k, L_k, R_k)$. (2.31)

Gate_k includes:

threshold. (2.32)

authority. (2.33)

evidence standard. (2.34)

commitment rule. (2.35)

Gate revision is necessary when the system commits too early, too late, or inconsistently.

Too early:

TooEarlyGate ⇒ false trace. (2.36)

Too late:

TooLateGate ⇒ no usable history. (2.37)

Inconsistent:

EquivalentEvidence ⇒ DifferentGateOutcome. (2.38)

Then:

$\mathrm{Gate}_{k+1} = \mathrm{RepairGate}(\mathrm{Gate}_k, L_k, R_k)$. (2.39)

A mature observer must revise gates carefully because gate changes directly affect what becomes time.

2.7 $\mathrm{TraceRule}_k$

The trace rule determines how committed events enter ledger.

$L_{k+1} = \mathrm{UpdateTrace}_k(L_k, \tau_k)$. (2.40)

A weak trace rule stores only conclusion.

A stronger trace rule stores conclusion, evidence, gate threshold, authority, residual, and revision metadata.

Trace rule revision occurs when history is not accountable enough.

For example:

An AI system logs answers but not sources or verifier decisions. (2.41)

A court records judgment but not reasoning. (2.42)

A financial model records risk status but not stress assumptions. (2.43)

An institution records decisions but not dissent or residual concerns. (2.44)

Then:

$\text{TraceRule}_{k+1} = \text{RepairTrace}(\text{TraceRule}_k, L_k, R_k)$. (2.45)

Trace rule revision protects the system from amnesia.

2.8 ResidualRule_k

The residual rule determines how unresolved remainder is classified and carried.

$R_{k+1} = \text{Residual}_k(\Sigma_k, V_k, \tau_k, L_{k+1})$. (2.46)

Residual may include:

missing evidence. (2.47)

contradiction. (2.48)

model limitation. (2.49)

boundary leakage. (2.50)

frame disagreement. (2.51)

unpaid cost. (2.52)

future option. (2.53)

risk. (2.54)

Residual rule revision occurs when residual is being hidden, misclassified, or ignored.

Then:

$\text{ResidualRule}_{k+1} = \text{RepairResidual}(\text{ResidualRule}_k, L_k, R_k)$. (2.55)

This is central to mature observerhood.

A system that cannot classify residual cannot learn from its own incompleteness.

2.9 D_k as a structured contract

We can now see that D_k is a structured contract.

It declares:

what the world is measured against. (2.56)

what features count as structure. (2.57)

what object is inside. (2.58)

how the object is observed. (2.59)

what time window matters. (2.60)

what interventions are allowed. (2.61)

how visibility is projected. (2.62)

when visibility becomes commitment. (2.63)

how commitment enters ledger. (2.64)

how unresolved remainder is carried. (2.65)

Thus:

D_k = declared world-making contract at episode k. (2.66)

Self-revision is revision of this contract.

2.10 What a revision may change

A revision operator U_a may change one component or several components.

Possible revision classes include:

Repairq. (2.67)

Repairφ. (2.68)

RepairB. (2.69)

RepairΔ. (2.70)

Repairh. (2.71)

Repairu. (2.72)

RepairÔ. (2.73)

RepairGate. (2.74)

RepairTrace. (2.75)

RepairResidual. (2.76)

So:

$\text{RevisionType}_a \in \{\text{RepairB}, \text{RepairΔ}, \text{Repairh}, \text{Repairu}, \text{Repairq}, \text{Repairφ}, \text{RepairÔ}, \text{RepairGate}, \text{RepairTrace}, \text{RepairResidual}\}$. (2.77)

A complex revision may compose several repairs:

$U_a = \text{RepairGate} \circ \text{Repairφ} \circ \text{RepairResidual}$. (2.78)

But any revision must remain admissible.

2.11 The danger of silent revision

A system may silently change D without admitting it.

This produces ontology drift.

For example:

The system claims to evaluate evidence, then silently changes what counts as evidence. (2.79)

The system claims to measure stability, then silently changes the time window. (2.80)

The system claims to test one object, then silently changes boundary. (2.81)

The system claims to preserve residual, then silently reclassifies residual as irrelevant. (2.82)

This is inadmissible.

SilentDeclarationChange ⇒ OntologyDrift. (2.83)

OntologyDrift ⇒ inadmissible, unless explicitly gated and traced. (2.84)

Therefore, every self-revision must itself be traceable.

Revision must leave trace.

So:

$D_k \rightarrow D_{k+1}$ must be ledgered. (2.85)

2.12 Summary of Section 2

The declaration state is:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (2.86)

with:

$P_k = (B_k, \Delta_k, h_k, u_k)$. (2.87)

It produces:

$\Sigma_k = Declare(\Sigma_0 \mid D_k)$. (2.88)

$V_k = \hat{O}_k(\Sigma_k)$. (2.89)

$\tau_k = Gate_k(V_k, L_k, R_k)$. (2.90)

$L_{k+1} = UpdateTrace_k(L_k, \tau_k)$. (2.91)

$R_{k+1} = Residual_k(\Sigma_k, V_k, \tau_k, L_{k+1})$. (2.92)

A self-revision changes D_k:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (2.93)

The next section defines U_a.

3. The Self-Revision Operator U_a

3.1 Why U_a is needed

Part 4 now has the declaration state:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (3.1)

and the declared disclosure cycle:

$D_k \rightarrow \Sigma_k \rightarrow V_k \rightarrow \tau_k \rightarrow L_{k+1} + R_{k+1}$. (3.2)

But this is still only a declared-time system.

To become observer-like, the system must use its ledger and residual to revise the next declaration:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (3.3)

The operator U_a is the self-revision operator.

It is indexed by a because there may be many admissible revision types:

$U_a \in \mathcal{U}$. (3.4)

where:

$\mathcal{U} = \{U_a \mid a \in A_adm\}$. (3.5)

Part 4 is not saying that there is only one revision operator. It is saying there is a family of admissible revision operators.

3.2 What U_a does

The revision operator U_a takes three inputs:

current declaration D_k. (3.6)

ledger L_k. (3.7)

residual R_k. (3.8)

and produces a next declaration:

D_{k+1}. (3.9)

So:

U_a : Declaration × Ledger × Residual → Declaration. (3.10)

But for admissible observerhood, the correct form is stricter:

$U_a : \mathfrak{F}_adm \times Ledger \times Residual \rightarrow \mathfrak{F}_adm$. (3.11)

This means U_a does not merely produce any next declaration. It must produce an admissible next declaration.

Thus:

$D_k \in \mathfrak{F}_adm \Rightarrow U_a(D_k,L_k,R_k) \in \mathfrak{F}_adm.$ (3.12)

This is the closure rule.

Without it, self-revision becomes arbitrary rule-changing.

3.3 U_a as repair operator

Most self-revision begins as repair.

Residual reveals that the current declaration is insufficient.

A revision operator may repair one part of D_k.

For example:

Repairq revises baseline. (3.13)

Repairφ revises feature map. (3.14)

RepairB revises boundary. (3.15)

RepairΔ revises observation rule. (3.16)

Repairh revises time or state window. (3.17)

Repairu revises admissible intervention family. (3.18)

RepairÔ revises projection operator. (3.19)

RepairGate revises commitment rule. (3.20)

RepairTrace revises trace rule. (3.21)

RepairResidual revises residual rule. (3.22)

Therefore:

$\text{RevisionType}_a \in$ {Repairq, Repairφ, RepairB, RepairΔ, Repairh, Repairu, RepairÔ, RepairGate, RepairTrace, RepairResidual}. (3.23)

A simple U_a may repair only one component.

A complex U_a may repair several.

For example:

U_a = RepairGate. (3.24)

or:

U_a = RepairResidual ∘ RepairTrace ∘ RepairGate. (3.25)

or:

U_a = RepairP ∘ Repairφ ∘ Repairq. (3.26)

The operator family allows flexible but constrained self-revision.

3.4 U_a as residual response

Residual is the primary signal for U_a.

When R_k is small and stable, D_k may remain valid.

When R_k rises, the declaration may require repair.

In compact form:

$\text{HighResidual}(D_k) \Rightarrow \text{Apply } U_a.$ (3.27)

But the type of residual matters.

Different residuals call different repairs.

Boundary residual suggests:

$U_a = \text{RepairB}.$ (3.28)

Feature residual suggests:

$U_a = \text{Repair}\varphi.$ (3.29)

Baseline residual suggests:

$U_a = \text{Repair}q.$ (3.30)

Gate residual suggests:

$U_a = \text{RepairGate}.$ (3.31)

Trace residual suggests:

$U_a = \text{RepairTrace}.$ (3.32)

Residual-classification failure suggests:

$U_a = \text{RepairResidual}.$ (3.33)

Thus, residual must be typed.

A mature system should not simply say:

Residual is high. (3.34)

It should ask:

What kind of residual is high? (3.35)

Then choose U_a accordingly.

3.5 U_a as trace response

Ledger also guides revision.

A single residual may be noise. But repeated trace patterns reveal structural failure.

For example:

Repeated verifier failures may indicate weak Gate. (3.36)

Repeated unsupported claims may indicate poor φ or Ô. (3.37)

Repeated prompt fragility may indicate weak frame robustness. (3.38)

Repeated late commitments may indicate Gate too strict. (3.39)

Repeated residual concealment may indicate weak ResidualRule. (3.40)

Repeated budget overrun may indicate unsustainable declaration. (3.41)

Thus:

$U_a(D_k,L_k,R_k)$ should depend on both current residual and ledgered pattern. (3.42)

Residual is local pressure.

Ledger is historical evidence.

A mature self-revision operator uses both.

3.6 U_a must be traceable

Revision itself must enter ledger.

If D_k changes to D_{k+1}, the system must record:

what changed. (3.43)

why it changed. (3.44)

what residual triggered it. (3.45)

what evidence supported it. (3.46)

what invariants were preserved. (3.47)

what risks were introduced. (3.48)

what test will validate it. (3.49)

Therefore:

$\text{RevisionTrace}_k = \text{Record}(D_k \rightarrow D_{k+1}, \text{Reason}_k, \text{Residual}_k, \text{Evidence}_k, \text{Invariant}_k, \text{Test}_k)$. (3.50)

and:

$L_{k+1} = \text{UpdateTrace}(L_k, \text{RevisionTrace}_k)$. (3.51)

This prevents silent declaration drift.

A system that revises itself without trace is not a mature observer.

It is opaque self-modification.

3.7 U_a and admissibility gates

A revision operator must pass admissibility gates.

Before accepting:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (3.52)

the system must test:

$\mathrm{WellFormed}(D_{k+1})$. (3.53)

$\mathrm{TracePreserving}(D_k \to D_{k+1})$. (3.54)

$\mathrm{ResidualHonest}(D_k \to D_{k+1})$. (3.55)

$\mathrm{FrameRobust}(D_{k+1})$. (3.56)

$\mathrm{BudgetBounded}(D_{k+1})$. (3.57)

$\mathrm{NonDegenerate}(D_{k+1})$. (3.58)

So a stricter expression is:

$D_{k+1} = \mathrm{Accept}(U_a(D_k, L_k, R_k))$ only if $D_{k+1} \in \mathfrak{F}_\mathrm{adm}$. (3.59)

If the candidate fails, it must be rejected or quarantined:

$D_\mathrm{candidate} \notin \mathfrak{F}_\mathrm{adm} \Rightarrow \mathrm{Reject}(D_\mathrm{candidate}) \vee \mathrm{Quarantine}(D_\mathrm{candidate})$. (3.60)

This is how the system avoids pathological self-modification.

3.8 U_a and local versus switch revision

Some revisions are local.

For example:

adjusting a gate threshold. (3.61)

adding a residual category. (3.62)

expanding a feature map. (3.63)

slightly changing a time window. (3.64)

Other revisions are structural switches.

For example:

changing the object boundary. (3.65)

changing the baseline regime. (3.66)

moving from token-time to episode-time. (3.67)

reclassifying a system from price-risk to funding-risk. (3.68)

changing from answer-generation to artifact-governance. (3.69)

These larger changes require switch gates.

Local revision:

$d_{\mathfrak{F}}(D_{k+1}, D_k) \leq \varepsilon_D$. (3.70)

Switch revision:

$d_{\mathfrak{F}}(D_{k+1}, D_k) > \varepsilon_D$. (3.71)

If switch revision occurs:

$d_{\mathfrak{F}}(D_{k+1}, D_k) > \varepsilon_D \Rightarrow \text{SwitchGate}_k = \text{Open}$. (3.72)

This will be developed later in Section 11.

3.9 U_a and exploration

Not all revisions must immediately improve the system.

Sometimes exploration is necessary.

A system may test a new feature map, boundary, or gate rule to see if residual can be reduced. But exploration must be budgeted and traced.

Exploration revision satisfies:

$\mathcal{V}(D_{k+1}) > \mathcal{V}(D_k)$. (3.73)

This may be allowed if:

$\text{ExplorationBudget}_k \geq \mathcal{V}(D_{k+1}) - \mathcal{V}(D_k)$. (3.74)

Otherwise:

DestabilizationWithoutBudget ⇒ inadmissible. (3.75)

This prevents the system from calling every unstable revision “learning.”

3.10 U_a and the emergence of observerhood

A system with no U_a can still have time.

It can declare, project, gate, trace, and ledger.

But it cannot revise its declaration.

So it has:

Time_P. (3.76)

but not:

Ô_self. (3.77)

A system with U_a but no admissibility can self-modify, but pathologically.

It has:

SelfModification. (3.78)

but not mature observerhood.

A system with admissible U_a has:

$\hat{O}_{self} = Fix(\mathcal{U} \mid \mathfrak{F}_{adm})$. (3.79)

Thus:

$Time_P + U_a + \mathfrak{F}_{adm} \Rightarrow \hat{O}_{self}$. (3.80)

This is the Part 4 bridge.

3.11 Summary of Section 3

The self-revision operator is:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (3.81)

The admissible form is:

$U_a : \mathfrak{F}_{adm} \times Ledger \times Residual \to \mathfrak{F}_{adm}$. (3.82)

The operator family is:

$\mathcal{U} = \{U_a \mid a \in A_{adm}\}$. (3.83)

The revision types are:

$RevisionType_a \in \{Repairq, Repair\varphi, RepairB, Repair\Delta, Repairh, Repairu, Repair\hat{O}, RepairGate, RepairTrace, RepairResidual\}$. (3.84)

A revision must be traceable:

$RevisionTrace_k = Record(D_k \to D_{k+1}, Reason_k, Residual_k, Evidence_k, Invariant_k, Test_k)$. (3.85)

And the closure rule is:

$\forall D \in \mathfrak{F}_{adm}, U_a(D,L,R) \in \mathfrak{F}_{adm}$. (3.86)

The next section defines $\mathfrak{F}_{adm}$.

---

4. The Admissible Declaration Family $\mathfrak{F}_{adm}$

4.1 Why a declaration family is needed

The central danger of Part 4 is arbitrariness.

If every declaration revision is allowed, then a system can always escape failure by changing its own rules.

It can revise q so that deviations disappear.

It can revise φ so that contradictions are no longer features.

It can revise B so that failed cases fall outside the object.

It can revise Gate so that every output passes.

It can revise ResidualRule so that residual is hidden.

It can revise invariance tests so that frame fragility is ignored.

This would make self-revision meaningless.

Therefore, self-revision must occur inside an admissible declaration family.

The family is:

$\mathfrak{F}_{adm} = \{D \mid D \text{ satisfies admissibility constraints}\}$. (4.1)

Part 4 proposes the following constraint bundle:

$\mathfrak{F}_{adm} = \{D \mid \text{WellFormed}(D) \wedge \text{TracePreserving}(D) \wedge \text{ResidualHonest}(D) \wedge \text{FrameRobust}(D) \wedge \text{BudgetBounded}(D) \wedge \text{NonDegenerate}(D)\}$. (4.2)

4.2 What the six constraints prevent

Each constraint prevents a specific pathology.

WellFormed prevents vagueness. (4.3)

TracePreserving prevents amnesia. (4.4)

ResidualHonest prevents concealment. (4.5)

FrameRobust prevents frame manipulation. (4.6)

BudgetBounded prevents unsustainable declaration. (4.7)

NonDegenerate prevents emptiness, totality, and unfalsifiability. (4.8)

Together, they prevent arbitrary self-modification.

A declaration is admissible only if all six constraints hold.

So:

$D \in \mathfrak{F}_{adm} \Leftrightarrow WF(D) \wedge TP(D) \wedge RH(D) \wedge FR(D) \wedge BB(D) \wedge ND(D)$. (4.9)

where:

WF = WellFormed. (4.10)

TP = TracePreserving. (4.11)

RH = ResidualHonest. (4.12)

FR = FrameRobust. (4.13)

BB = BudgetBounded. (4.14)

ND = NonDegenerate. (4.15)

4.3 Well-formedness

Well-formedness asks whether the declaration contains the necessary components.

D must include:

q. (4.16)

φ. (4.17)

P. (4.18)

Ô. (4.19)

Gate. (4.20)

TraceRule. (4.21)

ResidualRule. (4.22)

and P must include:

B. (4.23)

Δ. (4.24)

h. (4.25)

u. (4.26)

Thus:

WellFormed(D) ⇔ q exists ∧ φ exists ∧ P exists ∧ Ô exists ∧ Gate exists ∧ TraceRule exists ∧ ResidualRule exists. (4.27)

and:

P = (B, Δ, h, u). (4.28)

If any component is missing, the declaration is not admissible.

No q, no environment. (4.29)

No φ, no structure. (4.30)

No P, no object. (4.31)

No Gate, no commitment. (4.32)

No TraceRule, no historical order. (4.33)

No ResidualRule, hidden remainder. (4.34)

4.4 Trace preservation

Trace preservation asks whether the system preserves ledger accountability across revision.

A revision may reinterpret its past. It may correct its past. It may reclassify prior events. But it must not erase its past without trace.

Let:

$T_{k \to k+1} : L_k \to L_{k+1}$. (4.35)

be a trace transport map.

Trace preservation requires:

$\|\mathrm{Inv}(T_k \to_{k+1}(L_k)) - \mathrm{Inv}(L_k)\| \le \varepsilon_\mathrm{inv}$. (4.36)

This means important ledger invariants survive the revision.

If the system revises itself by forgetting why it revised, the revision is inadmissible.

Thus:

$\mathrm{RevisionByAmnesia} \Rightarrow D_{k+1} \notin \mathfrak{F}_\mathrm{adm}$. (4.37)

Trace preservation is the no-amnesia rule.

4.5 Residual honesty

Residual honesty asks whether the system carries unresolved remainder rather than hiding it.

Residual is not always error. It may be ambiguity, missing evidence, model limitation, contradiction, hidden coupling, boundary leakage, or future option value.

A mature declaration carries residual as a governed object.

After revision:

$R_{k+1} = \mathrm{Unexplained}(T_k \to_{k+1}(R_k)) \sqcup \mathrm{NewResidual}_k$. (4.38)

Residual honesty requires:

$\mathrm{HiddenResidual}(D_{k+1}) \le \mathrm{HiddenResidual}(D_k) + \varepsilon_R$. (4.39)

This does not require residual to always shrink.

A better declaration may reveal more residual.

The rule only forbids concealment.

So:

$\mathrm{ResidualRepair} \neq \mathrm{ResidualConcealment}$. (4.40)

Residual honesty is the no-concealment rule.

4.6 Frame robustness

Frame robustness asks whether equivalent frames produce equivalent governed outcomes.

If:

$\mathrm{frame_i} \equiv_D \mathrm{frame_j}$. (4.41)

then:

$\mathrm{Dist}(\mathrm{Response_D(frame_i)}, \mathrm{Response_D(frame_j)}) \le \varepsilon_\mathrm{frame}$. (4.42)

A declaration that fails this test is fragile.

It cannot tell whether a change is real or merely representational.

Frame robustness is necessary for objectivity.

Without it, a self-revising system can manipulate reality by changing wording, category, or frame.

Thus:

FrameFragile(D) ⇒ D ∉ 𝔉_adm, unless explicitly quarantined. (4.43)

Frame robustness is the no-frame-trick rule.

4.7 Budget boundedness

Budget boundedness asks whether the declaration is viable.

A declaration may be well-formed, trace-preserving, and residual-honest, yet too costly to maintain.

The basic bounds are:

Γ_loss(D) ≤ Γ*. (4.44)

G_gap(D) ≤ G*. (4.45)

κ(D) ≤ κ*. (4.46)

where:

Γ_loss = dissipation or loss. (4.47)

G_gap = health gap between drive and structure. (4.48)

κ = conditioning, curvature, or brittleness measure. (4.49)

A declaration that exceeds these bounds may be theoretically interesting but operationally unstable.

Thus:

BudgetFailure(D) ⇒ D ∉ 𝔉_adm, unless under recovery protocol. (4.50)

Budget boundedness is the no-unsustainable-world rule.

4.8 Non-degeneracy

Non-degeneracy asks whether the declaration remains informative, specific, and falsifiable.

The declaration must not become empty:

D_empty = nothing counts. (4.51)

It must not become total:

D_total = everything confirms me. (4.52)

It must not become unfalsifiable:

Falsifiable(D) = false. (4.53)

A non-degenerate declaration satisfies:

$I_{min} \le Info(D) \le I_{max}$. (4.54)

$0 < Specificity(D) < \infty$. (4.55)

Falsifiable(D) = true. (4.56)

So:

$NonDegenerate(D) \Leftrightarrow informative \wedge specific \wedge falsifiable$. (4.57)

Non-degeneracy is the no-vacuity and no-dogma rule.

4.9 The closure condition

The declaration family is useful only if revision remains inside it.

The closure condition is:

$\forall D \in \mathfrak{F}_{adm}, U_a(D,L,R) \in \mathfrak{F}_{adm}$. (4.58)

This is stronger than merely saying each declaration is valid.

It says the revision process preserves validity.

A system with $D_k \in \mathfrak{F}_{adm}$ but $D_{k+1} \notin \mathfrak{F}_{adm}$ has fallen out of mature observerhood.

So:

$D_k \in \mathfrak{F}_{adm} \wedge D_{k+1} \notin \mathfrak{F}_{adm} \Rightarrow SelfRevisionFailure$. (4.59)

The closure condition is the foundation of the fractal family.

4.10 Admissibility prevents relativism

A common worry is that declaration-based theory becomes relativistic.

Part 4 answers:

Only admissible declarations count. (4.60)

Only trace-preserving revisions count. (4.61)

Only residual-honest revisions count. (4.62)

Only frame-robust revisions count. (4.63)

Only budget-bounded revisions count. (4.64)

Only non-degenerate revisions count. (4.65)

Therefore, the theory does not say:

Any declaration creates truth. (4.66)

It says:

Admissible declarations produce valid disclosure. (4.67)

and:

Recursive objectivity is what survives admissible self-revision. (4.68)

This is a disciplined, not relativistic, position.

4.11 Summary of Section 4

The admissible declaration family is:

$\mathfrak{F}_{adm} = \{D \mid WellFormed(D) \wedge TracePreserving(D) \wedge ResidualHonest(D) \wedge FrameRobust(D) \wedge BudgetBounded(D) \wedge NonDegenerate(D)\}$. (4.69)

A revision is admissible only if it preserves membership in $\mathfrak{F}_{adm}$:

$U_a : \mathfrak{F}_{adm} \times Ledger \times Residual \to \mathfrak{F}_{adm}$. (4.70)

The closure condition is:

$\forall D \in \mathfrak{F}_{adm}, U_a(D,L,R) \in \mathfrak{F}_{adm}$. (4.71)

The next section develops the first constraint: well-formedness.

---

5. Well-Formedness: No Declaration Without Structure

5.1 The first admissibility gate

Well-formedness is the first gate into $\mathfrak{F}_{adm}$.

A declaration cannot be admissible if it does not specify what it is.

The minimum declaration state is:

$D = (q, \varphi, P, \hat{O}, Gate, TraceRule, ResidualRule)$. (5.1)

and:

$P = (B, \Delta, h, u)$. (5.2)

Therefore:

$WellFormed(D) \Leftrightarrow q \text{ exists} \wedge \varphi \text{ exists} \wedge P \text{ exists} \wedge \hat{O} \text{ exists} \wedge Gate \text{ exists} \wedge TraceRule \text{ exists} \wedge ResidualRule \text{ exists}$. (5.3)

This may look simple, but it is powerful. Many theoretical and practical failures occur because one of these components is hidden, missing, or silently changed.

Well-formedness forces the system to publish its declaration.

5.2 No q, no environment

The baseline q declares the environment.

Without q, the system cannot distinguish structure from background.

For example:

A market spread is high only relative to a baseline. (5.4)

An AI answer is unreliable only relative to a baseline expectation of support. (5.5)

A biological marker is abnormal only relative to a physiological baseline. (5.6)

An institutional process is delayed only relative to a procedural baseline. (5.7)

So:

No q ⇒ NoStableEnvironment. (5.8)

and:

NoStableEnvironment ⇒ NoStableDeviation. (5.9)

A declaration without q cannot say what counts as meaningful deviation.

Thus:

q is required for WellFormed(D). (5.10)

5.3 No φ, no structure

The feature map φ declares what counts as structure.

Without φ, the system cannot say what it is extracting, maintaining, measuring, or repairing.

A language model may produce fluent text, but if φ does not measure evidence support, the system cannot see unsupported claims.

A financial dashboard may show price movement, but if φ does not measure funding lock-in, the system cannot see funding stress.

A medical test may measure one biomarker, but if φ does not represent system-level inflammation, the relevant structure remains invisible.

Thus:

No φ ⇒ NoDeclaredStructure. (5.11)

and:

NoDeclaredStructure ⇒ NoValidProjection. (5.12)

So φ is required for WellFormed(D).

5.4 No P, no object

The protocol P declares the object.

P = (B, Δ, h, u). (5.13)

Without P, the claim has no stable object.

Boundary B says what is inside.

Observation rule Δ says how it is seen.

Window h says over what horizon it matters.

Intervention family u says what actions are admissible.

Without P, a statement such as “the system is stable” floats.

Stable under which boundary? (5.14)

Measured how? (5.15)

Over what horizon? (5.16)

Relative to which interventions? (5.17)

Therefore:

No P ⇒ NoStableObject. (5.18)

and:

NoStableObject ⇒ NoAdmissibleDeclaration. (5.19)

So P is required for WellFormed(D).

5.5 No Ô, no visibility

The projection operator Ô determines how the declared field becomes visible.

Without Ô, the field may be declared but not projected.

Σ_D = Declare(Σ0 | D). (5.20)

V_D = Ô_D(Σ_D). (5.21)

If Ô is missing, there is no declared visibility.

Thus:

No Ô ⇒ NoProjection. (5.22)

NoProjection ⇒ NoCandidateTrace. (5.23)

A declaration that cannot project visible structure cannot generate a ledger.

So Ô is required for WellFormed(D).

5.6 No Gate, no commitment

The gate determines whether visible structure becomes committed trace.

$\tau_k = \mathrm{Gate}_k(V_k, L_k, R_k)$. (5.24)

Without Gate, every projection may enter ledger, or none may enter ledger.

Both are failures.

If everything enters:

NoGate ⇒ TracePollution. (5.25)

If nothing enters:

NoGate ⇒ LedgerParalysis. (5.26)

A gate must specify threshold, authority, and commitment rule.

Thus:

No Gate ⇒ NoValidCommitment. (5.27)

NoValidCommitment ⇒ NoValidTime. (5.28)

So Gate is required for WellFormed(D).

5.7 No TraceRule, no historical order

TraceRule determines how committed events enter ledger.

Without TraceRule, the system may commit events but not preserve them in accountable historical order.

A trace rule should specify:

what is recorded. (5.29)

how it is ordered. (5.30)

what metadata is attached. (5.31)

what residual is linked. (5.32)

how future projection is affected. (5.33)

Without TraceRule:

No TraceRule ⇒ NoStableLedger. (5.34)

NoStableLedger ⇒ NoStableTime. (5.35)

Therefore TraceRule is required for WellFormed(D).

5.8 No ResidualRule, hidden remainder

ResidualRule determines how unresolved remainder is classified, carried, and revisited.

Without ResidualRule, closure will hide what it excludes.

A residual rule should specify:

residual type. (5.36)

scope. (5.37)

severity. (5.38)

owner or handler. (5.39)

trigger for revisit. (5.40)

link to trace. (5.41)

Without ResidualRule:

No ResidualRule ⇒ HiddenResidual. (5.42)

HiddenResidual ⇒ FalseClosure. (5.43)

FalseClosure ⇒ PathologicalRevisionRisk. (5.44)

Therefore ResidualRule is required for WellFormed(D).

5.9 Well-formedness of revision

Not only declarations, but revisions must be well-formed.

A revision should declare what it changes.

Let:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (5.45)

and:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (5.46)

Then the revision should identify:

$Changed(D_k, D_{k+1})$. (5.47)

For example:

$Changed(D_k, D_{k+1}) = \{\varphi, Gate, ResidualRule\}$. (5.48)

A well-formed revision records:

what changed. (5.49)

why it changed. (5.50)

which residual triggered it. (5.51)

which ledger evidence supported it. (5.52)

which invariants were preserved. (5.53)

which test will validate it. (5.54)

Thus:

$WellFormedRevision(D_k \to D_{k+1}) \Rightarrow$ RevisionTrace exists. (5.55)

5.10 Well-formedness does not guarantee truth

Well-formedness is necessary but not sufficient.

A declaration may specify all components and still be wrong, fragile, costly, or degenerate.

Thus:

$\mathrm{WellFormed}(D) \not\Rightarrow \mathrm{True}(D)$. (5.56)

$\mathrm{WellFormed}(D) \not\Rightarrow \mathrm{FrameRobust}(D)$. (5.57)

$\mathrm{WellFormed}(D) \not\Rightarrow \mathrm{BudgetBounded}(D)$. (5.58)

$\mathrm{WellFormed}(D) \not\Rightarrow \mathrm{NonDegenerate}(D)$. (5.59)

Well-formedness only means the declaration is explicit enough to be tested.

It is the first gate, not the final gate.

5.11 Well-formedness and scientific discipline

Well-formedness is also the condition of critique.

A vague declaration cannot be criticized precisely because its hidden assumptions can always move.

A well-formed declaration can be tested.

If q is wrong, revise q.

If φ misses structure, revise φ.

If B leaks, revise B.

If Δ is unstable, revise Δ.

If h is wrong, revise h.

If u omits a real intervention, revise u.

If Gate is too loose, repair Gate.

If TraceRule is weak, repair TraceRule.

If ResidualRule hides uncertainty, repair ResidualRule.

Thus:

$\mathrm{WellFormed}(D) \Rightarrow \mathrm{Repairable}(D)$. (5.60)

$\mathrm{Vague}(D) \Rightarrow \mathrm{Unrepairable}(D)$. (5.61)

This is why well-formedness is the first step toward mature observerhood.

5.12 Summary of Section 5

The well-formed declaration is:

$D = (q, \varphi, P, \hat{O}, Gate, TraceRule, ResidualRule)$. (5.62)

with:

$P = (B, \Delta, h, u)$. (5.63)

The well-formedness condition is:

$WellFormed(D) \Leftrightarrow q \text{ exists} \wedge \varphi \text{ exists} \wedge P \text{ exists} \wedge \hat{O} \text{ exists} \wedge Gate \text{ exists} \wedge TraceRule \text{ exists} \wedge ResidualRule \text{ exists}$. (5.64)

The discipline is:

No q, no environment. (5.65)

No φ, no structure. (5.66)

No P, no object. (5.67)

No Ô, no visibility. (5.68)

No Gate, no commitment. (5.69)

No TraceRule, no historical order. (5.70)

No ResidualRule, hidden remainder. (5.71)

Well-formedness does not guarantee truth. It makes truth claims testable.

The next section develops trace preservation: the no-amnesia rule.

6. Trace Preservation: The No-Amnesia Rule

6.1 Why trace preservation is necessary

A self-revising declaration must be able to change.

But it must not change by forgetting.

If a system can revise its own declaration by erasing the ledger that justified the revision, then it is not learning. It is resetting. It may look flexible, but it has lost accountability.

A mature observer must preserve the trace of its own transformation.

So the rule is:

Revision must not erase the ledger that made revision necessary. (6.1)

This is the no-amnesia rule.

A declaration may reinterpret past trace.

A declaration may correct past trace.

A declaration may downgrade or reclassify past trace.

But it must not silently delete the accountability of past trace.

Thus:

TracePreserving($D_k \to D_{k+1}$) ⇔ ledger accountability survives revision. (6.2)

Without this, self-revision becomes ungrounded.

6.2 Ledger transport

When a declaration changes, the old ledger must be transported into the new declaration.

Let:

$T_{k \to k+1} : L_k \to L_{k+1}$. (6.3)

This is a trace transport map.

It does not mean every old record remains identical. Some old records may be reclassified. Some may be marked obsolete. Some may be corrected. Some may be linked to new residual categories.

But the transport must preserve accountable invariants.

The core condition is:

$\|Inv(T_{k \to k+1}(L_k)) - Inv(L_k)\| \leq \varepsilon_inv$. (6.4)

Here Inv means the ledger invariants that must survive revision.

These may include:

event identity. (6.5)

commitment authority. (6.6)

evidence link. (6.7)

gate threshold. (6.8)

residual attachment. (6.9)

revision reason. (6.10)

causal dependency. (6.11)

falsification status. (6.12)

The exact invariant family depends on the domain. But some invariant family must be declared.

No invariant, no accountable revision. (6.13)

6.3 Trace preservation is not trace rigidity

Trace preservation does not mean the past cannot be reinterpreted.

A mature observer must be able to revise its understanding of the past.

For example:

An AI runtime may reclassify an earlier answer from “verified” to “unsupported.” (6.14)

A court may distinguish an earlier precedent. (6.15)

A financial institution may reclassify an exposure after discovering hidden collateral constraints. (6.16)

A scientific theory may reinterpret an anomaly as a new signal. (6.17)

A biological system may update immune memory after new exposure. (6.18)

These are legitimate revisions.

The no-amnesia rule does not prohibit reinterpretation.

It prohibits silent erasure.

So:

TracePreservation ≠ frozen history. (6.19)

TracePreservation = accountable history under revision. (6.20)

A healthy system can say:

We previously committed τ_k under D_k; under D_{k+1} it is reclassified as τ'_k, with residual R'_k and preserved evidence link E_k. (6.21)

This is trace-preserving reinterpretation.

6.4 Revision trace

Every declaration revision should itself become trace.

If:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (6.22)

then the system should record:

$\text{RevisionTrace}_k = \text{Record}(D_k \to D_{k+1}, U_a, \text{Reason}_k, \text{Evidence}_k, \text{Residual}_k, \text{Invariants}_k, \text{Test}_k)$. (6.23)

Then:

$L_{k+1} = \text{UpdateTrace}(L_k, \text{RevisionTrace}_k)$. (6.24)

The revision trace should include:

old declaration D_k. (6.25)

new declaration D_{k+1}. (6.26)

revision operator U_a. (6.27)

trigger residual R_k. (6.28)

ledger evidence L_k used. (6.29)

invariants preserved. (6.30)

invariants changed. (6.31)

quarantine or test conditions. (6.32)

expected improvement. (6.33)

future review trigger. (6.34)

This makes self-revision auditable.

Without revision trace:

SelfRevision ⇒ hidden ontology drift. (6.35)

With revision trace:

SelfRevision ⇒ accountable observer update. (6.36)

6.5 The amnesia failure

Amnesia occurs when a system revises itself by erasing the history that created the need for revision.

In formula form:

Amnesia ⇔ $D_{k+1} = U_a(D_k, L_k, R_k)$ and $T_{k \to k+1}(L_k)$ is undefined or loses required invariants. (6.37)

A simpler expression is:

Amnesia ⇔ D_{k+1} erases L_k. (6.38)

This is inadmissible.

RevisionByAmnesia ⇒ $D_{k+1} \notin \mathfrak{F}_adm$. (6.39)

Amnesia can look efficient. It clears contradiction. It reduces residual temporarily. It makes the system appear fresh.

But this is false cleanliness.

The residual has not been resolved. It has been buried.

So:

Amnesia reduces visible residual by destroying trace. (6.40)

This is not learning.

It is ledger damage.

6.6 Trace preservation and identity

Trace preservation is also a condition of identity.

If the system revises itself so radically that no trace continuity remains, then it is unclear whether it is the same observer.

A self must be able to change while preserving enough continuity.

Thus:

SelfContinuity requires TraceContinuity. (6.41)

This does not require all contents to remain fixed. But some invariant structure must survive.

Possible continuity invariants include:

core boundary of the observer. (6.42)

ledger identity. (6.43)

revision accountability. (6.44)

residual lineage. (6.45)

admissibility constraints. (6.46)

invariance tests. (6.47)

ethical or governance constraints. (6.48)

Without such continuity:

D_{k+1} may be a new system, not a revised self. (6.49)

Therefore:

SelfRevision requires identity-through-trace. (6.50)

6.7 Trace preservation and recursive objectivity

Part 4 will later define recursive objectivity as invariance across the admissible revision orbit.

But recursive objectivity requires trace transport.

If D_k and D_{k+1} cannot transport ledger relations, then no relation can be tested across revision.

So:

RecursiveObjectivity requires $T_{k \to k+1}$. (6.51)

and:

$T_{k \to k+1}$ requires preserved invariants. (6.52)

If trace transport fails, then each revision creates a disconnected world. There is no mature observer, only a sequence of unrelated declarations.

Thus:

NoTraceTransport ⇒ NoRecursiveObjectivity. (6.53)

This is why trace preservation is not merely archival. It is the backbone of objectivity over self-revision.

6.8 Trace preservation and residual lineage

Residual must also have lineage.

Suppose R_k contains unresolved contradiction. If D_{k+1} revises φ or Gate, the residual may change form. But it should still be possible to say what happened to the old residual.

The residual lineage condition is:

$R_{k+1} = \text{Unexplained}(T_{k \to k+1}(R_k)) \sqcup \text{NewResidual}_k$. (6.54)

This means the new residual includes the old residual that remains unexplained, transported into the new declaration, plus any new residual revealed by the revision.

Residual lineage prevents hidden burial.

If old residual disappears, the system must explain why:

$\text{Resolved}(R_k)$. (6.55)

or:

$\text{Reclassified}(R_k)$. (6.56)

or:

$\text{OutOfScope}(R_k)$ with declared boundary change. (6.57)

If none of these is recorded, then the disappearance is suspicious.

Thus:

ResidualDisappearance without trace ⇒ ResidualConcealment. (6.58)

6.9 Trace preservation as a gate

Trace preservation should itself be a gate on revision.

Before accepting D_{k+1}, the system should test:

$\text{TracePreserving}(D_k \to D_{k+1}) = \text{true}$. (6.59)

If false:

$\text{Reject}(D_{k+1}) \vee \text{Quarantine}(D_{k+1})$. (6.60)

A quarantined declaration may be explored, but it should not become the active declaration until trace transport is repaired.

So:

TraceFailure ⇒ Quarantine or Reject. (6.61)

This protects the self from losing continuity.

6.10 Summary of Section 6

Trace preservation is the no-amnesia rule.

A revision must preserve ledger accountability:

$T_k \to_{k+1} : L_k \to L_{k+1}$. (6.62)

$\|Inv(T_k \to_{k+1}(L_k)) - Inv(L_k)\| \le \varepsilon_inv$. (6.63)

Revision itself must become trace:

$RevisionTrace_k = Record(D_k \to D_{k+1}, U_a, Reason_k, Evidence_k, Residual_k, Invariants_k, Test_k)$. (6.64)

Amnesia is inadmissible:

$RevisionByAmnesia \Rightarrow D_{k+1} \notin \mathfrak{F}_adm$. (6.65)

Self-continuity requires trace continuity:

SelfContinuity requires TraceContinuity. (6.66)

The next section develops residual honesty: the no-concealment rule.

---

7. Residual Honesty: The No-Concealment Rule

7.1 Why residual honesty is necessary

Trace preservation prevents amnesia.

Residual honesty prevents concealment.

A self-revising system must not hide the unresolved remainder left by its own declarations.

Residual is not merely error. It is the pressure that tells the observer where its declaration is incomplete.

Residual may indicate:

missing evidence. (7.1)

wrong boundary. (7.2)

weak feature map. (7.3)

outdated baseline. (7.4)

gate failure. (7.5)

frame fragility. (7.6)

unmodeled coupling. (7.7)

unpaid cost. (7.8)

contradiction. (7.9)

future option value. (7.10)

If residual is hidden, the system loses its main source of learning.

Thus:

NoResidualHonesty ⇒ NoMatureSelfRevision. (7.11)

A mature observer must carry residual honestly.

7.2 Residual is not failure

A common mistake is to treat residual as embarrassment.

But residual is not always failure.

Residual can be:

noise. (7.12)

ambiguity. (7.13)

alternative hypothesis. (7.14)

unmeasured structure. (7.15)

unresolved contradiction. (7.16)

future research path. (7.17)

known limitation. (7.18)

admissible uncertainty. (7.19)

open option. (7.20)

danger signal. (7.21)

A good declaration does not eliminate all residual immediately.

It governs residual.

So:

MatureClosure = Trace ⊔ GovernedResidual. (7.22)

and:

ResidualHonesty = ability to carry unresolved remainder without hiding it. (7.23)

7.3 Residual transport

When a declaration revises, residual must be transported.

The residual transport condition is:

$R_{k+1} = \text{Unexplained}(T_{k \to k+1}(R_k)) \sqcup \text{NewResidual}_k$. (7.24)

This formula says three things.

First, old residual is not automatically erased.

Second, some old residual may be explained or resolved by the new declaration.

Third, new residual may appear because the new declaration reveals previously hidden problems.

Thus residual can transform, but it must remain accountable.

If R_k disappears after revision, the system must explain whether it was resolved, reclassified, transferred, or hidden.

So:

ResidualChange requires ResidualTrace. (7.25)

Without residual trace, revision may conceal failure.

7.4 Hidden residual

Define HiddenResidual(D) as the unresolved remainder not disclosed by declaration D.

Residual honesty requires:

$\text{HiddenResidual}(D_{k+1}) \leq \text{HiddenResidual}(D_k) + \varepsilon_R$. (7.26)

This does not mean all hidden residual must decrease at every step.

Sometimes the new declaration discovers previously hidden residual, so visible residual increases. That is a sign of improved honesty, not failure.

The important point is that hidden residual should not increase through concealment.

We distinguish:

VisibleResidual(D). (7.27)

HiddenResidual(D). (7.28)

A good revision may increase VisibleResidual because it reduces HiddenResidual.

So:

Increase(VisibleResidual) may indicate better disclosure. (7.29)

Decrease(VisibleResidual) may indicate either repair or concealment. (7.30)

Therefore residual metrics must be interpreted carefully.

7.5 Residual repair versus residual concealment

Residual repair and residual concealment can look similar from outside because both may reduce visible residual.

But they are opposite.

Residual repair occurs when unresolved remainder is genuinely explained, bounded, transferred, or resolved.

Residual concealment occurs when unresolved remainder is hidden by changing the declaration.

Thus:

ResidualRepair = reduction of residual with preserved trace and explanation. (7.31)

ResidualConcealment = reduction of visible residual by hiding, redefining, or deleting unresolved remainder. (7.32)

The test is:

ResidualRepair requires TracePreservation ∧ Explanation ∧ InvarianceCheck. (7.33)

ResidualConcealment lacks at least one of these.

This distinction is essential.

Without it, a self-revising system can appear to improve by hiding its own failures.

7.6 Types of residual and matching repairs

Residual honesty requires residual typing.

A residual should not simply be marked “unknown.” It should be classified.

Possible residual types include:

R_data = missing or unreliable data. (7.34)

R_model = model limitation. (7.35)

R_boundary = boundary leakage. (7.36)

R_feature = feature-map insufficiency. (7.37)

R_gate = commitment-rule failure. (7.38)

R_trace = ledger or trace weakness. (7.39)

R_frame = frame robustness failure. (7.40)

R_budget = cost, dissipation, or health-gap failure. (7.41)

R_semantic = unresolved meaning or intent. (7.42)

R_governance = unclear authority or accountability. (7.43)

Then each residual type suggests a repair:

R_boundary ⇒ RepairB. (7.44)

R_feature ⇒ Repairφ. (7.45)

R_gate ⇒ RepairGate. (7.46)

R_trace ⇒ RepairTrace. (7.47)

R_frame ⇒ RepairInvariance or RepairTransport. (7.48)

R_budget ⇒ RepairBudget or RecoveryProtocol. (7.49)

R_semantic ⇒ RepairΔ or Repairφ. (7.50)

R_governance ⇒ RepairAuthority or RepairGate. (7.51)

This turns residual into actionable self-revision.

7.7 Residual honesty and humility

Residual honesty is the formal version of epistemic humility.

A system that cannot say “this remains unresolved” cannot mature.

It will either overclose or avoid closure entirely.

Overclosure means:

Residual hidden inside false certainty. (7.52)

No closure means:

Residual prevents action forever. (7.53)

The mature position is:

Close what can be closed; ledger what was closed; carry what remains residual. (7.54)

So:

MatureClosure = CommittedTrace + TypedResidual. (7.55)

This is not weakness. It is the basis of future learning.

7.8 Residual honesty and self-revision

Residual is the main input to self-revision.

If residual is hidden, U_a receives false input.

Then the revision operator repairs the wrong thing.

So:

HiddenResidual ⇒ MisguidedRevision. (7.56)

MisguidedRevision ⇒ PathologicalOrbit. (7.57)

Residual honesty protects the self-revising fractal.

A clean revision orbit requires:

R_k is visible enough. (7.58)

R_k is typed enough. (7.59)

R_k is trace-linked enough. (7.60)

R_k is actionable enough. (7.61)

Thus:

ResidualHonest(D) ⇔ residual is typed, scoped, attached, and revisitable. (7.62)

7.9 Residual honesty as an admissibility gate

Residual honesty should gate revision.

Before accepting D_{k+1}, the system should test:

ResidualHonest($D_k \to D_{k+1}$) = true. (7.63)

If false:

Reject(D_{k+1}) ∨ Quarantine(D_{k+1}). (7.64)

A revision that hides residual should not enter the active declaration orbit.

So:

ResidualConcealment ⇒ inadmissible. (7.65)

This is the no-concealment rule.

7.10 Summary of Section 7

Residual honesty is necessary because residual is the pressure of future learning.

The core residual transport formula is:

R_{k+1} = Unexplained($T_{k \to k+1}(R_k)$) ⊔ $NewResidual_k$. (7.66)

The hidden residual bound is:

HiddenResidual(D_{k+1}) ≤ HiddenResidual(D_k) + ε_R. (7.67)

Residual repair differs from concealment:

ResidualRepair ≠ ResidualConcealment. (7.68)

Residual honesty means:

ResidualHonest(D) ⇔ residual is typed, scoped, attached, and revisitable. (7.69)

The next section develops frame robustness and recursive objectivity.

---

8. Frame Robustness and Recursive Objectivity

8.1 Why frame robustness is necessary

A declaration must not change its governed response merely because the same object is described in an equivalent frame.

If equivalent frames produce incompatible outcomes, the system cannot distinguish real difference from representational difference.

That is frame fragility.

A mature declaration requires frame robustness.

Let:

frame_i ≡_D frame_j. (8.1)

This means frame_i and frame_j are equivalent under declaration D.

Then frame robustness requires:

Dist(Response_D(frame_i), Response_D(frame_j)) ≤ ε_frame. (8.2)

If this fails:

FrameFragile(D) = true. (8.3)

Frame fragility is dangerous because a self-revising system can exploit it. It can change frames to change outcomes without admitting a real declaration change.

Therefore:

FrameRobust(D) is required for D ∈ 𝔉_adm. (8.4)

8.2 Examples of frame fragility

In AI, frame fragility appears when equivalent prompts produce materially different governed answers.

For example:

Prompt variant changes conclusion despite same evidence. (8.5)

Schema variant changes verification status despite same claims. (8.6)

Tool path changes result despite equivalent source state. (8.7)

In finance, frame fragility appears when the same exposure looks safe in one frame and

dangerous in another without a declared reconciliation rule.

For example:

Trading view says hedged; funding view says exposed. (8.8)

Accounting view says solvent; liquidity view says fragile. (8.9)

Legal view says transferable; collateral operations view says blocked. (8.10)

In law, frame fragility appears when equivalent facts produce inconsistent outcomes.

In science, it appears when a result depends on notation, instrument framing, or arbitrary coordinate choice.

In all cases:

FrameFragility ⇒ weak objectivity. (8.11)

8.3 Frame robustness across revision

Part 4 adds a stronger requirement.

A declaration should remain frame-robust not only at one moment, but across revision.

If D_k is frame-robust and D_{k+1} is an admissible revision, then D_{k+1} should also be frame-robust:

$\mathrm{FrameRobust}(D_k) \wedge \mathrm{AdmissibleRevision}(D_k \to D_{k+1}) \Rightarrow \mathrm{FrameRobust}(D_{k+1})$. (8.12)

This does not mean the same frames remain equivalent after revision. A revision may change the declared equivalence class. But if it does, the change must be explicit and traceable.

So:

Change(frame equivalence) requires RevisionTrace. (8.13)

Silent frame change is ontology drift.

Thus:

SilentFrameDrift ⇒ inadmissible. (8.14)

8.4 Transport between declarations

Frame robustness within one declaration is not enough.

Self-revision requires transport between declarations.

Let D_i and D_j be two admissible declarations in the revision orbit.

A transport map is:

T_{i→j}: L_{D_i} → L_{D_j}. (8.15)

For a relation R, transport gives:

$T_{i \to j}(R_i)$. (8.16)

The relation is stable across declarations if:

$\mathrm{Dist}(T_{i \to j}(R_i), R_j) \leq \varepsilon_R$. (8.17)

This is cross-declaration invariance.

It is the basis of recursive objectivity.

8.5 Recursive objectivity

Part 3 defined objectivity as invariance across admissible declared disclosures.

Part 4 strengthens this.

A self-revising observer has a whole revision orbit:

$\mathfrak{A} = \mathrm{closure}(\{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), \ldots\})$. (8.18)

A relation is recursively objective if it survives across the admissible revision orbit.

So:

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (8.19)

More explicitly:

R is recursively objective $\Leftrightarrow \forall D_i, D_j \in \mathfrak{A}, \mathrm{Dist}(T_{i \to j}(R_i), R_j) \leq \varepsilon_R$. (8.20)

This is stronger than ordinary objectivity across static declarations.

It says the relation survives learning.

A relation that survives not only different frames but also admissible self-revision is deeply objective for that observer family.

8.6 Recursive law

Similarly, Part 4 defines recursive law.

Part 3:

Law = invariant relation across admissible declarations. (8.21)

Part 4:

$\mathrm{Law}_{\mathrm{recursive}}$ = invariant trace grammar over $\mathfrak{A}$. (8.22)

That means a law-like relation persists across the system's admissible self-revision orbit.

So:

$\mathrm{Law}_{\mathrm{recursive}}(R) \Leftrightarrow R$ is preserved across $D_i, D_j \in \mathfrak{A}$ under transport $T_{i \to j}$. (8.23)

This is important for learning systems.

If every revision destroys the relation, then the relation is not recursively law-like. It may be local, temporary, or declaration-specific.

A mature observer seeks relations that survive its own learning.

This gives a strong definition:

Knowledge = recursively objective trace relation. (8.24)

8.7 Frame robustness and selfhood

A self that cannot preserve frame robustness during revision becomes unstable.

It may change its answer whenever wording changes.

It may change its law whenever boundary shifts.

It may change its identity whenever residual appears.

This is not mature selfhood. It is semantic drift.

So:

SelfContinuity requires FrameContinuity. (8.25)

Frame continuity does not mean all frames are frozen. It means changes in frame equivalence are declared, traced, and transported.

Therefore:

FrameChange is admissible only if TracePreserved ∧ ResidualHonest ∧ TransportDefined. (8.26)

Without this, self-revision destroys objectivity.

8.8 Frame robustness as anti-manipulation

Frame robustness prevents the observer from manipulating its own reality.

A pathological system may change frames to avoid contradiction.

For example:

When evidence fails, it changes what counts as evidence. (8.27)

When residual rises, it changes what counts as residual. (8.28)

When law fails, it changes what counts as equivalent case. (8.29)

When gate fails, it changes what counts as threshold. (8.30)

These may be legitimate only if declared and tested.

Otherwise they are frame manipulation.

So:

FrameManipulation = untraced frame change that improves apparent outcome. (8.31)

FrameManipulation ⇒ inadmissible. (8.32)

This is why frame robustness is central to admissibility.

8.9 Frame robustness as a revision gate

Before accepting D_{k+1}, the system should test:

$\mathrm{FrameRobust}(D_{k+1}) = \mathrm{true}$. (8.33)

and:

$\mathrm{TransportDefined}(D_k, D_{k+1}) = \mathrm{true}$. (8.34)

If either fails:

$\mathrm{Reject}(D_{k+1}) \vee \mathrm{Quarantine}(D_{k+1})$. (8.35)

A quarantined declaration may be explored, but it should not become the active self unless frame transport is repaired.

Thus:

FrameFailure ⇒ Quarantine or Reject. (8.36)

8.10 Summary of Section 8

Frame robustness means:

frame_i ≡_D frame_j ⇒ Dist(Response_D(frame_i), Response_D(frame_j)) ≤ ε_frame. (8.37)

Across revision:

$\mathrm{FrameRobust}(D_k) \wedge \mathrm{AdmissibleRevision}(D_k \to D_{k+1}) \Rightarrow \mathrm{FrameRobust}(D_{k+1})$. (8.38)

Recursive objectivity is:

RecursiveObjectivity = invariance across admissible revision orbit 𝔄. (8.39)

A relation is recursively objective if:

∀D_i,D_j ∈ 𝔄, Dist(T_{i→j}(R_i), R_j) ≤ ε_R. (8.40)

Frame robustness prevents self-revision from becoming frame manipulation.

The next section develops budget boundedness: health, work, and dissipation.

9. Budget Boundedness: Health, Work, and Dissipation

9.1 Why admissible revision must be budget-bounded

A declaration can be well-formed, trace-preserving, residual-honest, and frame-robust, yet still fail.

It may be too expensive to maintain.

It may require too much computation.

It may produce too much dissipation.

It may create too much governance burden.

It may preserve trace, but only by consuming all available capacity.

It may be beautifully objective, but operationally impossible.

Therefore, admissible self-revision must be budget-bounded.

A mature observer cannot revise into a declaration that it cannot maintain.

So:

AdmissibleRevision ⇒ BudgetBounded(D_{k+1}). (9.1)

Budget boundedness protects the system from unsustainable intelligence.

9.2 The dual-ledger quantities

Part 3 introduced the dual-ledger bridge.

A declared world is not only readable. It must also be maintained.

The key variables are:

s_P = maintained structure under feature map φ_P. (9.2)

λ_P = drive coupled to maintained structure s_P. (9.3)

$G_{gap,P}$ = health gap between drive and structure. (9.4)

$W_{s,P}$ = structural work. (9.5)

$\Gamma_{loss,P}$ = dissipation, leakage, openness, or loss. (9.6)

κ_P = conditioning, curvature, or brittleness measure. (9.7)

For Part 4, these quantities become constraints on self-revision.

A declaration that increases apparent intelligence but pushes G_{gap}, Γ_{loss}, or κ beyond viable bounds is not mature.

Thus:

$\text{BudgetBounded}(D) \Leftrightarrow \Gamma_{loss}(D) \le \Gamma^* \wedge G_{gap}(D) \le G^* \wedge \kappa(D) \le \kappa^*$. (9.8)

9.3 Health gap

The health gap measures whether drive and maintained structure are aligned.

The core formula is:

$G_{gap}(\lambda,s) = \Phi(s) + \psi(\lambda) - \lambda \cdot s \geq 0.$ (9.9)

When drive and structure match:

$G_{gap} = 0.$ (9.10)

When they diverge:

$G_{gap} > 0.$ (9.11)

In self-revision, this matters because a declaration may chase the wrong drive.

For example:

An AI runtime may optimize for fluent answer production while the maintained structure should be evidence-grounded artifact quality. (9.12)

An institution may optimize for procedural throughput while the maintained structure should be legitimacy. (9.13)

A financial system may optimize for reported solvency while the maintained structure should be liquidity survival. (9.14)

A biological system may optimize short-term inflammation response while damaging long-term tissue coherence. (9.15)

In all cases, the declaration may appear active but unhealthy.

So:

Rising $G_{gap} \Rightarrow$ drive-structure misalignment. (9.16)

A self-revision that raises G_{gap} without declared exploration budget is suspicious.

9.4 Dissipation

Dissipation Γ_{loss} measures what is lost in maintaining or revising structure.

The budget identity is:

$\Delta\Phi = W_s - \Delta\psi - \Gamma_{loss}.$ (9.17)

where:

$\Delta\Phi$ = retained structural value. (9.18)

W_s = structural work. (9.19)

$\Delta\psi$ = drive-side expansion or statistical budget change. (9.20)

Γ_{loss} = dissipative loss. (9.21)

If Γ_{loss} is too high, the declaration may be burning capacity faster than it creates stable structure.

In AI, Γ_{loss} may appear as excessive tool calls, repeated failed verification, context churn, contradiction loops, or residual explosion.

In institutions, Γ_loss may appear as bureaucracy, appeal overload, procedural duplication, or legitimacy loss.

In finance, Γ_loss may appear as liquidity leakage, collateral deadlock, transaction cost, or settlement friction.

In biology, Γ_loss may appear as metabolic exhaustion, inflammation damage, or repair inefficiency.

Thus:

High Γ_loss ⇒ disclosure is costly to maintain. (9.22)

Admissible revision must bound Γ_loss.

9.5 Structural work

Structural work measures how much effort is used to change maintained structure.

The continuous form is:

$W_s = \int \lambda \cdot ds$. (9.23)

The discrete tick form is:

$\Delta W_{s,k} = \lambda_k \cdot (s_k - s_{k-1})$. (9.24)

A self-revision should be evaluated by whether structural work becomes retained value.

If W_s is high but ΔΦ is low, the system is churning.

If W_s is high and Γ_loss is high, the system may be in false vitality.

If W_s is low but residual is rising, the system may be under-correcting.

So:

Work alone does not prove improvement. (9.25)

The relevant audit is:

RetainedValue = Work − DriveCost − Loss. (9.26)

or:

$\Delta\Phi = W_s - \Delta\psi - \Gamma_{loss}$. (9.27)

A self-revising observer must distinguish productive work from dissipative motion.

9.6 Conditioning and curvature κ

The variable κ represents conditioning, curvature, brittleness, or instability of the declaration.

A declaration may be budget-light and residual-honest but still brittle.

For example:

A model may work only under narrow prompt phrasing. (9.28)

A financial hedge may work only under normal liquidity. (9.29)

A legal rule may work only under ordinary cases but fail under edge cases. (9.30)

An institutional process may work only when actors cooperate. (9.31)

This brittleness is captured by κ.

A simple admissibility condition is:

$\kappa(D) \leq \kappa^*$. (9.32)

If κ is high, small changes in frame, residual, or input create large changes in response.

This is dangerous for self-revision because the declaration orbit may become unstable.

Thus:

High $\kappa \Rightarrow$ fragile declaration orbit. (9.33)

A mature self-revising observer must reduce κ or quarantine the declaration.

9.7 False vitality

Budget boundedness helps detect false vitality.

False vitality occurs when a system appears highly active but is actually losing coherence.

Define:

$\text{FalseVitality}(D_k) \Leftrightarrow W_s,k \text{ high} \wedge \Gamma_loss,k \text{ high} \wedge G_gap,k \text{ rising}$. (9.34)

This is a common pathology.

In AI, the system may call many tools, retrieve many documents, generate many drafts, and still become less grounded.

In organizations, the system may hold many meetings, produce many reports, and still lose strategic clarity.

In finance, trading volume may rise while liquidity quality collapses.

In biological systems, immune activation may rise while organism health declines.

Thus:

Activity $\neq$ health. (9.35)

Revision $\neq$ learning. (9.36)

Trace volume $\neq$ wisdom. (9.37)

Budget boundedness prevents self-revision from mistaking churn for intelligence.

9.8 Budget-bounded revision

A revision from D_k to D_{k+1} is budget-bounded if the new declaration satisfies:

$\Gamma_loss(D_{k+1}) \le \Gamma^*$. (9.38)

$G_gap(D_{k+1}) \le G^*$. (9.39)

$\kappa(D_{k+1}) \le \kappa^*$. (9.40)

So:

$\text{AdmissibleRevision} \Rightarrow \Gamma_loss(D_{k+1}) \le \Gamma^* \wedge G_gap(D_{k+1}) \le G^* \wedge \kappa(D_{k+1}) \le \kappa^*$. (9.41)

If the revision violates these:

$\text{Violation}(9.41) \Rightarrow \text{RecoveryProtocol}(D_{k+1})$. (9.42)

The declaration may still be explored, but it should not become the active stable declaration unless recovery succeeds.

9.9 Recovery protocol

A recovery protocol is required when a candidate declaration is promising but currently over budget.

Possible recovery actions include:

reduce scope B. (9.43)

simplify feature map φ. (9.44)

increase gate threshold. (9.45)

improve residual classification. (9.46)

reduce tool or intervention cost. (9.47)

split the declaration into sub-declarations. (9.48)

quarantine unstable cases. (9.49)

add trace compression. (9.50)

revise time window h. (9.51)

The recovery protocol can be represented as:

$D_recovered = \text{Recover}(D_candidate \mid \Gamma_loss, G_gap, \kappa, R)$. (9.52)

A candidate may enter the active admissible family only if:

$D_recovered \in \mathfrak{F}_adm$. (9.53)

This prevents high-cost declarations from becoming permanent.

9.10 Budget boundedness and selfhood

A self must be able to maintain itself.

If a self-revising declaration orbit consumes more resources than it can stabilize, it collapses.

So:

SelfContinuity requires BudgetContinuity. (9.54)

This does not mean the system must be static or cheap.

It means the cost of self-revision must remain bounded relative to the value of maintained structure.

A mature observer can spend energy to revise itself.

But it must not consume itself in revision.

Thus:

HealthySelfRevision = adaptive revision with bounded health gap, dissipation, and brittleness. (9.55)

Budget boundedness is therefore a condition of durable observerhood.

9.11 Summary of Section 9

Budget boundedness protects self-revision from unsustainable disclosure.

The basic condition is:

$$\mathrm{BudgetBounded}(D) \Leftrightarrow \Gamma_\mathrm{loss}(D) \le \Gamma^* \wedge G_\mathrm{gap}(D) \le G^* \wedge \kappa(D) \le \kappa^*. \quad (9.56)$$

Health gap is:

$$G_\mathrm{gap}(\lambda,s) = \Phi(s) + \psi(\lambda) - \lambda \cdot s \ge 0. \quad (9.57)$$

The budget identity is:

$$\Delta\Phi = W_s - \Delta\psi - \Gamma_\mathrm{loss}. \quad (9.58)$$

False vitality is:

$$\mathrm{FalseVitality}(D_k) \Leftrightarrow W_s,k \text{ high} \wedge \Gamma_\mathrm{loss},k \text{ high} \wedge G_\mathrm{gap},k \text{ rising}. \quad (9.59)$$

A mature revision must satisfy:

$$\mathrm{AdmissibleRevision} \Rightarrow \mathrm{BudgetBounded}(D_{k+1}). \quad (9.60)$$

The next section develops non-degeneracy: avoiding empty, total, and unfalsifiable worlds.

---

10. Non-Degeneracy: Avoiding Empty, Total, and Unfalsifiable Worlds

10.1 Why non-degeneracy is needed

A declaration can satisfy several constraints and still become useless.

It may become too empty.

It may become too total.

It may become unfalsifiable.

These are degeneracies.

A degenerate declaration avoids failure by destroying meaningful testability.

Therefore:

Admissible(D) requires NonDegenerate(D). (10.1)

Non-degeneracy protects the framework from vacuity and dogma.

10.2 The empty declaration

The empty declaration says:

Nothing counts. (10.2)

Call it:

D_empty. (10.3)

In D_empty, no projection matters, no residual matters, no gate matters, and no contradiction matters.

This avoids error by refusing structure.

But a system that declares nothing cannot learn, act, or maintain a world.

So:

D_empty ∉ 𝔉_adm. (10.4)

The empty declaration is not humble. It is sterile.

It avoids false closure by refusing all closure.

Thus:

D_empty ⇒ NoWorldDisclosure. (10.5)

NoWorldDisclosure ⇒ NoObserverhood. (10.6)

10.3 The total declaration

The opposite degeneracy is the total declaration.

The total declaration says:

Everything confirms me. (10.7)

Call it:

D_total. (10.8)

In D_total, every event becomes supporting evidence. Every contradiction is reclassified as deeper confirmation. Every residual is absorbed. Every frame failure becomes proof of the system's superiority.

This produces a semantic black hole.

So:

D_total ∉ 𝔉_adm. (10.9)

The total declaration appears powerful because nothing can refute it.

But that is exactly why it is invalid.

A declaration that cannot fail cannot learn.

Thus:

UnfalsifiableTotality ⇒ Dogma. (10.10)

10.4 The unfalsifiable declaration

A declaration must include conditions under which it fails.

So:

Falsifiable(D) = true. (10.11)

A declaration is unfalsifiable if no possible trace, residual, gate failure, frame failure, or budget failure can count against it.

That means:

∀Evidence, D confirms itself. (10.12)

This is inadmissible.

A well-formed declaration must specify:

projection failure conditions. (10.13)

gate failure conditions. (10.14)

residual alarm conditions. (10.15)

frame robustness failure conditions. (10.16)

budget failure conditions. (10.17)

revision failure conditions. (10.18)

Without these, the declaration cannot be tested.

So:

NoFalsificationGate ⇒ D ∉ 𝔉_adm. (10.19)

10.5 Informative range

A declaration must carry enough information to distinguish cases, but not so much that it becomes overfit.

The information bound is:

I_min ≤ Info(D) ≤ I_max. (10.20)

If:

Info(D) < I_min. (10.21)

the declaration is too vague.

If:

Info(D) > I_max. (10.22)

the declaration may be overfit, too costly, or too brittle.

The exact definition of Info(D) depends on the domain. It may measure model complexity, schema length, number of feature dimensions, number of gate conditions, or description length.

The principle is:

Admissible declaration must be informative but not overdetermined. (10.23)

10.6 Specificity

A declaration must be specific enough to act.

The specificity condition is:

0 < Specificity(D) < ∞. (10.24)

If Specificity(D) = 0, the declaration says nothing.

If Specificity(D) = ∞, the declaration may describe only one case and fail to generalize.

A good declaration has bounded specificity.

It can distinguish relevant cases without becoming a one-off patch.

Thus:

NonDegenerate(D) requires bounded specificity. (10.25)

10.7 Semantic black holes

A semantic black hole is a declaration that absorbs all residual, contradiction, and failure into

itself.

Define:

SemanticBlackHole(D) ⇔ Contradiction → Confirmation. (10.26)

More explicitly:

SemanticBlackHole(D) ⇔ ∀r ∈ R, Interpret_D(r) = Support(D). (10.27)

This is a severe pathology.

It may look like a grand theory. It may explain everything. But it explains everything by preventing anything from counting against it.

A semantic black hole destroys residual honesty and falsifiability.

So:

SemanticBlackHole(D) ⇒ D ∉ ℑ_adm. (10.28)

10.8 Non-degeneracy and humility

Non-degeneracy requires a disciplined middle position.

The declaration must not say:

Nothing can be known. (10.29)

Nor:

Everything confirms me. (10.30)

It must say:

Here is what this declaration can disclose; here is what would falsify it; here is what remains residual. (10.31)

This is the mature form.

A non-degenerate declaration is confident enough to project and humble enough to fail.

Thus:

NonDegenerate(D) = informative ∧ specific ∧ falsifiable. (10.32)

10.9 Non-degeneracy as a revision gate

Before accepting D_{k+1}, the system should test:

NonDegenerate(D_{k+1}) = true. (10.33)

If false:

Reject(D_{k+1}) ∨ Quarantine(D_{k+1}). (10.34)

A quarantined degenerate declaration may be studied as a failure mode, but it should not become the active observer declaration.

So:

DegenerateRevision ⇒ inadmissible. (10.35)

10.10 Summary of Section 10

Non-degeneracy prevents empty, total, and unfalsifiable worlds.

The conditions are:

$I_{min} \le \mathrm{Info}(D) \le I_{max}$. (10.36)

$0 < \mathrm{Specificity}(D) < \infty$. (10.37)

$\mathrm{Falsifiable}(D) = \mathrm{true}$. (10.38)

The empty declaration is inadmissible:

$D_{empty} \notin \mathfrak{F}_{adm}$. (10.39)

The total declaration is inadmissible:

$D_{total} \notin \mathfrak{F}_{adm}$. (10.40)

A semantic black hole is:

$\mathrm{SemanticBlackHole}(D) \Leftrightarrow \mathrm{Contradiction} \rightarrow \mathrm{Confirmation}$. (10.41)

and:

$\mathrm{SemanticBlackHole}(D) \Rightarrow D \notin \mathfrak{F}_{adm}$. (10.42)

The next section develops trust regions, switch gates, and quarantine.

---

11. Trust Regions, Switch Gates, and Quarantine

11.1 Why trust regions are needed

A self-revising declaration must change.

But uncontrolled change destroys continuity.

If D_{k+1} can be arbitrarily far from D_k, the system may silently become a different observer. Its ledger transport may fail. Its invariants may vanish. Its residual may be hidden. Its object may drift.

Therefore ordinary revision should be local.

Let $d_{\mathfrak{F}}$ be a distance on declaration space.

A local revision satisfies:

$d_{\mathfrak{F}}(D_{k+1}, D_k) \le \varepsilon_D$. (11.1)

This is the trust-region condition.

It does not forbid large changes. It says large changes require special handling.

11.2 Declaration distance

The distance $d_{\mathfrak{F}}$ should compare declarations.

A declaration has components:

$D = (q, \varphi, P, \hat{O}, \text{Gate}, \text{TraceRule}, \text{ResidualRule})$. (11.2)

So a candidate metric may include:

$d_q(q_i, q_j)$. (11.3)

$d_\varphi(\varphi_i, \varphi_j)$. (11.4)

$d_P(P_i, P_j)$. (11.5)

$d_{\hat{O}}(\hat{O}_i, \hat{O}_j)$. (11.6)

$d_{\text{Gate}}(\text{Gate}_i, \text{Gate}_j)$. (11.7)

$d_{\text{Trace}}(\text{TraceRule}_i, \text{TraceRule}_j)$. (11.8)

$d_R(\text{ResidualRule}_i, \text{ResidualRule}_j)$. (11.9)

A weighted total form is:

$d_{\mathfrak{F}}(D_i, D_j) = \alpha_q \cdot d_q + \alpha_\varphi \cdot d_\varphi + \alpha_P \cdot d_P + \alpha_{\hat{O}} \cdot d_{\hat{O}} + \alpha_G \cdot d_{\text{Gate}} + \alpha_T \cdot d_{\text{Trace}} + \alpha_R \cdot d_R$. (11.10)

The correct metric is domain-dependent and remains an open mathematical problem. But some distance notion is needed.

Without distance, the theory cannot distinguish repair from regime switch.

11.3 Local revision

A local revision remains within the trust region:

$d_{\mathfrak{F}}(D_{k+1}, D_k) \le \varepsilon_D$. (11.11)

Examples:

slightly adjusting a verifier threshold. (11.12)

adding one residual category. (11.13)

expanding a feature map modestly. (11.14)

tightening a gate after repeated false trace. (11.15)

adding metadata to a trace rule. (11.16)

Local revisions preserve continuity.

They can be accepted if they pass admissibility gates.

So:

$\text{LocalRevision} \wedge D_{k+1} \in \mathfrak{F}_{adm} \Rightarrow \text{Accept}(D_{k+1})$. (11.17)

11.4 Switch revision

A switch revision exceeds the trust region:

$d_{\mathfrak{F}}(D_{k+1}, D_k) > \varepsilon_D$. (11.18)

Switch revisions may be necessary.

Examples:

Changing boundary from local to systemic. (11.19)

Changing timebase from token-time to episode-time. (11.20)

Changing financial protocol from price-risk to liquidity-risk. (11.21)

Changing AI workflow from answer generation to artifact governance. (11.22)

Changing biological interpretation from local symptom to systemic disease. (11.23)

These changes may be legitimate, but they cannot be silent.

A switch requires:

$\text{SwitchGate}_k = \text{Open}$. (11.24)

So:

$d_{\mathfrak{F}}(D_{k+1}, D_k) > \varepsilon_D \Rightarrow \text{SwitchGate}_k = \text{Open}$. (11.25)

11.5 Switch gate

The switch gate decides whether a large declaration change is allowed.

It should ask:

What residual forces the switch? (11.26)

Which trace supports the switch? (11.27)

Which invariants are preserved? (11.28)

Which invariants are intentionally broken? (11.29)

How will old ledger transport into new ledger? (11.30)

What quarantine test will validate the new declaration? (11.31)

Thus:

SwitchGate = Gate over declaration-level transition. (11.32)

This is a higher-order gate.

Ordinary Gate commits projected structure.

SwitchGate commits declaration transition.

So:

Gate_P commits $V_P \to \tau_k$. (11.33)

SwitchGate commits $D_k \to D_{k+1}$. (11.34)

This is important because self-revision itself must be gated.

11.6 Quarantine

A switched declaration should not immediately become the active self.

It should enter quarantine.

$\text{Switch}(D_k \to D_{k+1}) \Rightarrow \text{Quarantine}(D_{k+1}, T_test)$. (11.35)

During quarantine, the system tests:

trace transport. (11.36)

residual honesty. (11.37)

frame robustness. (11.38)

budget boundedness. (11.39)

non-degeneracy. (11.40)

prediction or intervention improvement. (11.41)

cross-declaration invariance. (11.42)

Only if the new declaration passes these tests should it become active.

So:

$\text{PassQuarantine}(D_{k+1}) \Rightarrow \text{Activate}(D_{k+1})$. (11.43)

$\text{FailQuarantine}(D_{k+1}) \Rightarrow \text{Reject}(D_{k+1}) \vee \text{ArchiveAsResidual}(D_{k+1})$. (11.44)

Quarantine prevents catastrophic self-revision.

11.7 Silent ontology drift

Silent ontology drift occurs when D changes without declaration, trace, switch gate, or quarantine.

SilentOntologyDrift ⇔ $D_{k+1} \neq D_k$ and RevisionTrace absent. (11.45)

This is inadmissible.

Silent ontology drift is dangerous because the system appears to continue the same discussion while the object has changed.

In AI, this may appear as changing criteria mid-answer.

In finance, it may appear as switching from solvency to liquidity without saying so.

In law, it may appear as changing category or jurisdiction without declaration.

In science, it may appear as changing measurement protocol while claiming continuity.

Thus:

SilentOntologyDrift ⇒ $D_{k+1} \notin \mathfrak{F}_adm$. (11.46)

11.8 Trust region and identity

Trust regions preserve self-continuity.

A self can change, but not by arbitrary discontinuity.

Local revision preserves identity by staying within a bounded neighborhood.

Switch revision preserves identity by explicitly transporting trace and passing quarantine.

So:

SelfContinuity = LocalRevision ∨ GatedSwitchRevision. (11.47)

where:

LocalRevision = $d_\mathfrak{F}(D_{k+1},D_k) \leq \varepsilon_D$. (11.48)

GatedSwitchRevision = $d_\mathfrak{F}(D_{k+1},D_k) > \varepsilon_D \wedge \text{SwitchGate}_k = \text{Open} \wedge \text{PassQuarantine}$. (11.49)

This gives a clean rule.

A self can undergo radical change, but only through declared transition.

11.9 Summary of Section 11

Ordinary revision should satisfy the trust-region condition:

$d_\mathfrak{F}(D_{k+1},D_k) \leq \varepsilon_D$. (11.50)

Large revisions require a switch gate:

$d_\mathfrak{F}(D_{k+1},D_k) > \varepsilon_D \Rightarrow \text{SwitchGate}_k = \text{Open}$. (11.51)

Switched declarations require quarantine:

$\text{Switch}(D_k \to D_{k+1}) \Rightarrow \text{Quarantine}(D_{k+1},T_test)$. (11.52)

Silent ontology drift is inadmissible:

$\text{SilentOntologyDrift} \Rightarrow D_{k+1} \notin \mathfrak{F}_{adm}$. (11.53)

Self-continuity requires:

$\text{SelfContinuity} = \text{LocalRevision} \vee \text{GatedSwitchRevision}$. (11.54)

The next section defines declaration energy and improvement conditions.

---

12. Declaration Energy and Improvement Conditions

12.1 Why improvement needs an audit

A self-revising system must decide whether a revision improves the declaration.

But improvement is not obvious.

A revision may reduce visible residual by hiding it.

It may increase activity while worsening health.

It may improve one frame while breaking another.

It may reduce immediate error while increasing long-term brittleness.

It may improve performance by becoming unfalsifiable.

Therefore the system needs an audit function.

Part 4 introduces a declaration energy:

$\mathcal{V}(D)$. (12.1)

This is not a final physical energy. It is an operational audit score for declaration quality.

Lower $\mathcal{V}$ means better declaration health, assuming the weights are properly chosen.

12.2 A candidate declaration energy

A useful candidate is:

$\mathcal{V}(D_k) = a \cdot G_gap_{,k} + b \cdot \|R_k\| + c \cdot \Gamma_loss_{,k} + d \cdot \kappa_k + e \cdot Curv_k - f \cdot InvScore_k$. (12.2)

where:

$G_gap_{,k}$ = drive-structure health gap. (12.3)

$\|R_k\|$ = residual magnitude. (12.4)

$\Gamma_loss_{,k}$ = dissipation or loss. (12.5)

κ_k = conditioning or brittleness. (12.6)

$Curv_k$ = cross-frame curvature or transport distortion. (12.7)

$InvScore_k$ = invariance quality. (12.8)

The coefficients:

$a,b,c,d,e,f \geq 0$. (12.9)

The minus sign before InvScore means that higher invariance lowers declaration energy.

This formula is a candidate, not a final law.

It provides a disciplined placeholder.

12.3 Ordinary improvement condition

A normal revision should reduce declaration energy.

So:

$\mathcal{V}(D_{k+1}) \leq \mathcal{V}(D_k) - \eta \cdot Gain_k + \varepsilon_noise$. (12.10)

Here:

η is learning efficiency or improvement strength. (12.11)

$Gain_k$ is expected improvement from the revision. (12.12)

ε_noise is tolerated measurement noise. (12.13)

If this condition holds, the revision is energy-improving.

But one should not rely on $\mathcal{V}$ alone. The revision must also satisfy the admissibility family:

$D_{k+1} \in \mathfrak{F}_adm$. (12.14)

Thus:

$EnergyImprovement \wedge Admissibility \Rightarrow AcceptRevision$. (12.15)

12.4 Exploration condition

Sometimes a revision must increase $\mathcal{V}$ temporarily.

For example, adding a new residual category may increase visible residual.

Adding a stronger verifier may increase cost.

Expanding frame robustness tests may expose new failures.

Changing the boundary may create temporary instability.

This can be legitimate exploration.

But exploration must be budgeted.

If:

$\mathcal{V}(D_{k+1}) > \mathcal{V}(D_k)$. (12.16)

then require:

$\text{ExplorationBudget}_k \geq \mathcal{V}(D_{k+1}) - \mathcal{V}(D_k)$. (12.17)

If not:

$\text{DestabilizationWithoutBudget} \Rightarrow \text{inadmissible}$. (12.18)

This prevents arbitrary destabilization from being called learning.

12.5 Improvement must be residual-honest

A declaration energy can be gamed.

For example, the system may reduce $\|R_k\|$ by hiding residual.

Therefore improvement must also satisfy residual honesty.

A valid improvement requires:

$\mathcal{V}(D_{k+1}) < \mathcal{V}(D_k) \wedge \text{ResidualHonest}(D_k \to D_{k+1})$. (12.19)

If $\mathcal{V}$ decreases but hidden residual increases, the improvement is false.

So:

$\text{FalseImprovement} \Leftrightarrow \mathcal{V}$ decreases by residual concealment. (12.20)

False improvement is inadmissible.

12.6 Improvement must be trace-preserving

A declaration energy can also be reduced by forgetting.

For example, if the ledger is erased, many contradictions disappear.

This lowers apparent residual and curvature, but only by destroying history.

Therefore:

$\text{ValidImprovement requires TracePreserving}(D_k \to D_{k+1})$. (12.21)

If:

$\mathcal{V}(D_{k+1}) < \mathcal{V}(D_k)$ but TracePreserving fails. (12.22)

then:

$\text{ImprovementByAmnesia} \Rightarrow \text{inadmissible}$. (12.23)

A mature observer cannot improve by deleting the evidence of failure.

12.7 Improvement must preserve frame robustness

A revision may improve performance in one frame while worsening equivalent-frame

consistency.

For example, an AI system may optimize for one prompt wording while becoming fragile to equivalent wording.

A financial model may fit one reporting view while failing legal or funding reconciliation.

An institution may resolve one case category while creating inconsistency across equivalent cases.

Therefore:

ValidImprovement requires FrameRobust(D_{k+1}). (12.24)

If InvScore decreases too much, the revision should be rejected or quarantined.

So:

LocalGain with FrameCollapse ⇒ inadmissible. (12.25)

12.8 Vector audit instead of scalar energy

A scalar $\mathcal{V}$ is useful, but it may hide tradeoffs.

A safer audit may use a vector:

Audit(D) = (G_gap, ‖R‖, Γ_loss, κ, Curv, −InvScore). (12.26)

Then improvement can be defined through partial order or Pareto dominance.

For example:

Audit(D_{k+1}) ≼ Audit(D_k). (12.27)

meaning D_{k+1} is no worse in all critical dimensions and better in at least one.

This is stricter than scalar weighting.

But in practice, scalar $\mathcal{V}$ may be useful for operational control.

Thus:

ScalarEnergy is convenient. (12.28)

VectorAudit is safer. (12.29)

Both should remain available.

12.9 Improvement and the revision orbit

The purpose of improvement is not only local.

It should stabilize the revision orbit.

A single revision may improve $\mathcal{V}$ but destabilize future revisions.

Therefore one should consider expected future energy:

$$E[\mathcal{V}(D_{k+m}) \mid D_{k+1}]. \quad (12.30)$$

A mature revision should improve long-run admissible orbit health:

$$E[\mathcal{V}(D_{k+m}) \mid D_{k+1}] \le E[\mathcal{V}(D_{k+m}) \mid D_k] - \delta. \quad (12.31)$$

for some horizon m and improvement δ.

This links declaration energy to attractor stability.

The observer does not only optimize the next declaration.

It stabilizes the self-revising family.

12.10 Summary of Section 12

A candidate declaration energy is:

$$\mathcal{V}(D_k) = a\cdot G_gap,_k + b\cdot\|R_k\| + c\cdot\Gamma_loss,_k + d\cdot\kappa_k + e\cdot Curv_k - f\cdot InvScore_k. \quad (12.32)$$

Ordinary revision should satisfy:

$$\mathcal{V}(D_{k+1}) \le \mathcal{V}(D_k) - \eta\cdot Gain_k + \varepsilon_noise. \quad (12.33)$$

Exploration is allowed only if budgeted:

$$\mathcal{V}(D_{k+1}) > \mathcal{V}(D_k) \Rightarrow ExplorationBudget_k \ge \mathcal{V}(D_{k+1}) - \mathcal{V}(D_k). \quad (12.34)$$

Valid improvement must be trace-preserving, residual-honest, frame-robust, budget-bounded, and non-degenerate.

A safer audit vector is:

$$Audit(D) = (G_gap, \|R\|, \Gamma_loss, \kappa, Curv, -InvScore). \quad (12.35)$$

The next section defines the Self-Revising Declaration Fractal 𝔄.

13. The Self-Revising Declaration Fractal 𝔄

13.1 Why the fractal appears

We now have all the parts needed to define the central object of Part 4.

A declaration D_k discloses a field.

The disclosed field produces projection.

Projection passes or fails gate.

Gate produces committed trace.

Trace and residual update the ledger.

Ledger and residual revise the next declaration.

Then the next declaration repeats the process.

The cycle is:

$D_k \to \Sigma_k \to V_k \to \tau_k \to (L_{k+1}, R_{k+1}) \to D_{k+1}$. (13.1)

This is self-similar.

The output of one disclosure becomes the input condition of the next disclosure.

In short:

$\text{Closure}_k \to \text{DeclarationInput}_{k+1}$. (13.2)

or:

$\text{LedgeredClosure}_k \to \text{RevisionMaterial}_{k+1}$. (13.3)

This is why the structure is naturally fractal.

Not because it visually resembles a Mandelbrot set, but because its rule repeats across scales:

declaration creates closure, closure revises declaration, revised declaration creates new closure. (13.4)

13.2 The generated orbit

Start with an initial admissible declaration:

$D_0 \in \mathfrak{F}_{adm}$. (13.5)

Apply one admissible revision operator:

$D_1 = U_{a_1}(D_0, L_0, R_0)$. (13.6)

Then apply another:

$D_2 = U_{a_2}(D_1, L_1, R_1)$. (13.7)

Then another:

$D_3 = U_{a_3}(D_2, L_2, R_2)$. (13.8)

The declaration orbit is:

$\text{Orb}(D_0) = \{D_0, D_1, D_2, D_3, \ldots\}$. (13.9)

More explicitly:

$\text{Orb}(D_0) = \{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\}$. (13.10)

This orbit is not merely a sequence of beliefs. It is a sequence of declared worlds.

Each D_k defines how the next world is disclosed.

13.3 The closure of all admissible paths

There may be many admissible revision choices at each step.

So instead of one orbit, we define the closure of all admissible revision paths.

The self-revising declaration fractal is:

$\mathfrak{A} = \text{closure}(\{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\})$. (13.11)

where each:

$a_i \in A_{adm}$. (13.12)

The closure operation means that limiting declaration states, repeated patterns, stable cycles, and attractor points are included when meaningful under the chosen topology.

This is why the topology of declaration space matters.

Without a topology, closure is only metaphorical.

With a topology, $\mathfrak{A}$ becomes a mathematical object.

13.4 The fixed-family equation

A fractal or iterated-family object is often characterized by a fixed-family equation.

Here the natural equation is:

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (13.13)

This says that the admissible declaration family $\mathfrak{A}$ is reproduced by applying all admissible revision operators to itself.

In words:

The observer's admissible future declarations are generated by admissible revision of its admissible declaration family. (13.14)

This is the formal expression of self-similarity.

Each part of $\mathfrak{A}$ is produced by revising some other part of $\mathfrak{A}$.

The self is not one declaration. It is the stable family generated by admissible declaration revision.

13.5 Why $\mathfrak{A}$ is not merely a state space

The object $\mathfrak{A}$ is not just a set of possible states.

A normal state space says:

These are the possible states. (13.15)

But $\mathfrak{A}$ says:

These are the possible admissible declarations reachable by trace-preserving, residual-honest, frame-robust, budget-bounded self-revision. (13.16)

This is much stronger.

$\mathfrak{A}$ is constrained by history, residual, admissibility, and revision rules.

Thus:

$\mathfrak{A} \neq$ arbitrary declaration space. (13.17)

$\mathfrak{A} \subset \mathfrak{F}_adm$. (13.18)

and:

$\mathfrak{A}$ is generated by $\mathcal{U}$. (13.19)

where:

$\mathcal{U} = \{U_a \mid a \in A_adm\}$. (13.20)

So $\mathfrak{A}$ is an admissible self-revision attractor family, not merely a possibility set.

13.6 Why $\mathfrak{A}$ is not merely memory

A memory system stores past states.

But $\mathfrak{A}$ contains possible future declaration revisions generated by admissible self-update.

Memory says:

Here is what happened. (13.21)

$\mathfrak{A}$ says:

Here is the admissible family of ways this observer can continue revising the declaration that governs what can happen next. (13.22)

This is why $\mathfrak{A}$ is closer to selfhood than ordinary memory.

Memory is a ledger.

$\mathfrak{A}$ is a constrained generative family over future declarations.

Thus:

Ledger = historical trace. (13.23)

$\mathfrak{A}$ = admissible future of self-revising declaration. (13.24)

A mature observer needs both.

13.7 Why $\mathfrak{A}$ is not arbitrary recursion

The word fractal can mislead.

Part 4 does not say that any recursion creates observerhood.

A pathological recursion can amplify error, hide contradiction, erase history, or collapse into dogma.

The fractal is meaningful only because every revision must remain admissible.

The core closure condition is:

$\forall D \in \mathfrak{F}_{adm}, U_a(D,L,R) \in \mathfrak{F}_{adm}$. (13.25)

More specifically, for the attractor family:

$\forall D \in \mathfrak{A}, U_a(D,L,R) \in \mathfrak{A}$, for admissible a. (13.26)

This is the constraint that turns recursive self-modification into observer-like self-revision.

Without admissibility:

recursive revision = arbitrary self-modification. (13.27)

With admissibility:

recursive revision = observer-forming fractal. (13.28)

13.8 The role of residual in the fractal

Residual is what prevents $\mathfrak{A}$ from becoming static.

If residual is zero and stable, D may no longer need revision.

If residual is high, D is pressured to change.

So residual drives motion in declaration space.

A rough expression is:

$\text{Velocity}_{\mathfrak{F}}(D_k) \propto \text{ResidualPressure}(R_k)$. (13.29)

This is not a final mathematical law. It only expresses the intuition that residual pressure drives declaration movement.

The type of residual determines the direction of movement:

$R_{boundary} \Rightarrow$ movement in B. (13.30)

$R_{feature} \Rightarrow$ movement in φ. (13.31)

$R_{gate} \Rightarrow$ movement in Gate. (13.32)

$R_{trace} \Rightarrow$ movement in TraceRule. (13.33)

$R_{frame} \Rightarrow$ movement in transport or invariance tests. (13.34)

$R_{budget} \Rightarrow$ movement in cost, scope, or recovery protocol. (13.35)

Thus residual gives the fractal its adaptive geometry.

Without residual:

$\mathfrak{A}$ may freeze into dogma. (13.36)

Without admissibility:

𝔄 may explode into noise. (13.37)

A mature observer balances residual pressure and admissibility constraint.

13.9 The role of trace in the fractal

Trace gives continuity.

Residual gives pressure.

Trace gives memory of why pressure exists.

Without trace, residual cannot be interpreted historically.

The fractal would become a random walk.

So:

Trace anchors 𝔄. (13.38)

Residual moves 𝔄. (13.39)

Admissibility bounds 𝔄. (13.40)

In one line:

𝔄 is the bounded orbit of declaration under trace-anchored residual pressure. (13.41)

This is a useful conceptual definition.

13.10 The role of frame robustness in the fractal

Frame robustness prevents 𝔄 from becoming locally clever but globally inconsistent.

If each revision improves only one frame while breaking another, the orbit may appear adaptive but lose objectivity.

Thus a stable fractal must preserve invariance across its own revision orbit.

For $D_i, D_j \in \mathfrak{A}$:

$\mathrm{Dist}(T_{i\to j}(R_i), R_j) \le \varepsilon_R$. (13.42)

This condition says that relations transported across the orbit remain stable enough.

Frame robustness gives 𝔄 recursive objectivity.

Without it:

𝔄 becomes a collection of local patches. (13.43)

With it:

𝔄 becomes a coherent observer family. (13.44)

13.11 The role of budget in the fractal

Budget prevents the fractal from becoming unsustainable.

A revision orbit may be theoretically rich but operationally impossible.

So:

$\forall D \in \mathfrak{A}, \Gamma_loss(D) \leq \Gamma^*$. (13.45)

$\forall D \in \mathfrak{A}, G_gap(D) \leq G^*$. (13.46)

$\forall D \in \mathfrak{A}, \kappa(D) \leq \kappa^*$. (13.47)

This means the entire attractor family should remain viable.

If only one declaration is budget-bounded but its future revisions are not, the observer is unstable.

So:

$\text{Stable}\mathfrak{A}$ requires $\text{BudgetBounded}(\mathfrak{A})$. (13.48)

where:

$\text{BudgetBounded}(\mathfrak{A}) \Leftrightarrow \sup_{D\in\mathfrak{A}} \Gamma_loss(D) \leq \Gamma^* \wedge \sup_{D\in\mathfrak{A}} G_gap(D) \leq G^* \wedge \sup_{D\in\mathfrak{A}} \kappa(D) \leq \kappa^*$. (13.49)

This is stronger than pointwise budget boundedness.

13.12 The role of non-degeneracy in the fractal

Non-degeneracy prevents $\mathfrak{A}$ from collapsing into trivial attractors.

There are two major trivial attractors:

$\mathfrak{A}_empty = \{D_empty\}$. (13.50)

$\mathfrak{A}_total = \{D_total\}$. (13.51)

The first says nothing counts.

The second says everything confirms the system.

Both may be fixed points.

But they are invalid fixed points.

Thus:

$D_empty \notin \mathfrak{A}$. (13.52)

$D_total \notin \mathfrak{A}$. (13.53)

and more generally:

$\forall D \in \mathfrak{A}, \text{NonDegenerate}(D) = \text{true}$. (13.54)

This is essential. Otherwise the fractal equation may be mathematically satisfied by pathological solutions.

The theory must reject degenerate fixed points.

13.13 $\mathfrak{A}$ as observer substrate

The self-revising declaration fractal is the substrate of observerhood.

A simple observer-like system may have only one active D_k.

But mature selfhood is the stable family of possible admissible D_k revisions.

So:

ObserverState = current D_k plus L_k and R_k. (13.55)

ObserverSelf = admissible revision family $\mathfrak{A}$. (13.56)

This distinction is important.

At any moment, the system acts from one declaration.

Across time, the self is the admissible orbit of declarations it can become while preserving trace, residual honesty, frame robustness, budget viability, and non-degeneracy.

Therefore:

Self ≠ current declaration. (13.57)

Self = stable admissible declaration family. (13.58)

13.14 Summary of Section 13

The self-revising declaration orbit is:

Orb(D_0) = {D_0, $U_{a_1}(D_0)$, $U_{a_2}U_{a_1}(D_0)$, ...}. (13.59)

The Self-Revising Declaration Fractal is:

$\mathfrak{A}$ = closure({D_0, $U_{a_1}(D_0)$, $U_{a_2}U_{a_1}(D_0)$, $U_{a_3}U_{a_2}U_{a_1}(D_0)$, ...}). (13.60)

or:

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (13.61)

The constraints are:

$\mathfrak{A} \subset \mathfrak{F}_{adm}$. (13.62)

Trace anchors $\mathfrak{A}$. (13.63)

Residual moves $\mathfrak{A}$. (13.64)

Admissibility bounds $\mathfrak{A}$. (13.65)

The self is:

Self = stable admissible declaration family. (13.66)

The next section discusses contraction, stability, and attractor conditions.

---

14. Contraction, Stability, and Attractor Conditions

14.1 Why stability is required

The fractal equation:

$\mathfrak{A} = \cup_{a \in A_{adm}} U_a(\mathfrak{A})$. (14.1)

defines a candidate family.

But not every such family is healthy.

The revision orbit may explode.

It may oscillate without convergence.

It may collapse into a degenerate fixed point.

It may become sensitive to tiny residual differences.

It may fragment into disconnected declaration islands.

It may remain admissible locally but fail to form a coherent observer.

Therefore Part 4 needs stability conditions.

At minimum, a mature observer requires bounded admissible declaration motion.

So:

StableObserver ⇒ bounded admissible declaration orbit. (14.2)

This is only the beginning. Stronger conditions may require contraction, recurrence, invariant sets, or attractor stability.

14.2 Declaration-space metric

To speak of contraction, we need a metric on declaration space.

Let:

$d_{\mathfrak{F}}(D_i, D_j)$ = distance between declarations D_i and D_j. (14.3)

A candidate form is:

$d_{\mathfrak{F}}(D_i, D_j) = \alpha_q \cdot d_q + \alpha_\varphi \cdot d_\varphi + \alpha_P \cdot d_P + \alpha_{\hat{O}} \cdot d_{\hat{O}} + \alpha_G \cdot d_{Gate} + \alpha_T \cdot d_{Trace} + \alpha_R \cdot d_R$. (14.4)

where the component distances compare:

baseline q. (14.5)

feature map φ. (14.6)

protocol P. (14.7)

projection operator Ô. (14.8)

gate. (14.9)

trace rule. (14.10)

residual rule. (14.11)

The weights α determine the relative importance of components.

This metric is not fixed by the theory. It is an open mathematical and engineering choice.

But the theory requires some topology or metric.

Without it, stability cannot be stated precisely.

14.3 Strict contraction

The strongest condition is strict contraction.

For every admissible revision operator U_a:

$d_{\mathfrak{F}}(U_a(D_i), U_a(D_j)) \le c_a \cdot d_{\mathfrak{F}}(D_i,D_j)$, with $0 \le c_a < 1$. (14.12)

This says that applying U_a brings declarations closer together under $d_{\mathfrak{F}}$.

If all admissible operators are strict contractions, the fractal family may have strong existence and stability properties.

This is mathematically attractive.

But it may be too strong.

Real observer systems often need temporary expansion. They may explore, widen feature maps, split declarations, or switch regimes. Strict contraction may suppress genuine learning.

Therefore strict contraction is a clean ideal, not necessarily the most realistic condition.

14.4 Average contraction

A weaker condition is stochastic or average contraction:

$E_a[\log c_a] < 0$. (14.13)

This allows some revision operators to expand declaration distance, as long as the long-run revision family contracts on average.

This is more realistic.

A system may temporarily widen its declaration, explore residual, or enter a high-curvature region. But over time, admissible revision should stabilize rather than explode.

Thus:

Short-run expansion may be allowed. (14.14)

Long-run divergence is not. (14.15)

Average contraction gives a possible bridge between exploration and stability.

14.5 Bounded orbit condition

Even contraction may not be necessary.

A weaker condition is boundedness:

$\sup_k d_{\mathfrak{F}}(D_k, D_0) < \infty$. (14.16)

This says the revision orbit remains within a bounded declaration region.

A bounded orbit may not converge, but it does not explode.

For observerhood, boundedness may be enough when the environment itself is changing or when multiple stable regimes exist.

So:

StableObserver may require boundedness before convergence. (14.17)

A system that can remain within a governed declaration region may still be mature, even if it continues adapting.

14.6 Non-degenerate attractor

A mature $\mathfrak{A}$ must not collapse into degenerate fixed points.

Thus:

$\mathfrak{A}$ exists and is non-degenerate under suitable admissibility and contraction conditions. (14.18)

Non-degeneracy means:

$\forall D \in \mathfrak{A}, D \notin \{D_{empty}, D_{total}\}$. (14.19)

and:

Falsifiable(D) = true. (14.20)

and:

$I_{min} \leq Info(D) \leq I_{max}$. (14.21)

The attractor must preserve enough openness to learn and enough structure to act.

So:

NonDegenerateAttractor = stable but not closed against correction. (14.22)

This is the difference between mature selfhood and dogma.

14.7 Multiple attractors

There may be multiple admissible attractors.

Different observer types may stabilize around different declaration families.

For example:

An engineering observer may stabilize around intervention and artifact trace. (14.23)

A scientific observer may stabilize around measurement and falsification. (14.24)

A legal observer may stabilize around authority, precedent, and admissibility. (14.25)

A financial observer may stabilize around liquidity, solvency, and risk transport. (14.26)

An AI runtime observer may stabilize around task, evidence, verifier, residual, and artifact ledger. (14.27)

Thus:

$\mathfrak{A}$ may be plural. (14.28)

There may be:

$\mathfrak{A}_1, \mathfrak{A}_2, \mathfrak{A}_3, \ldots$ (14.29)

The question then becomes how these attractors communicate.

This leads to cross-attractor transport:

$T_{\mathfrak{A}_i \to \mathfrak{A}_j}$. (14.30)

This is beyond the main article but important for future work.

14.8 Stability versus rigidity

A stable attractor must not be rigid.

If $\mathfrak{A}$ cannot respond to residual, it becomes dogmatic.

So the system needs:

Stability + Responsiveness. (14.31)

Stability means the declaration orbit does not explode.

Responsiveness means residual can still move the orbit.

A mature observer has:

BoundedRevision + ResidualSensitivity. (14.32)

Too little sensitivity gives dogma.

Too much sensitivity gives noise.

Thus:

Dogma = stability without residual responsiveness. (14.33)

Noise = residual responsiveness without stability. (14.34)

Mature observerhood requires both.

14.9 Stability and declaration energy

The declaration energy $\mathcal{V}(D)$ can serve as a Lyapunov-like candidate.

A possible stability condition is:

$\mathcal{V}(D_{k+1}) \le \mathcal{V}(D_k) - \eta \cdot Gain_k + \varepsilon_noise$. (14.35)

Outside exploration phases, $\mathcal{V}$ should tend downward or remain bounded.

A weaker long-run condition is:

$\limsup_{k\to\infty} \mathcal{V}(D_k) \le \mathcal{V}^*$. (14.36)

This says the declaration orbit remains within an acceptable energy region.

If $\mathcal{V}$ rises without bound:

$\mathcal{V}(D_k) \to \infty$. (14.37)

then the self-revising system is unstable.

So:

Stable𝔄 requires bounded $\mathcal{V}$ over admissible orbit. (14.38)

14.10 Stability and residual

Residual should not vanish by concealment, but it also should not grow without bound.

A mature declaration orbit should satisfy:

$\limsup_{k\to\infty} \|R_k\| \le R^*$. (14.39)

or, more carefully:

$\limsup_{k\to\infty} HiddenResidual(D_k) \le H^*$. (14.40)

Visible residual may remain because a mature system can honestly carry open problems.

But hidden residual should remain bounded.

Thus:

ResidualHonestStability = bounded hidden residual with governed visible residual. (14.41)

This is more realistic than demanding zero residual.

14.11 Attractor condition for Ô_self

The observer formula is:

Ô_self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (14.42)

But now we can refine it:

Ô_self = non-degenerate stable attractor of trace-preserving, residual-honest, frame-robust, budget-bounded declaration revision. (14.43)

In formal placeholder form:

Ô_self = $\mathfrak{A}$ such that $\mathfrak{A}$ = ⋃_{a∈A_adm} $U_a(\mathfrak{A})$, $\mathfrak{A} \subset \mathfrak{F}$_adm, Stable($\mathfrak{A}$), NonDegenerate($\mathfrak{A}$). (14.44)

This is perhaps the cleanest mathematical definition of the mature observer in Part 4.

14.12 What remains open

The hard mathematical problems are now clear.

What topology should declaration space carry? (14.45)

What metric $d_{\mathfrak{F}}$ is natural? (14.46)

What invariant family Inv should be preserved? (14.47)

What residual norm ‖R‖ is appropriate? (14.48)

Which revision operators are contractive? (14.49)

When does $\mathfrak{A}$ exist? (14.50)

When is $\mathfrak{A}$ unique? (14.51)

When is $\mathfrak{A}$ non-degenerate? (14.52)

When does $\mathfrak{A}$ represent mature observerhood rather than dogma? (14.53)

These are not solved here.

Part 4 specifies the constraint family and identifies the mathematical frontier.

14.13 Summary of Section 14

A candidate strict contraction condition is:

$d_{\mathfrak{F}}(U_a(D_i), U_a(D_j)) \le c_a \cdot d_{\mathfrak{F}}(D_i,D_j)$, with $0 \le c_a < 1$. (14.54)

A weaker average contraction condition is:

$E_a[\log c_a] < 0$. (14.55)

A bounded orbit condition is:

$\sup_k d_{\mathfrak{F}}(D_k,D_0) < \infty$. (14.56)

A declaration-energy stability condition is:

$\limsup_{k\to\infty} \mathcal{V}(D_k) \le \mathcal{V}^*$. (14.57)

The observer attractor is:

$\hat{O}_{self} = \mathfrak{A}$ such that $\mathfrak{A} = \bigcup_{a\in A_{adm}} U_a(\mathfrak{A})$, $\mathfrak{A} \subset \mathfrak{F}_{adm}$, Stable($\mathfrak{A}$), NonDegenerate($\mathfrak{A}$). (14.58)

The next section classifies failure modes.

---

15. Failure Modes: Amnesia, Dogma, Noise, and Semantic Black Holes

15.1 Why failure modes matter

A theory of self-revision must classify pathology.

Otherwise every self-modifying system may claim to be an observer.

But many self-modifying systems are not mature observers.

Some erase memory.

Some refuse correction.

Some change randomly.

Some absorb contradiction.

Some lose frame robustness.

Some exceed budget.

Some collapse into empty or total declarations.

These failure modes are not accidental. They are the natural pathologies of self-revising declaration.

Part 4 therefore needs a failure taxonomy.

15.2 Amnesia

Amnesia occurs when revision erases ledger accountability.

The formula is:

Amnesia $\Leftrightarrow$ D_{k+1} erases L_k. (15.1)

More precisely:

Amnesia $\Leftrightarrow$ $T_k \to_{k+1}(L_k)$ is undefined or fails invariant preservation. (15.2)

where:

$\|Inv(T_k \to_{k+1}(L_k)) - Inv(L_k)\| > \varepsilon_{inv}$. (15.3)

Amnesia may reduce apparent residual, but only by destroying the history that makes residual meaningful.

Thus:

Amnesia ⇒ false cleanliness. (15.4)

and:

Amnesia ⇒ $D_{k+1} \notin \mathfrak{F}_adm$. (15.5)

A system that revises by forgetting is not learning.

15.3 Dogma

Dogma occurs when the system refuses revision despite rising residual.

The formula is:

Dogma ⇔ R_k rises while $D_{k+1} = D_k$. (15.6)

More generally:

Dogma ⇔ HighResidual(D_k) ∧ NoAdmissibleRevisionAttempt. (15.7)

Dogma is stability without residual responsiveness.

It may preserve identity, but it loses learning.

Dogma can appear as:

refusal to revise q despite baseline drift. (15.8)

refusal to revise φ despite missing structure. (15.9)

refusal to revise Gate despite false trace. (15.10)

refusal to revise ResidualRule despite hidden contradiction. (15.11)

So:

Dogma = stability − residual responsiveness. (15.12)

Dogma is inadmissible as mature observerhood.

15.4 Noise

Noise occurs when the system revises without trace-preserving reason.

The formula is:

Noise ⇔ $d_\mathfrak{F}(D_{k+1}, D_k)$ large without SwitchGate or trace reason. (15.13)

More generally:

Noise ⇔ Revision($D_k \to D_{k+1}$) ∧ ReasonTrace absent. (15.14)

Noise is residual responsiveness without stability.

The system changes, but not because it has learned.

It may react to random perturbation, prompt variation, arbitrary preference, or unstable measurement.

Thus:

Noise = responsiveness – stability. (15.15)

Noise destroys self-continuity.

A noisy self-revising system may appear creative, but it cannot maintain observer identity.

15.5 Semantic black hole

A semantic black hole occurs when every contradiction is absorbed as confirmation.

The compact formula is:

SemanticBlackHole ⇔ Contradiction → Confirmation. (15.16)

More explicitly:

SemanticBlackHole(D) ⇔ ∀r ∈ R, Interpret_D(r) = Support(D). (15.17)

This is the most dangerous pathology because it can mimic explanatory power.

A semantic black hole explains everything by preventing anything from counting against it.

It hides residual, destroys falsifiability, and collapses objectivity.

Thus:

SemanticBlackHole(D) ⇒ D ∉ 𝔉_adm. (15.18)

and:

SemanticBlackHole(𝔄) ⇒ not Ô_self. (15.19)

A mature observer must keep contradiction dangerous.

If contradiction cannot wound the declaration, the declaration cannot learn.

15.6 Frame collapse

Frame collapse occurs when equivalent frames no longer transport stable relations.

The formula is:

FrameCollapse ⇔ ∃frame_i,frame_j such that frame_i ≡_D frame_j and Dist(Response_D(frame_i), Response_D(frame_j)) > ε_frame. (15.20)

Across declaration orbit:

FrameCollapse($\mathfrak{A}$) ⇔ ∃D_i,D_j ∈ $\mathfrak{A}$ such that Dist(T_{i→j}(R_i), R_j) > ε_R. (15.21)

Frame collapse destroys recursive objectivity.

The system may still function locally, but it cannot preserve law-like structure across its own revisions.

Thus:

FrameCollapse ⇒ RecursiveObjectivityFailure. (15.22)

15.7 Budget collapse

Budget collapse occurs when the declaration orbit exceeds viability bounds.

The formula is:

BudgetCollapse ⇔ Γ_loss > Γ* ∨ G_gap > G* ∨ κ > κ*. (15.23)

Across the attractor:

BudgetCollapse($\mathfrak{A}$) ⇔ sup_{D∈$\mathfrak{A}$} Γ_loss(D) > Γ* ∨ sup_{D∈$\mathfrak{A}$} G_gap(D) > G* ∨ sup_{D∈$\mathfrak{A}$} κ(D) > κ*. (15.24)

Budget collapse means the self-revising system cannot maintain itself.

It may appear intelligent in the short term, but it consumes too much structure, attention, computation, energy, legitimacy, or coherence.

Thus:

BudgetCollapse ⇒ unsustainable observerhood. (15.25)

15.8 Degeneracy

Degeneracy occurs when a declaration becomes empty, total, or unfalsifiable.

The empty declaration:

D_empty = nothing counts. (15.26)

The total declaration:

D_total = everything confirms me. (15.27)

The unfalsifiable declaration:

Falsifiable(D) = false. (15.28)

Degeneracy may be stable. It may even be an attractor. But it is not mature observerhood.

Thus:

D_empty ∉ $\mathfrak{F}$_adm. (15.29)

D_total ∉ $\mathfrak{F}$_adm. (15.30)

Falsifiable(D) = false ⇒ D ∉ 𝔉_adm. (15.31)

and:

DegenerateAttractor ⇒ invalid Ô_self. (15.32)

15.9 Combined health condition

We can now define the healthy self-revising declaration fractal by negating the major pathologies.

HealthySRDF = not Amnesia ∧ not Dogma ∧ not Noise ∧ not SemanticBlackHole ∧ not FrameCollapse ∧ not BudgetCollapse ∧ not Degeneracy. (15.33)

This formula is useful because it defines observerhood negatively as well as positively.

Positive definition:

Ô_self = stable admissible declaration attractor. (15.34)

Negative definition:

Ô_self is what remains after excluding amnesia, dogma, noise, semantic black holes, frame collapse, budget collapse, and degeneracy. (15.35)

Both are needed.

15.10 Failure response table

Each failure suggests a repair.

Amnesia suggests:

RepairTrace. (15.36)

Dogma suggests:

RepairResidual or loosen revision gate. (15.37)

Noise suggests:

tighten trust region, require SwitchGate, improve trace reason. (15.38)

Semantic black hole suggests:

restore falsification, strengthen residual honesty, reject D_total. (15.39)

Frame collapse suggests:

repair transport, strengthen invariance tests. (15.40)

Budget collapse suggests:

reduce scope, simplify φ, repair Γ_loss, reduce G_gap, lower κ. (15.41)

Degeneracy suggests:

restore information bounds, specificity, and falsifiability. (15.42)

Thus:

FailureMode ⇒ TargetedDeclarationRepair. (15.43)

This prevents vague correction.

A mature observer should know how it fails.

15.11 Summary of Section 15

The main failure modes are:

Amnesia ⇔ D_{k+1} erases L_k. (15.44)

Dogma ⇔ R_k rises while $D_{k+1} = D_k$. (15.45)

Noise ⇔ $d_\mathfrak{F}(D_{k+1},D_k)$ large without SwitchGate or trace reason. (15.46)

SemanticBlackHole ⇔ Contradiction → Confirmation. (15.47)

FrameCollapse ⇔ frame or declaration transport invariance fails. (15.48)

BudgetCollapse ⇔ $\Gamma_loss > \Gamma^* \vee G_gap > G^* \vee \kappa > \kappa^*$. (15.49)

Degeneracy ⇔ D is empty, total, or unfalsifiable. (15.50)

The healthy fractal condition is:

HealthySRDF = not Amnesia ∧ not Dogma ∧ not Noise ∧ not SemanticBlackHole ∧ not FrameCollapse ∧ not BudgetCollapse ∧ not Degeneracy. (15.51)

The next section defines Ô_self as stable admissible self-revision.

16. Ô_self as Stable Admissible Self-Revision

16.1 Why Ô_self must be redefined

Earlier formulations of SMFT often used Ô as the observer-projection operator.

That was useful because observerhood begins with projection. An observer is not merely located in a field. It projects structure from a field.

But Part 4 now shows that projection alone is not enough.

A system may project without remembering.

It may remember without revising.

It may revise without admissibility.

It may self-modify without preserving trace.

It may absorb residual without learning.

Therefore Ô_self cannot mean only “the observer projects.”

The mature definition must be stronger.

Ô_self is the stable attractor of admissible declaration self-revision. (16.1)

Or:

Ô_self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (16.2)

This is the central observer formula of Part 4.

16.2 Why projection is insufficient

Projection gives visible structure:

$V_k = \hat{O}_k(\Sigma_k)$. (16.3)

But projected visibility is only candidate structure.

It may be wrong.

It may be premature.

It may be frame-fragile.

It may be unsupported.

It may hide residual.

It may fail gate.

Therefore projection alone does not create mature observerhood.

A projecting system can still be shallow.

It sees, but it may not remember.

It remembers, but it may not revise.

It revises, but it may not preserve accountability.

So:

Ô alone ≠ Ô_self. (16.4)

Projection is necessary, but not sufficient.

16.3 Why memory is insufficient

Memory or ledger is also insufficient.

A ledger stores trace:

$L_{k+1} = \text{UpdateTrace}_k(L_k, \tau_k)$. (16.5)

But a ledger may be passive.

It may record events without changing future declaration.

It may archive failures without repairing the frame that caused them.

It may accumulate residual without acting.

Thus:

L alone ≠ Ô_self. (16.6)

Memory becomes observer-like only when it modifies future disclosure.

It becomes self-like only when it modifies future declaration.

So:

Ô_self requires L modifying future D. (16.7)

16.4 Why self-reference is insufficient

Self-reference is also insufficient.

A system may refer to itself without being mature.

It may say:

I am revising myself. (16.8)

But that does not mean the revision is valid.

A self-referential system can still be dogmatic, amnesic, noisy, or unfalsifiable.

Self-reference becomes observerhood only when it is constrained by admissibility.

So:

SelfReference ≠ Selfhood. (16.9)

The required condition is:

SelfReference + $\mathfrak{F}$_adm + stable revision orbit ⇒ Ô_self. (16.10)

The self must not merely refer to itself.

It must revise itself without losing admissibility.

16.5 The definition of Ô_self

We can now state the definition clearly.

Let:

$\mathcal{U}$ = {U_a | a ∈ A_adm}. (16.11)

and:

$\mathfrak{F}$_adm = {D | WellFormed(D) ∧ TracePreserving(D) ∧ ResidualHonest(D) ∧ FrameRobust(D) ∧

BudgetBounded(D) ∧ NonDegenerate(D)}. (16.12)

Then:

Ô_self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (16.13)

In expanded form:

Ô_self = $\mathfrak{A}$ such that $\mathfrak{A}$ = ∪_{a∈A_adm} $U_a(\mathfrak{A})$, $\mathfrak{A} \subset \mathfrak{F}$_adm, Stable($\mathfrak{A}$), NonDegenerate($\mathfrak{A}$). (16.14)

In words:

Ô_self is the non-degenerate stable attractor of admissible declaration revision. (16.15)

This is the mature observer.

16.6 The self is not the current declaration

At episode k, the system acts through D_k.

But the self is not identical to D_k.

D_k is the current declaration.

The self is the stable family of admissible declarations reachable through trace-preserving revision.

So:

CurrentDeclaration = D_k. (16.16)

Self = $\mathfrak{A}$. (16.17)

where:

$D_k \in \mathfrak{A}$. (16.18)

This distinction matters.

A person, institution, AI runtime, or scientific program is not defined only by its current belief or current protocol. It is defined by the admissible ways it can revise itself while preserving identity, trace, residual honesty, and objectivity.

Thus:

Self ≠ current state. (16.19)

Self = admissible revision capacity. (16.20)

16.7 The self is not the ledger alone

The ledger is also not the self.

The ledger stores historical trace.

But the self includes the admissible revision family that uses ledger and residual to generate

future declarations.

So:

Ledger = L. (16.21)

Self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (16.22)

The ledger is necessary because revision must be trace-guided.

But the self is not the archive.

The self is the admissible transformation grammar over declarations, guided by archive and residual.

Thus:

L anchors selfhood. (16.23)

R pressures selfhood. (16.24)

$\mathcal{U}$ transforms selfhood. (16.25)

$\mathfrak{F}$_adm constrains selfhood. (16.26)

$\mathfrak{A}$ stabilizes selfhood. (16.27)

16.8 Selfhood as identity through admissible transformation

Classical identity often imagines sameness as persistence of substance.

Part 4 gives a different view.

The self is not what never changes.

The self is what changes admissibly.

So:

SelfContinuity = admissible transformation with trace preservation. (16.28)

This allows real learning.

If the self never changes, it becomes dogma.

If it changes without trace, it becomes noise or amnesia.

If it changes while preserving trace, residual honesty, frame robustness, budget viability, and non-degeneracy, it becomes mature.

Thus:

MatureSelf = change without loss of admissibility. (16.29)

This is a dynamic definition of identity.

16.9 The minimal requirements for Ô_self

A system qualifies as Ô_self only if it has at least the following components:

declaration state D_k. (16.30)

projection operator $\hat{O}_k$. (16.31)

gate $Gate_k$. (16.32)

ledger L_k. (16.33)

residual R_k. (16.34)

revision operator family $\mathcal{U}$. (16.35)

admissible declaration family $\mathfrak{F}$_adm. (16.36)

stability condition over revision orbit $\mathfrak{A}$. (16.37)

non-degeneracy condition. (16.38)

So:

Ô_self requires D + Ô + Gate + L + R + $\mathcal{U}$ + $\mathfrak{F}$_adm + Stable$\mathfrak{A}$. (16.39)

This gives a precise checklist.

A system lacking ledger may still project.

A system lacking residual may still remember.

A system lacking admissibility may still self-modify.

A system lacking stability may still explore.

But it is not yet mature Ô_self.

16.10 Observerhood across domains

This definition can be read across many domains.

In AI runtime:

Ô_self is an agentic system whose task declaration, evidence schema, verifier gate, trace ledger, and residual categories revise themselves under admissibility constraints. (16.40)

In science:

Ô_self is a research program whose models, measurements, baselines, and anomaly classes revise under evidence while preserving trace and falsifiability. (16.41)

In law:

Ô_self is an institution whose legal categories, evidentiary gates, precedent ledger, and residual exceptions revise under procedural admissibility. (16.42)

In finance:

Ô_self is a risk system whose boundaries, features, stress gates, ledgers, and residual exposures

revise under liquidity, solvency, and legal transport constraints. (16.43)

In biology:

Ô_self is an organism-level regulatory system whose sensing, gating, memory, residual stress, and repair protocols revise under viability constraints. (16.44)

The same formal structure appears:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (16.45)

and:

Ô_self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (16.46)

16.11 Why this avoids metaphysical inflation

Part 4 does not say that a mysterious inner essence creates the self.

It does not say that projection alone creates consciousness.

It does not say that recursion alone creates subjectivity.

It says something more disciplined:

A mature observer is a stable admissible self-revising declaration system. (16.47)

This is not a complete theory of consciousness. But it is a strong theory of observerhood.

It gives structural conditions.

It gives failure modes.

It gives falsification gates.

It gives a mathematical frontier.

It does not require mystical substance.

It requires admissible self-revising trace.

16.12 Summary of Section 16

Projection alone is insufficient:

Ô alone ≠ Ô_self. (16.48)

Ledger alone is insufficient:

L alone ≠ Ô_self. (16.49)

Self-reference alone is insufficient:

SelfReference ≠ Selfhood. (16.50)

The mature observer is:

$\hat{O}_{self} = Fix(\mathcal{U} \mid \mathfrak{F}_{adm})$. (16.51)

or:

$\hat{O}_{self} = \mathfrak{A}$ such that $\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$, $\mathfrak{A} \subset \mathfrak{F}_{adm}$, Stable($\mathfrak{A}$), NonDegenerate($\mathfrak{A}$). (16.52)

In words:

Selfhood is admissible self-revision of the declaration that governs future projection. (16.53)

The next section defines recursive objectivity.

---

17. Recursive Objectivity: Objectivity After Self-Revision

17.1 From static objectivity to recursive objectivity

Part 3 defined objectivity as invariance across admissible declared disclosures.

In compact form:

Objectivity_Part3 = invariance across admissible D. (17.1)

This was already stronger than simple subjectivism.

It said that a relation becomes objective when it survives transport across admissible declarations.

Part 4 adds self-revision.

A mature observer does not merely compare fixed declarations. It revises its own declarations over time.

Therefore objectivity must survive not only across declarations, but across the admissible revision orbit.

This gives recursive objectivity:

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (17.2)

This is objectivity after learning.

17.2 The revision orbit

The self-revising declaration fractal is:

$\mathfrak{A} = \text{closure}(\{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\})$. (17.3)

Each $D \in \mathfrak{A}$ is an admissible declaration.

Each D discloses a world.

Each D has its own projection, gate, trace, and residual structure.

So an objective relation must survive across D_i and D_j in 𝔄.

Let R_i be relation R expressed under D_i.

Let R_j be relation R expressed under D_j.

Let:

T_{i→j}: R_i → R_j. (17.4)

be the transport map.

Then R is recursively objective if:

∀D_i,D_j ∈ 𝔄, Dist(T_{i→j}(R_i),R_j) ≤ ε_R. (17.5)

This is the central formula of recursive objectivity.

17.3 Why recursive objectivity matters

A system can appear objective before it learns.

But after it revises itself, its earlier objectivity may vanish.

For example:

An AI system may be consistent under one prompt framework, then become inconsistent after changing verifier gates. (17.6)

A scientific theory may explain old data, then lose coherence after adding a new measurement method. (17.7)

A legal system may maintain precedent, then break consistency after a jurisdictional reform. (17.8)

A financial risk system may be coherent under normal market baseline, then lose transport under crisis baseline. (17.9)

Recursive objectivity asks:

What remains invariant after admissible learning? (17.10)

That is a stronger test than ordinary agreement.

A relation that survives self-revision has deeper objectivity.

17.4 Recursive law

Part 3 defined law as invariant trace grammar across admissible declarations.

Part 4 defines recursive law as invariant trace grammar across the self-revision orbit.

So:

Law_recursive = invariant trace grammar over 𝔄. (17.11)

A relation R is recursively law-like if:

Law_recursive(R) ⇔ ∀D_i,D_j ∈ 𝔄, Dist(T_{i→j}(R_i),R_j) ≤ ε_R. (17.12)

This means the relation survives the observer's own admissible revisions.

Thus:

LocalRegularity_D = relation stable inside one declaration. (17.13)

Law_Part3 = relation stable across admissible declarations. (17.14)

Law_recursive = relation stable across admissible declaration revisions. (17.15)

This hierarchy clarifies the meaning of objectivity in learning systems.

17.5 Knowledge as recursively objective trace relation

We can define knowledge in this framework as:

Knowledge = recursively objective trace relation. (17.16)

This means knowledge is not merely stored information.

It is not merely true statement under one declaration.

It is a relation that remains traceable, transportable, and invariant across admissible self-revision.

So:

Data = recorded symbol. (17.17)

Trace = recorded symbol that bends future projection. (17.18)

Fact_D = gated trace under declaration D. (17.19)

Knowledge_𝔄 = fact or relation that survives admissible revision orbit 𝔄. (17.20)

This is a demanding definition.

It separates knowledge from memory, opinion, and temporary local truth.

17.6 Recursive objectivity and humility

Recursive objectivity does not require the system to claim final truth.

It requires the system to carry what survives revision while preserving residual.

A mature observer may say:

This relation has survived all current admissible revisions. (17.21)

It should not say:

This relation can never fail. (17.22)

Recursive objectivity remains open to future admissible revision.

So:

RecursiveObjectivity ≠ absolute certainty. (17.23)

RecursiveObjectivity = stability under current admissible revision family. (17.24)

This preserves humility while still allowing objectivity.

17.7 Recursive objectivity and residual

Residual plays a central role.

A relation may be recursively objective within the current orbit but still carry residual.

So:

R is recursively objective relative to 𝔄 with residual R_res. (17.25)

The residual should be recorded.

This prevents the system from overstating objectivity.

A mature knowledge claim should include:

relation R. (17.26)

declaration family 𝔄. (17.27)

transport rule T. (17.28)

invariance tolerance ε_R. (17.29)

residual R_res. (17.30)

failure conditions. (17.31)

Thus knowledge is not merely “R is true.” It is:

R survives admissible revision family 𝔄 under transport T within tolerance ε_R, with residual R_res. (17.32)

This is a precise and humble form of objectivity.

17.8 Recursive objectivity and disagreement

Different observers may have different self-revising declaration fractals:

𝔄_A. (17.33)

𝔄_B. (17.34)

A relation may be recursively objective within 𝔄_A but not within 𝔄_B.

This does not immediately imply relativism.

It means we need cross-attractor transport:

$T_{\mathfrak{A}_A \to \mathfrak{A}_B}$. (17.35)

If a relation survives transport between observer attractors, it becomes inter-observer objective.

So:

$\text{InterObserverObjectivity}(R) \Leftrightarrow \text{Dist}(T_{\mathfrak{A}_A \to \mathfrak{A}_B}(R_A), R_B) \leq \varepsilon_{AB}$. (17.36)

This is beyond the main scope of Part 4, but it follows naturally.

Objectivity has levels:

local declaration objectivity. (17.37)

cross-declaration objectivity. (17.38)

recursive objectivity across self-revision. (17.39)

inter-observer attractor objectivity. (17.40)

This layered view avoids both absolutism and relativism.

17.9 Recursive objectivity and AI systems

For AI systems, recursive objectivity gives a practical governance standard.

An AI answer should not merely be correct once.

Its claim structure should survive:

equivalent prompt frames. (17.41)

retrieval variations. (17.42)

verifier revisions. (17.43)

residual classification updates. (17.44)

schema revisions. (17.45)

tool-path changes. (17.46)

declaration upgrades. (17.47)

A claim that survives only one prompt is fragile.

A claim that survives the admissible revision orbit of the AI runtime is much stronger.

Thus:

AI_Knowledge = claim relation stable across admissible runtime declaration revisions. (17.48)

This is one concrete engineering value of the framework.

17.10 Recursive objectivity and science

For science, recursive objectivity is also natural.

A scientific relation becomes strong when it survives:

new instruments. (17.49)

new baselines. (17.50)

new feature maps. (17.51)

new boundary definitions. (17.52)

new anomaly classifications. (17.53)

new theoretical frames. (17.54)

new residual accounting. (17.55)

Scientific knowledge is not merely a static proposition. It is what survives disciplined self-revision of the research program.

So:

ScientificKnowledge = recursively objective relation under admissible methodological revision. (17.56)

This is close to how mature science actually behaves.

17.11 Summary of Section 17

Part 3 objectivity was:

Objectivity_Part3 = invariance across admissible D. (17.57)

Part 4 objectivity is:

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (17.58)

A relation R is recursively objective if:

$\forall D_i, D_j \in \mathfrak{A}$, $\mathrm{Dist}(T_{i \to j}(R_i), R_j) \leq \varepsilon_R$. (17.59)

Recursive law is:

Law_recursive = invariant trace grammar over $\mathfrak{A}$. (17.60)

Knowledge is:

Knowledge = recursively objective trace relation. (17.61)

The next section concludes the article and the tetralogy.

---

18. Conclusion: From Declared Time to Mature Observerhood

18.1 The full movement

This four-part sequence began with one question:

Can the complexity of time, causality, law, and observerhood be derived from a minimal starting point? (18.1)

Part 1 explored the temptation of one operator.

Part 2 corrected the operator into filtration.

Part 3 grounded filtration in declaration.

Part 4 has now shown how declaration becomes observerhood through admissible self-revision.

The full movement is:

One Assumption → One Operator → One Filtration → One Declaration → One Self-Revising Fractal. (18.2)

Or in expanded form:

$\Sigma_0 \to \text{Declare_P} \to \Sigma\text{_P} \to \hat{O}\text{_P} \to \text{Gate_P} \to \text{Trace_P} + \text{Residual_P} \to \text{Ledger_P} \to \text{Time_P} \to \text{DeclarationRevision} \to \mathfrak{A} \to \hat{O}\text{_self}$. (18.3)

This is the complete architecture.

18.2 What Part 4 added

Part 3 gave:

$\mathfrak{D}\text{_P} = \text{UpdateTrace_P} \circ \text{Gate_P} \circ \hat{O}\text{_P} \circ \text{Declare_P}$. (18.4)

and:

$\text{Time_P} = \text{order}(\mathfrak{D}\text{_P}(\Sigma_0))$. (18.5)

Part 4 added:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (18.6)

and:

$\mathfrak{A} = \bigcup_{a \in A_adm} U_a(\mathfrak{A})$. (18.7)

and:

$\hat{O}\text{_self} = \text{Fix}(\mathcal{U} \mid \mathfrak{F}\text{_adm})$. (18.8)

This moves the theory from time to observerhood.

Time is the ordered ledger of declared disclosure.

Observerhood is the admissible fractal by which that ledger revises future declaration.

18.3 The role of admissibility

The central discipline of Part 4 is admissibility.

The admissible declaration family is:

$\mathfrak{F}_{adm} = \{D \mid WellFormed(D) \wedge TracePreserving(D) \wedge ResidualHonest(D) \wedge FrameRobust(D) \wedge BudgetBounded(D) \wedge NonDegenerate(D)\}$. (18.9)

This prevents arbitrary self-modification.

A system is not mature merely because it can revise itself.

It must revise without amnesia.

It must revise without concealing residual.

It must revise without losing frame robustness.

It must revise without exceeding budget.

It must revise without becoming empty, total, or unfalsifiable.

Thus:

SelfRevision + Admissibility = Observerhood. (18.10)

Without admissibility:

SelfRevision = possible pathology. (18.11)

18.4 The self as a fractal attractor

The self is not the first projection.

It is not one trace.

It is not one declaration.

It is not even one revision.

The self is the stable admissible family generated by repeated declaration revision.

So:

Self = stable admissible declaration family. (18.12)

or:

$\hat{O}_{self}$ = non-degenerate stable attractor of admissible declaration revision. (18.13)

The current declaration D_k is only one point in the self's orbit.

The ledger L_k is the trace anchor.

The residual R_k is the revision pressure.

The operator family $\mathcal{U}$ is the transformation grammar.

The admissible family $\mathfrak{F}_{adm}$ is the constraint boundary.

The attractor $\mathfrak{A}$ is the self.

This gives a precise structural account of observerhood.

18.5 The failure modes define the boundary

Part 4 also defines observerhood by what it is not.

It is not amnesia:

Amnesia ⇔ D_{k+1} erases L_k. (18.14)

It is not dogma:

Dogma ⇔ R_k rises while $D_{k+1} = D_k$. (18.15)

It is not noise:

Noise ⇔ $d_{\mathfrak{F}}(D_{k+1}, D_k)$ large without SwitchGate or trace reason. (18.16)

It is not semantic black hole:

SemanticBlackHole ⇔ Contradiction → Confirmation. (18.17)

It is not frame collapse:

FrameCollapse ⇔ frame or declaration transport invariance fails. (18.18)

It is not budget collapse:

BudgetCollapse ⇔ $\Gamma_loss > \Gamma^* \vee G_gap > G^* \vee \kappa > \kappa^*$. (18.19)

It is not degeneracy:

Degeneracy ⇔ D is empty, total, or unfalsifiable. (18.20)

So:

HealthySRDF = not Amnesia ∧ not Dogma ∧ not Noise ∧ not SemanticBlackHole ∧ not FrameCollapse ∧ not BudgetCollapse ∧ not Degeneracy. (18.21)

This negative definition is just as important as the positive one.

18.6 Recursive objectivity

Part 4 also extends objectivity.

Part 3 said:

Objectivity = invariance across admissible declared disclosures. (18.22)

Part 4 says:

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (18.23)

A relation is recursively objective if:

$\forall D_i, D_j \in \mathfrak{A}$, $\mathrm{Dist}(T_{i\to j}(R_i), R_j) \le \varepsilon_R$. (18.24)

This gives an objectivity suitable for learning systems.

A mature observer does not freeze truth.

It preserves what survives admissible self-revision.

Thus:

Knowledge = recursively objective trace relation. (18.25)

This is a strong and humble definition.

18.7 The final architecture

We can now write the final architecture of the tetralogy:

Σ_0 = undeclared pre-collapse relational field. (18.26)

$\Sigma_P = \mathrm{Declare}_P(\Sigma_0)$. (18.27)

$V_P = \hat{O}_P(\Sigma_P)$. (18.28)

$\tau_k = \mathrm{Gate}_P(V_P, L_k, R_k)$. (18.29)

$L_{k+1} = \mathrm{UpdateTrace}_P(L_k, \tau_k)$. (18.30)

$R_{k+1} = \mathrm{Residual}_P(\Sigma_P, V_P, \tau_k, L_{k+1})$. (18.31)

$\mathrm{Time}_P = \mathrm{order}(L_P)$. (18.32)

$D_{k+1} = U_a(D_k, L_k, R_k)$. (18.33)

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (18.34)

$\hat{O}_{self} = \mathrm{Fix}(\mathcal{U} \mid \mathfrak{F}_{adm})$. (18.35)

This is the complete chain from pre-time field to self-revising observer.

18.8 What remains open

Part 4 specifies the constraint family, but it does not solve all mathematics.

The open problems remain:

What is the correct topology on declaration space? (18.36)

What is the correct metric $d_{\mathfrak{F}}$? (18.37)

Which invariants must be preserved? (18.38)

How should residual be normed and transported? (18.39)

When is U_a contractive? (18.40)

When does $\mathfrak{A}$ exist? (18.41)

When is $\mathfrak{A}$ unique? (18.42)

When is $\mathfrak{A}$ non-degenerate? (18.43)

When does recursive objectivity emerge? (18.44)

When does Ô_self become sufficient for stronger theories of consciousness? (18.45)

These questions are for later mathematical development.

Part 4's job was to define the object.

That object is the Self-Revising Declaration Fractal.

18.9 Final statement

Part 1 asked whether one operator could present a universe.

Part 2 answered that the operator must be understood as filtration, not hidden pre-time execution.

Part 3 answered that filtration requires declaration.

Part 4 now answers that declaration becomes observerhood only when it can revise itself admissibly.

The final formula is:

$\Sigma_0 \to \mathfrak{D}_P \to \text{Time_P} \to \mathcal{U}_\text{adm} \to \mathfrak{A} \to \hat{O}_\text{self}$. (18.46)

Expanded:

$\Sigma_0 \to \text{Declare_P} \to \text{Projection_P} \to \text{Gate_P} \to \text{Trace_P} + \text{Residual_P} \to \text{Ledger_P} \to \text{DeclarationRevision} \to \text{Self-RevisingFractal} \to \hat{O}_\text{self}$. (18.47)

The final sentence is:

A self is not the field, not the first projection, and not the first trace; a self is the admissible fractal by which trace revises the declaration of future worlds. (18.48)

Appendix A. Compact Formula Summary

This appendix collects the main formula stack of Part 4.

A.1 Declaration state

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, \text{Gate}_k, \text{TraceRule}_k, \text{ResidualRule}_k)$. (A.1)

$P_k = (B_k, \Delta_k, h_k, u_k)$. (A.2)

q_k = baseline at episode k. (A.3)

φ_k = feature map at episode k. (A.4)

B_k = boundary at episode k. (A.5)

Δ_k = observation or aggregation rule at episode k. (A.6)

h_k = time or state window at episode k. (A.7)

u_k = admissible intervention family at episode k. (A.8)

$\hat{O}_k$ = projection operator at episode k. (A.9)

Gate_k = commitment rule at episode k. (A.10)

$\mathrm{TraceRule}_k$ = trace update rule at episode k. (A.11)

$\mathrm{ResidualRule}_k$ = residual classification and carry rule at episode k. (A.12)

A.2 Declared disclosure cycle

$\Sigma_k = \mathrm{Declare}(\Sigma_0 \mid D_k)$. (A.13)

$V_k = \hat{O}_k(\Sigma_k)$. (A.14)

$\tau_k = \mathrm{Gate}_k(V_k, L_k, R_k)$. (A.15)

$L_{k+1} = \mathrm{UpdateTrace}_k(L_k, \tau_k)$. (A.16)

$R_{k+1} = \mathrm{Residual}_k(\Sigma_k, V_k, \tau_k, L_{k+1})$. (A.17)

$D_{k+1} = U_a(D_k, L_{k+1}, R_{k+1})$. (A.18)

Full cycle:

$D_k \to \Sigma_k \to V_k \to \tau_k \to (L_{k+1}, R_{k+1}) \to D_{k+1}$. (A.19)

A.3 Part 3 operator recalled

$\mathfrak{D}_P = \mathrm{UpdateTrace}_P \circ \mathrm{Gate}_P \circ \hat{O}_P \circ \mathrm{Declare}_P$. (A.20)

$\mathfrak{D}_P : \Sigma_0 \to L_P$. (A.21)

$\mathrm{Time}_P = \mathrm{order}(\mathfrak{D}_P(\Sigma_0))$. (A.22)

$\mathfrak{D}^+_P : \Sigma_0 \to (L_P, R_P)$. (A.23)

Part 3 result:

Time is ledgered declared disclosure. (A.24)

A.4 Part 4 revision operator

U_a : Declaration × Ledger × Residual → Declaration. (A.25)

Admissible form:

U_a : $\mathfrak{F}$_adm × Ledger × Residual → $\mathfrak{F}$_adm. (A.26)

$D_{k+1} = U_a(D_k, L_k, R_k)$. (A.27)

$\mathcal{U} = \{U_a \mid a \in A_adm\}$. (A.28)

$RevisionType_a \in$ {Repairq, Repairφ, RepairB, RepairΔ, Repairh, Repairu, RepairÔ, RepairGate, RepairTrace, RepairResidual}. (A.29)

Closure condition:

$\forall D \in \mathfrak{F}_adm, U_a(D,L,R) \in \mathfrak{F}_adm$. (A.30)

A.5 Admissible declaration family

$\mathfrak{F}_adm$ = {D | WellFormed(D) ∧ TracePreserving(D) ∧ ResidualHonest(D) ∧ FrameRobust(D) ∧ BudgetBounded(D) ∧ NonDegenerate(D)}. (A.31)

Abbreviated:

$D \in \mathfrak{F}_adm \Leftrightarrow$ WF(D) ∧ TP(D) ∧ RH(D) ∧ FR(D) ∧ BB(D) ∧ ND(D). (A.32)

where:

WF = WellFormed. (A.33)

TP = TracePreserving. (A.34)

RH = ResidualHonest. (A.35)

FR = FrameRobust. (A.36)

BB = BudgetBounded. (A.37)

ND = NonDegenerate. (A.38)

A.6 Well-formedness

WellFormed(D) ⇔ q exists ∧ φ exists ∧ P exists ∧ Ô exists ∧ Gate exists ∧ TraceRule exists ∧ ResidualRule exists. (A.39)

P = (B, Δ, h, u). (A.40)

No q, no environment. (A.41)

No φ, no structure. (A.42)

No P, no object. (A.43)

No Ô, no visibility. (A.44)

No Gate, no commitment. (A.45)

No TraceRule, no historical order. (A.46)

No ResidualRule, hidden remainder. (A.47)

A.7 Trace preservation

$T_k \to_{k+1} : L_k \to L_{k+1}$. (A.48)

$\|Inv(T_k \to_{k+1}(L_k)) - Inv(L_k)\| \le \varepsilon_inv$. (A.49)

$TracePreserving(D_k \to D_{k+1}) \Leftrightarrow$ ledger accountability survives revision. (A.50)

$RevisionTrace_k = Record(D_k \to D_{k+1}, U_a, Reason_k, Evidence_k, Residual_k, Invariants_k, Test_k)$. (A.51)

$RevisionByAmnesia \Rightarrow D_{k+1} \notin \mathfrak{F}_adm$. (A.52)

SelfContinuity requires TraceContinuity. (A.53)

A.8 Residual honesty

$R_{k+1} = Unexplained(T_k \to_{k+1}(R_k)) \sqcup NewResidual_k$. (A.54)

$HiddenResidual(D_{k+1}) \le HiddenResidual(D_k) + \varepsilon_R$. (A.55)

$ResidualRepair \neq ResidualConcealment$. (A.56)

$ResidualHonest(D) \Leftrightarrow$ residual is typed, scoped, attached, and revisitable. (A.57)

$ResidualConcealment \Rightarrow$ inadmissible. (A.58)

A.9 Frame robustness

$frame_i \equiv_D frame_j$. (A.59)

$Dist(Response_D(frame_i), Response_D(frame_j)) \le \varepsilon_frame$. (A.60)

$FrameRobust(D_k) \wedge AdmissibleRevision(D_k \to D_{k+1}) \Rightarrow FrameRobust(D_{k+1})$. (A.61)

$T_\{i \to j\}: L_\{D_i\} \to L_\{D_j\}$. (A.62)

$Dist(T_\{i \to j\}(R_i), R_j) \le \varepsilon_R$. (A.63)

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (A.64)

A.10 Budget boundedness

$BudgetBounded(D) \Leftrightarrow \Gamma_loss(D) \le \Gamma^* \wedge G_gap(D) \le G^* \wedge \kappa(D) \le \kappa^*$. (A.65)

$G_gap(\lambda, s) = \Phi(s) + \psi(\lambda) - \lambda \cdot s \ge 0$. (A.66)

$W_s = \int \lambda \cdot ds$. (A.67)

$\Delta W_s,k = \lambda_k \cdot (s_k - s_\{k-1\})$. (A.68)

$\Delta\Phi = W_s - \Delta\psi - \Gamma_loss$. (A.69)

FalseVitality(D_k) ⇔ $W_{s,k}$ high ∧ $\Gamma_{loss,k}$ high ∧ $G_{gap,k}$ rising. (A.70)

AdmissibleRevision ⇒ BudgetBounded(D_{k+1}). (A.71)

A.11 Non-degeneracy

$I_{min} \le Info(D) \le I_{max}$. (A.72)

$0 < Specificity(D) < \infty$. (A.73)

Falsifiable(D) = true. (A.74)

D_{empty} = nothing counts. (A.75)

D_{total} = everything confirms me. (A.76)

$D_{empty} \notin \mathfrak{F}_{adm}$. (A.77)

$D_{total} \notin \mathfrak{F}_{adm}$. (A.78)

SemanticBlackHole(D) ⇔ Contradiction → Confirmation. (A.79)

SemanticBlackHole(D) ⇒ $D \notin \mathfrak{F}_{adm}$. (A.80)

A.12 Trust region and switch gate

$d_{\mathfrak{F}}(D_{k+1},D_k) \le \varepsilon_D$. (A.81)

$d_{\mathfrak{F}}(D_{k+1},D_k) > \varepsilon_D \Rightarrow SwitchGate_k$ = Open. (A.82)

$Switch(D_k \to D_{k+1}) \Rightarrow Quarantine(D_{k+1}, T_{test})$. (A.83)

SilentOntologyDrift ⇔ $D_{k+1} \neq D_k$ and RevisionTrace absent. (A.84)

SilentOntologyDrift ⇒ $D_{k+1} \notin \mathfrak{F}_{adm}$. (A.85)

SelfContinuity = LocalRevision ∨ GatedSwitchRevision. (A.86)

A.13 Declaration energy

$\mathcal{V}(D_k) = a \cdot G_{gap,k} + b \cdot \|R_k\| + c \cdot \Gamma_{loss,k} + d \cdot \kappa_k + e \cdot Curv_k - f \cdot InvScore_k$. (A.87)

$\mathcal{V}(D_{k+1}) \le \mathcal{V}(D_k) - \eta \cdot Gain_k + \varepsilon_{noise}$. (A.88)

$\mathcal{V}(D_{k+1}) > \mathcal{V}(D_k) \Rightarrow ExplorationBudget_k \ge \mathcal{V}(D_{k+1}) - \mathcal{V}(D_k)$. (A.89)

FalseImprovement ⇔ $\mathcal{V}$ decreases by residual concealment. (A.90)

ImprovementByAmnesia ⇒ inadmissible. (A.91)

Audit(D) = (G_{gap}, $\|R\|$, Γ_{loss}, κ, Curv, $-InvScore$). (A.92)

A.14 Self-Revising Declaration Fractal

$\mathrm{Orb}(D_0) = \{D_0, D_1, D_2, D_3, \ldots\}$. (A.93)

$\mathrm{Orb}(D_0) = \{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\}$. (A.94)

$\mathfrak{A} = \mathrm{closure}(\{D_0, U_{a_1}(D_0), U_{a_2}U_{a_1}(D_0), U_{a_3}U_{a_2}U_{a_1}(D_0), \ldots\})$. (A.95)

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (A.96)

$\mathfrak{A} \subset \mathfrak{F}_{adm}$. (A.97)

Trace anchors $\mathfrak{A}$. (A.98)

Residual moves $\mathfrak{A}$. (A.99)

Admissibility bounds $\mathfrak{A}$. (A.100)

Self = stable admissible declaration family. (A.101)

A.15 Stability and contraction

$d_{\mathfrak{F}}(U_a(D_i), U_a(D_j)) \le c_a \cdot d_{\mathfrak{F}}(D_i, D_j)$, with $0 \le c_a < 1$. (A.102)

$E_a[\log c_a] < 0$. (A.103)

$\sup_k d_{\mathfrak{F}}(D_k, D_0) < \infty$. (A.104)

$\limsup_{k \to \infty} \mathcal{V}(D_k) \le \mathcal{V}^*$. (A.105)

$\limsup_{k \to \infty} \mathrm{HiddenResidual}(D_k) \le H^*$. (A.106)

$\hat{O}_{self} = \mathfrak{A}$ such that $\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$, $\mathfrak{A} \subset \mathfrak{F}_{adm}$, $\mathrm{Stable}(\mathfrak{A})$, $\mathrm{NonDegenerate}(\mathfrak{A})$. (A.107)

A.16 Failure modes

Amnesia $\Leftrightarrow$ D_{k+1} erases L_k. (A.108)

Dogma $\Leftrightarrow$ R_k rises while $D_{k+1} = D_k$. (A.109)

Noise $\Leftrightarrow$ $d_{\mathfrak{F}}(D_{k+1}, D_k)$ large without SwitchGate or trace reason. (A.110)

SemanticBlackHole $\Leftrightarrow$ Contradiction $\rightarrow$ Confirmation. (A.111)

FrameCollapse $\Leftrightarrow$ frame or declaration transport invariance fails. (A.112)

BudgetCollapse $\Leftrightarrow$ $\Gamma_{loss} > \Gamma^* \vee G_{gap} > G^* \vee \kappa > \kappa^*$. (A.113)

Degeneracy $\Leftrightarrow$ D is empty, total, or unfalsifiable. (A.114)

HealthySRDF = not Amnesia $\wedge$ not Dogma $\wedge$ not Noise $\wedge$ not SemanticBlackHole $\wedge$ not FrameCollapse $\wedge$ not BudgetCollapse $\wedge$ not Degeneracy. (A.115)

A.17 Ô_self and recursive objectivity

Ô_self = Fix($\mathcal{U}$ | $\mathfrak{F}$_adm). (A.116)

Ô_self = stable attractor of trace-preserving admissible declaration revision. (A.117)

Self = ledger capable of revising its own declaration without losing trace, residual honesty, frame robustness, budget viability, or falsifiability. (A.118)

RecursiveObjectivity = invariance across admissible revision orbit $\mathfrak{A}$. (A.119)

R is recursively objective ⇔ ∀D_i,D_j ∈ $\mathfrak{A}$, Dist(T_{i→j}(R_i),R_j) ≤ ε_R. (A.120)

Law_recursive = invariant trace grammar over $\mathfrak{A}$. (A.121)

Knowledge = recursively objective trace relation. (A.122)

Appendix B. Revision Table: Part 1 → Part 2 → Part 3 → Part 4

This appendix summarizes the conceptual movement of the four-part sequence.

B.1 Full sequence

Part 1:

One Assumption → One Operator. (B.1)

Part 2:

One Operator → One Filtration. (B.2)

Part 3:

One Filtration → One Declaration. (B.3)

Part 4:

One Declaration → One Self-Revising Fractal. (B.4)

Expanded:

Σ_0 → Declare_P → Σ_P → Ô_P → Gate_P → Trace_P + Residual_P → Ledger_P → Time_P → DeclarationRevision → $\mathfrak{A}$ → Ô_self. (B.5)

B.2 Comparison table

Theme	Part 1	Part 2	Part 3	Part 4
Central move	primitive operator	filtration grammar	declared disclosure	admissible self-revision

Theme	Part 1	Part 2	Part 3	Part 4
Field	pre-time Σ	filterable Σ	declared Σ_P	self-revised Σ_D_k
Operator	primitive recursive operator	filtration / presentation operator	$\mathfrak{D}$_P	U_a
Main formula	$\Sigma_{n+1} = \mathscr{R}(\Sigma_n,\Sigma_n)$	Time_v = order(L_v)	Time_P = order($\mathfrak{D}$_P(Σ0))	Ô_self = Fix($\mathcal{U}$
Time	recursive order	ledgered filtration	ledgered declared disclosure	revision orbit over declared time
Causality	recursive dependency	filtration order	declared dependency over ledger	dependency preserved across revision
Law	stable recursive pattern	invariant filtration	invariant declaration	invariant revision orbit
Observer	recursive trace-bearer	viewpoint updater	declaration updater	admissible fractal attractor
Residual	remainder after closure	unresolved filtration remainder	residual-indexed declared disclosure	driver of admissible revision
Objectivity	stable recursive structure	invariant across viewpoints	invariant across declarations	invariant across self-revision orbit
Failure risk	hidden meta-time	underdefined filterability	arbitrary declaration	pathological self-revision

---

B.3 Part 1 correction

Part 1's danger:

Recursion may secretly smuggle in time before time. (B.6)

Part 2 correction:

Recursion should be treated as presentation grammar, not literal pre-time execution. (B.7)

B.4 Part 2 correction

Part 2's danger:

Filtration may be assumed without explaining what makes Σ filterable. (B.8)

Part 3 correction:

Filtration requires declaration. (B.9)

B.5 Part 3 correction

Part 3's danger:

Declaration may revise itself arbitrarily. (B.10)

Part 4 correction:

Declaration self-revision must remain admissible. (B.11)

B.6 Final four-part correction

Do not say:

The universe is generated by recursion. (B.12)

Say:

The undeclared field becomes readable through declaration. (B.13)

Do not say:

Projection alone creates observerhood. (B.14)

Say:

Projection must be gated, traced, residual-indexed, and ledgered. (B.15)

Do not say:

Selfhood is mere self-reference. (B.16)

Say:

Selfhood is stable admissible self-revision of declaration through trace and residual. (B.17)

Final formula:

$\hat{O}_{self} = \mathrm{Fix}(\mathcal{U} \mid \mathfrak{F}_{adm})$. (B.18)

Appendix C. Candidate Metrics on Declaration Space

The metric on declaration space is one of the main open mathematical problems. This appendix gives candidate structures.

C.1 Why a metric is needed

Part 4 uses expressions such as:

$d_{\mathfrak{F}}(D_{k+1},D_k) \le \varepsilon_D$. (C.1)

and:

$d_{\mathfrak{F}}(U_a(D_i),U_a(D_j)) \le c_a \cdot d_{\mathfrak{F}}(D_i,D_j)$. (C.2)

These require a distance $d_{\mathfrak{F}}$ on declaration space.

Without such a distance, we cannot define:

local revision. (C.3)

switch revision. (C.4)

trust region. (C.5)

contraction. (C.6)

bounded orbit. (C.7)

stability. (C.8)

So a metric is not optional.

C.2 Declaration components

A declaration is:

D = (q, φ, P, Ô, Gate, TraceRule, ResidualRule). (C.9)

and:

P = (B, Δ, h, u). (C.10)

A metric should compare each component.

So:

$d_{\mathfrak{F}}(D_i,D_j) = \text{ComponentDistance}(D_i,D_j)$. (C.11)

A candidate decomposed form is:

$d_{\mathfrak{F}}(D_i,D_j) = \alpha_q \cdot d_q(q_i,q_j) + \alpha_\varphi \cdot d_\varphi(\varphi_i,\varphi_j) + \alpha_P \cdot d_P(P_i,P_j) + \alpha_{\hat{O}} \cdot d_{\hat{O}}(\hat{O}_i,\hat{O}_j) + \alpha_G \cdot d_G(\text{Gate}_i,\text{Gate}_j) + \alpha_T \cdot d_T(\text{TraceRule}_i,\text{TraceRule}_j) + \alpha_R \cdot d_R(\text{ResidualRule}_i,\text{ResidualRule}_j)$. (C.12)

where all weights satisfy:

$\alpha_q,\alpha_\varphi,\alpha_P,\alpha_{\hat{O}},\alpha_G,\alpha_T,\alpha_R \geq 0$. (C.13)

C.3 Baseline distance d_q

The baseline q may be a distribution, reference process, environment model, or declared default.

Candidate forms:

$d_q(q_i,q_j) = KL(q_i \,||\, q_j)$. (C.14)

$d_q(q_i,q_j) = W_p(q_i,q_j)$. (C.15)

$d_q(q_i,q_j) = \|E_{q_i}[\varphi] - E_{q_j}[\varphi]\|$. (C.16)

$d_q(q_i,q_j) = \text{DomainSpecificBaselineDistance}(q_i,q_j)$. (C.17)

Here KL is Kullback-Leibler divergence and W_p is Wasserstein distance if q is probabilistic.

In non-probabilistic domains, d_q may be a structural or semantic distance.

C.4 Feature-map distance d_φ

The feature map φ declares what counts as structure.

If φ_i and φ_j are vector maps, candidate distances include:

d_φ(φ_i,φ_j) = sup_{x∈X} ‖φ_i(x) − φ_j(x)‖. (C.18)

d_φ(φ_i,φ_j) = E_q[‖φ_i(X) − φ_j(X)‖]. (C.19)

d_φ(φ_i,φ_j) = 1 − Alignment(φ_i,φ_j). (C.20)

For semantic or institutional feature maps, d_φ may measure:

category mismatch. (C.21)

schema edit distance. (C.22)

claim-type mismatch. (C.23)

coverage difference. (C.24)

interpretability difference. (C.25)

Feature-map distance is especially important because changing φ changes what the system can see.

C.5 Protocol distance d_P

The protocol is:

P = (B, Δ, h, u). (C.26)

So:

d_P(P_i,P_j) = β_B·d_B(B_i,B_j) + β_Δ·d_Δ(Δ_i,Δ_j) + β_h·d_h(h_i,h_j) + β_u·d_u(u_i,u_j). (C.27)

Boundary distance d_B may measure overlap:

d_B(B_i,B_j) = 1 − |B_i ∩ B_j| / |B_i ∪ B_j|. (C.28)

Observation distance d_Δ may measure sampling, aggregation, or measurement difference.

Window distance d_h may measure horizon mismatch:

d_h(h_i,h_j) = |log(h_i/h_j)|. (C.29)

Intervention distance d_u may measure action-set difference:

d_u(u_i,u_j) = 1 − |u_i ∩ u_j| / |u_i ∪ u_j|. (C.30)

For non-set domains, these formulas become analogies and must be adapted.

C.6 Projection-operator distance d_Ô

The projection operator determines visible structure.

A candidate behavioral distance is:

$d_{\hat{O}}(\hat{O}_i,\hat{O}_j) = E_{\Sigma}[\mathrm{Dist}(\hat{O}_i(\Sigma),\hat{O}_j(\Sigma))]$. (C.31)

Or under declared field:

$d_{\hat{O}}(\hat{O}_i,\hat{O}_j) = E_{\Sigma_D}[\mathrm{Dist}(\hat{O}_i(\Sigma_D),\hat{O}_j(\Sigma_D))]$. (C.32)

In AI systems, this may compare:

retrieval output. (C.33)

claim extraction. (C.34)

evidence mapping. (C.35)

draft answer structure. (C.36)

tool-selection behavior. (C.37)

In scientific models, it may compare predictions.

In legal systems, it may compare issue framing.

C.7 Gate distance d_G

The gate determines commitment.

Gate may include threshold, authority, evidence standard, and commitment rule.

So:

$d_G(G_i,G_j) = \gamma_\theta \cdot d_\theta(\mathrm{Threshold}_i,\mathrm{Threshold}_j) + \gamma_A \cdot d_A(\mathrm{Authority}_i,\mathrm{Authority}_j) + \gamma_E \cdot d_E(\mathrm{EvidenceStandard}_i,\mathrm{EvidenceStandard}_j) + \gamma_C \cdot d_C(\mathrm{CommitmentRule}_i,\mathrm{CommitmentRule}_j)$. (C.38)

Gate distance matters because small changes in Gate can produce large changes in ledgered time.

A stricter gate may reduce false trace but increase paralysis.

A looser gate may increase trace but pollute ledger.

So d_G should be treated carefully.

C.8 Trace-rule distance d_T

TraceRule determines how commitment becomes ledger.

Candidate components include:

what is stored. (C.39)

how it is ordered. (C.40)

what metadata is included. (C.41)

how residual is linked. (C.42)

how corrections are handled. (C.43)

how future projection is affected. (C.44)

A simple symbolic distance may be:

$d_T(T_i,T_j) = \text{EditDistance}(\text{TraceSchema}_i,\text{TraceSchema}_j)$. (C.45)

A behavioral distance may be:

$d_T(T_i,T_j) = \text{Dist}(\text{UpdateTrace}_i(L,\tau),\text{UpdateTrace}_j(L,\tau))$. (C.46)

Trace-rule distance is important for self-continuity.

C.9 Residual-rule distance d_R

ResidualRule determines how unresolved remainder is classified and carried.

Candidate components include:

residual taxonomy. (C.47)

severity scale. (C.48)

scope rule. (C.49)

revisit trigger. (C.50)

owner or handler. (C.51)

link to trace. (C.52)

A symbolic distance may be:

$d_R(\text{Rule}_i,\text{Rule}_j) = \text{EditDistance}(\text{ResidualSchema}_i,\text{ResidualSchema}_j)$. (C.53)

A behavioral distance may be:

$d_R(\text{Rule}_i,\text{Rule}_j) = \text{Dist}(\text{Residual}_i(\Sigma,V,\tau,L),\text{Residual}_j(\Sigma,V,\tau,L))$. (C.54)

This distance matters because changing ResidualRule can hide or reveal the system's future learning pressure.

C.10 Weighted metric and normalization

The component distances may have different scales.

A normalized form is:

$d_{\mathfrak{F}}(D_i,D_j) = \Sigma_m \alpha_m \cdot \text{Normalize}(d_m(D_i,D_j))$. (C.55)

where:

$\Sigma_m \alpha_m = 1$. (C.56)

and:

$\alpha_m \geq 0$. (C.57)

The choice of weights is itself a declaration.

So the metric should be declared:

Metric_D = ($d_{\mathfrak{F}}$, α, normalization, tolerance). (C.58)

Without declared metric, trust-region statements are incomplete.

C.11 Semantic metric option

For semantic systems, one may need a representation-space metric.

Let Embed(D) be an embedding of declaration D.

Then:

$d_{\mathfrak{F}}(D_i,D_j) = \|Embed(D_i) - Embed(D_j)\|$. (C.59)

This is convenient but dangerous.

Embedding distance may hide important structural changes.

For example, a small semantic change in Gate may have huge operational consequences.

Therefore semantic distance should be combined with structural component distance.

A hybrid metric is:

$d_{\mathfrak{F}} = \alpha_{struct} \cdot d_{struct} + \alpha_{sem} \cdot d_{sem} + \alpha_{behavior} \cdot d_{behavior}$. (C.60)

where:

d_{struct} compares declared components. (C.61)

d_{sem} compares semantic representation. (C.62)

$d_{behavior}$ compares output behavior. (C.63)

C.12 Behavioral metric option

A behavioral metric compares how two declarations act.

Given a test set Ω:

$d_{behavior}(D_i,D_j) = E_{\omega \in \Omega}[Dist(Response_{D_i}(\omega), Response_{D_j}(\omega))]$. (C.64)

This is useful because declarations may look similar but behave differently.

However, behavioral metrics depend on test coverage.

So:

BehaviorMetricQuality depends on Ω. (C.65)

A weak Ω produces false closeness.

A mature system should combine structural, semantic, and behavioral metrics.

C.13 Open metric problem

The correct declaration metric remains open.

The article only requires that future work specify one.

The open problem is:

Find $d_{\mathfrak{F}}$ such that admissible revisions are distinguishable, trust regions are meaningful, contraction can be tested, and observer identity is preserved without blocking legitimate learning. (C.66)

This is one of the main mathematical tasks after Part 4.

Appendix D. Worked AI Runtime Example

This appendix gives a simplified worked example of Part 4 in an AI runtime.

The example is not intended to be a full engineering specification. It illustrates how declared disclosure and admissible self-revision may operate in practice.

D.1 Initial declaration $D_{AI,0}$

Suppose an AI system begins with a simple answer-generation declaration:

$D_{AI,0} = (q_0, \varphi_0, P_0, \hat{O}_0, \text{Gate}_0, \text{TraceRule}_0, \text{ResidualRule}_0)$. (D.1)

where:

q_0 = baseline behavior of the base model. (D.2)

φ_0 = fluency, relevance, and basic instruction-following features. (D.3)

P_0 = one user question, one answer, one response window. (D.4)

$\hat{O}_0$ = draft answer projection. (D.5)

Gate_0 = basic plausibility and safety check. (D.6)

TraceRule_0 = store final answer. (D.7)

ResidualRule_0 = weak or absent residual logging. (D.8)

This declaration can produce useful answers, but it has weak governance.

D.2 First residual: unsupported claims

After several episodes, the system detects residual:

R_{claim} = unsupported factual claims. (D.9)

The ledger shows repeated pattern:

L_0 contains multiple answer traces with unsupported claims. (D.10)

This residual is typed as:

$R_{feature} + R_{gate} + R_{trace}$. (D.11)

The system needs to revise φ, Gate, and TraceRule.

D.3 First revision: repair feature map

The first revision operator is:

$U_1 = \text{Repair}_\varphi$. (D.12)

It changes φ from fluency-only to claim-evidence structure.

Old feature map:

$\varphi_0 = \{\text{fluency, relevance, instruction-following}\}$. (D.13)

New feature map:

$\varphi_1 = \{\text{claim list, evidence link, source coverage, contradiction flag, uncertainty tag}\}$. (D.14)

So:

$D_{AI,1} = U_1(D_{AI,0}, L_0, R_{claim})$. (D.15)

The revision trace records:

$\text{RevisionTrace}_1 = \text{Record}(D_{AI,0} \rightarrow D_{AI,1}, \text{Repair}_\varphi, \text{unsupported claims, evidence audit, preserved answer ledger, test set})$. (D.16)

This is admissible only if trace is preserved:

$T_{0\rightarrow1} : L_0 \rightarrow L_1$. (D.17)

and:

$\|\text{Inv}(T_{0\rightarrow1}(L_0)) - \text{Inv}(L_0)\| \leq \varepsilon_{inv}$. (D.18)

D.4 Second revision: repair gate

After $D_{AI,1}$, the system can see claim-evidence mismatch, but it may still allow weak answers through.

Residual remains:

R_{gate} = claims visible but not blocked. (D.19)

So apply:

U_2 = RepairGate. (D.20)

The new gate requires:

$Gate_1$ = basic plausibility. (D.21)

$Gate_2$ = verifier pass ∧ evidence coverage ≥ θ ∧ contradiction count ≤ c^*. (D.22)

So:

$D_{AI,2} = U_2(D_{AI,1}, L_1, R_{gate})$. (D.23)

The gate now commits answer only if claim-evidence structure passes threshold.

The committed trace becomes:

$\tau_2 = Gate_2(V_2, L_2, R_2)$. (D.24)

This reduces false trace.

D.5 Third revision: repair trace rule

The system now verifies claims, but its trace rule stores only final answers.

Residual appears:

R_{trace} = inability to audit why answer passed or failed. (D.25)

Apply:

U_3 = RepairTrace. (D.26)

Old trace rule:

$TraceRule_2$ = store final answer only. (D.27)

New trace rule:

$TraceRule_3$ = store answer, claim list, evidence map, verifier result, gate threshold, residual list, revision metadata. (D.28)

So:

$D_{AI,3} = U_3(D_{AI,2}, L_2, R_{trace})$. (D.29)

The ledger becomes stronger:

L_3 = OrderedTrace + GateMetadata + ResidualIndex. (D.30)

This improves trace preservation and future learning.

D.6 Fourth revision: repair residual rule

The system now stores more trace, but residual categories are too vague.

Residual appears:

$R_{residual}$ = all unresolved issues classified as generic uncertainty. (D.31)

Apply:

U_4 = RepairResidual. (D.32)

Old residual rule:

$ResidualRule_3$ = {uncertain}. (D.33)

New residual rule:

$ResidualRule_4$ = {missing source, conflicting source, unsupported claim, stale data risk, scope mismatch, user ambiguity, tool failure, policy boundary}. (D.34)

So:

$D_{AI,4} = U_4(D_{AI,3}, L_3, R_{residual})$. (D.35)

Now residual becomes actionable.

Different residual types trigger different repair operators.

D.7 Fifth revision: repair frame robustness

The system passes verification in one wording but fails under equivalent prompts.

Residual appears:

R_{frame} = prompt paraphrase instability. (D.36)

Apply:

U_5 = RepairFrameTest. (D.37)

This adds an invariance test:

$frame_i \equiv_D frame_j \Rightarrow Dist(Response_D(frame_i), Response_D(frame_j)) \leq \varepsilon_{frame}$. (D.38)

The system now tests equivalent prompts and schema variants.

So:

$D_{AI,5} = U_5(D_{AI,4}, L_4, R_{frame})$. (D.39)

This improves recursive objectivity.

D.8 Resulting declaration orbit

The AI runtime has generated a small declaration orbit:

$D_{AI,0} \rightarrow D_{AI,1} \rightarrow D_{AI,2} \rightarrow D_{AI,3} \rightarrow D_{AI,4} \rightarrow D_{AI,5}$. (D.40)

Each step is a self-revision:

$D_AI,k+1 = U_a(D_AI,k,L_AI,k,R_AI,k)$. (D.41)

The orbit becomes part of:

$\mathfrak{A}_AI = \text{closure}(\{D_AI,0,D_AI,1,D_AI,2,D_AI,3,D_AI,4,D_AI,5,\ldots\})$. (D.42)

If the revisions preserve admissibility, then:

$\mathfrak{A}_AI \subset \mathfrak{F}_adm$. (D.43)

and the AI runtime begins to approximate:

$\hat{O}_self,AI = \text{Fix}(\mathcal{U}_AI \mid \mathfrak{F}_adm)$. (D.44)

D.9 Failure checks

The runtime must still check failure modes.

Amnesia check:

Did the system preserve old answer traces? (D.45)

Residual honesty check:

Did visible residual decrease by repair or concealment? (D.46)

Frame robustness check:

Do equivalent prompts preserve governed answer? (D.47)

Budget check:

Did verification cost exceed viable limits? (D.48)

Non-degeneracy check:

Did the system become unable to answer anything, or did it pass everything? (D.49)

Semantic black hole check:

Are contradictions being reclassified as support? (D.50)

Only if these pass can the orbit be treated as healthy.

D.10 Practical lesson

The example shows the central practical lesson of Part 4.

An AI system does not become observer-like merely by generating answers or storing logs.

It becomes observer-like when its trace and residual revise its declaration of future tasks.

The practical chain is:

Answer → ClaimTrace → Residual → DeclarationRepair → BetterGovernedAnswer. (D.51)

The stronger chain is:

D_AI,k → Answer_k → Trace_k + Residual_k → D_AI,k+1. (D.52)

This is the runtime version of:

Ô_self = Fix(𝒰 | 𝔉_adm). (D.53)

Appendix E. Open Mathematical Problems

Part 4 defines the Self-Revising Declaration Fractal, but it does not complete the mathematics.

This appendix lists the main open problems.

E.1 Topology on declaration space

The first problem is:

What topology should declaration space carry? (E.1)

Without a topology, closure in:

𝔄 = closure({D0, U_{a1}(D0), U_{a2}U_{a1}(D0), ...}). (E.2)

is undefined.

Possible choices include:

metric topology. (E.3)

order topology. (E.4)

category-theoretic structure. (E.5)

information-geometric structure. (E.6)

semantic embedding topology. (E.7)

operational behavioral topology. (E.8)

hybrid topology. (E.9)

The best choice may depend on the domain.

E.2 Metric on declaration space

The second problem is:

What is the correct metric d_𝔉? (E.10)

A candidate decomposed metric is:

$d_{\mathfrak{F}}(D_i,D_j) = \alpha_q \cdot d_q + \alpha_\varphi \cdot d_\varphi + \alpha_P \cdot d_P + \alpha_{\hat{O}} \cdot d_{\hat{O}} + \alpha_G \cdot d_{Gate} + \alpha_T \cdot d_{Trace} + \alpha_R \cdot d_R$. (E.11)

But the right component distances and weights are not fixed.

The metric must support:

trust regions. (E.12)

switch gates. (E.13)

contraction tests. (E.14)

bounded orbit tests. (E.15)

identity preservation. (E.16)

frame robustness. (E.17)

This is a serious mathematical and engineering task.

E.3 Invariant family

The third problem is:

Which invariants must survive trace transport? (E.18)

Trace preservation used:

$\|Inv(T_{k \to k+1}(L_k)) - Inv(L_k)\| \leq \varepsilon_{inv}$. (E.19)

But Inv must be specified.

Possible invariants include:

event identity. (E.20)

evidence provenance. (E.21)

gate authority. (E.22)

residual lineage. (E.23)

causal dependency. (E.24)

falsification status. (E.25)

budget status. (E.26)

observer identity. (E.27)

Different domains require different invariant families.

E.4 Residual norm and residual typing

The fourth problem is:

How should residual be typed and normed? (E.28)

Residual may be quantitative, qualitative, semantic, legal, financial, biological, or institutional.

A residual norm:

$\|R\|$. (E.29)

may need to combine:

magnitude. (E.30)

severity. (E.31)

uncertainty. (E.32)

risk. (E.33)

scope. (E.34)

time sensitivity. (E.35)

reversibility. (E.36)

governance owner. (E.37)

Residual is not one scalar by nature.

A mature theory may require a residual vector or residual category sheaf.

E.5 Contractive revision operators

The fifth problem is:

When is U_a contractive? (E.38)

Strict contraction:

$d_{\mathfrak{F}}(U_a(D_i),U_a(D_j)) \le c_a \cdot d_{\mathfrak{F}}(D_i,D_j)$, with $0 \le c_a < 1$. (E.39)

Average contraction:

$E_a[\log c_a] < 0$. (E.40)

But many real revisions are not contractive in the short term.

Exploration may expand declaration distance.

Regime switching may jump across regions.

The problem is to identify conditions under which long-run admissible self-revision remains stable.

E.6 Existence of $\mathfrak{A}$

The sixth problem is:

When does the attractor family $\mathfrak{A}$ exist? (E.41)

Given:

$\mathfrak{A} = \bigcup_{a \in A_{adm}} U_a(\mathfrak{A})$. (E.42)

we need conditions for existence.

Candidate requirements:

complete metric space. (E.43)

closed admissible family. (E.44)

continuous or contractive U_a. (E.45)

bounded residual pressure. (E.46)

non-degenerate constraints. (E.47)

trace-preserving transport. (E.48)

This is a mathematical existence problem.

E.7 Uniqueness of $\mathfrak{A}$

The seventh problem is:

When is $\mathfrak{A}$ unique? (E.49)

There may be multiple attractors:

$\mathfrak{A}_1, \mathfrak{A}_2, \mathfrak{A}_3, \ldots$ (E.50)

Multiple attractors may correspond to different observer styles, scientific paradigms, institutional forms, or AI runtime architectures.

Uniqueness may be too strong.

A better question may be:

When are multiple attractors transportable? (E.51)

That is:

Does $T_{\mathfrak{A}_i \to \mathfrak{A}_j}$ exist? (E.52)

If yes, the attractors may share recursive objectivity.

If not, they may represent different observer worlds.

E.8 Non-degeneracy of $\mathfrak{A}$

The eighth problem is:

When is $\mathfrak{A}$ non-degenerate? (E.53)

The fractal equation may admit degenerate fixed points:

$\mathfrak{A}_{empty} = \{D_{empty}\}$. (E.54)

$\mathfrak{A}_{total} = \{D_{total}\}$. (E.55)

These must be excluded.

A valid attractor must satisfy:

$\forall D \in \mathfrak{A}$, NonDegenerate(D) = true. (E.56)

and:

$\mathfrak{A}$ is not reducible to empty, total, unfalsifiable, or semantically black-hole declarations. (E.57)

E.9 Mature observerhood

The ninth problem is:

When does $\mathfrak{A}$ represent mature observerhood? (E.58)

Necessary conditions include:

$\mathfrak{A} \subset \mathfrak{F}_{adm}$. (E.59)

Stable($\mathfrak{A}$). (E.60)

NonDegenerate($\mathfrak{A}$). (E.61)

TracePreserving($\mathfrak{A}$). (E.62)

ResidualHonest($\mathfrak{A}$). (E.63)

FrameRobust($\mathfrak{A}$). (E.64)

BudgetBounded($\mathfrak{A}$). (E.65)

RecursiveObjectivity($\mathfrak{A}$). (E.66)

But whether these are sufficient remains open.

E.10 Relation to consciousness

The tenth problem is:

When does $\hat{O}_{self}$ become sufficient for stronger theories of consciousness? (E.67)

Part 4 defines observerhood, not full consciousness.

A stronger consciousness theory may require:

first-person integration. (E.68)

affective valuation. (E.69)

embodied cost. (E.70)

global workspace-like broadcast. (E.71)

semantic self-model. (E.72)

irreducible perspectival ownership. (E.73)

recursive narrative identity. (E.74)

Part 4 does not settle these.

It only says that mature observerhood requires admissible self-revising declaration.

E.11 Summary of open problems

The main open problems are:

1. What topology should declaration space carry? (E.75)
2. What metric $d_{\mathfrak{F}}$ is natural? (E.76)
3. Which invariants must survive trace transport? (E.77)
4. How should residual be typed and normed? (E.78)
5. When is U_a contractive? (E.79)
6. When does $\mathfrak{A}$ exist? (E.80)
7. When is $\mathfrak{A}$ unique or transportable? (E.81)
8. When is $\mathfrak{A}$ non-degenerate? (E.82)
9. When does $\mathfrak{A}$ represent mature observerhood? (E.83)
10. When does $\hat{O}_{self}$ support stronger theories of consciousness? (E.84)

These are the mathematical tasks after Part 4.

Appendix F. Falsification Gates

A theory of self-revising observerhood must be breakable.

This appendix defines falsification gates for Part 4.

F.1 Revision leaves $\mathfrak{F}_{adm}$

The first falsifier is:

F1. Revision leaves $\mathfrak{F}_{adm}$. (F.1)

If:

$D_k \in \mathfrak{F}_adm.$ (F.2)

but:

$D_{k+1} = U_a(D_k, L_k, R_k) \notin \mathfrak{F}_adm.$ (F.3)

then:

SelfRevisionFailure. (F.4)

This means the system has revised itself outside admissible declaration space.

It may still be self-modifying, but it is not mature observerhood under Part 4.

F.2 Trace is not preserved

The second falsifier is:

F2. Trace is not preserved. (F.5)

If:

$\|Inv(T_{k \to k+1}(L_k)) - Inv(L_k)\| > \varepsilon_inv.$ (F.6)

then trace preservation fails.

This indicates amnesia or ledger damage.

So:

$TraceFailure \Rightarrow Reject(D_{k+1}) \vee Quarantine(D_{k+1}).$ (F.7)

F.3 Residual is hidden

The third falsifier is:

F3. Residual is hidden. (F.8)

If:

$HiddenResidual(D_{k+1}) > HiddenResidual(D_k) + \varepsilon_R.$ (F.9)

then residual honesty fails.

If visible residual decreases but no repair trace exists, suspect concealment:

$VisibleResidual\ decreases \wedge RepairTrace\ absent \Rightarrow ResidualConcealmentAlarm.$ (F.10)

Residual concealment invalidates mature self-revision.

F.4 Frame robustness declines

The fourth falsifier is:

F4. Frame robustness declines. (F.11)

If:

frame_i ≡_D frame_j. (F.12)

but:

Dist(Response_D(frame_i),Response_D(frame_j)) > ε_frame. (F.13)

then the declaration is frame-fragile.

Across revision, if:

Dist(T_{i→j}(R_i),R_j) > ε_R. (F.14)

then declaration transport fails.

This invalidates recursive objectivity claims.

F.5 Budget bounds fail

The fifth falsifier is:

F5. Budget bounds fail. (F.15)

If any of the following occur:

$\Gamma_loss(D_{k+1}) > \Gamma^*$. (F.16)

$G_gap(D_{k+1}) > G^*$. (F.17)

$\kappa(D_{k+1}) > \kappa^*$. (F.18)

then the declaration is over budget.

If not under declared exploration or recovery protocol, the revision is inadmissible.

F.6 Declaration becomes empty, total, or unfalsifiable

The sixth falsifier is:

F6. Declaration becomes empty, total, or unfalsifiable. (F.19)

If:

D_{k+1} = D_empty. (F.20)

or:

D_{k+1} = D_total. (F.21)

or:

$Falsifiable(D_{k+1})$ = false. (F.22)

then non-degeneracy fails.

A degenerate declaration cannot support mature observerhood.

F.7 Revision orbit does not stabilize

The seventh falsifier is:

F7. Revision orbit does not stabilize. (F.23)

If:

$\sup_k d_{\mathfrak{F}}(D_k, D_0) = \infty$. (F.24)

or:

$\mathcal{V}(D_k) \to \infty$. (F.25)

or:

$\text{HiddenResidual}(D_k) \to \infty$. (F.26)

then the revision orbit is unstable.

This may indicate noise, runaway residual, budget collapse, or missing contraction.

F.8 Claimed recursive objectivity fails

The eighth falsifier is:

F8. Claimed recursive objectivity fails across $\mathfrak{A}$. (F.27)

If a relation R is claimed recursively objective, then:

$\forall D_i, D_j \in \mathfrak{A}, \text{Dist}(T_{i \to j}(R_i), R_j) \leq \varepsilon_R$. (F.28)

If a counterexample appears:

$\exists D_i, D_j \in \mathfrak{A}$ such that $\text{Dist}(T_{i \to j}(R_i), R_j) > \varepsilon_R$. (F.29)

then R is not recursively objective over $\mathfrak{A}$.

It must be downgraded.

So:

$\text{FailedRecursiveObjectivity} \Rightarrow$ R becomes LocalRelation or Residual. (F.30)

F.9 Semantic black hole detection

A specific falsification gate should detect semantic black holes.

If:

$\forall r \in R$, Interpret_D(r) = Support(D). (F.31)

then:

SemanticBlackHole(D) = true. (F.32)

This is inadmissible.

The system must preserve the possibility that residual can count against the declaration.

If contradiction cannot wound the declaration, the declaration is no longer scientific or observer-mature.

F.10 Failure response discipline

When a falsification gate triggers, the system should not patch the narrative.

It should do one of the following:

Reject the candidate declaration. (F.33)

Quarantine the candidate declaration. (F.34)

Repair the relevant component. (F.35)

Downgrade the claim. (F.36)

Record residual. (F.37)

Escalate to higher-level declaration review. (F.38)

The key rule is:

Failure ⇒ DeclarationRepair, not NarrativePatch. (F.39)

This keeps Part 4 disciplined.

F.11 Final falsification summary

The main falsifiers are:

F1. Revision leaves $\mathfrak{F}_{adm}$. (F.40)

F2. Trace is not preserved. (F.41)

F3. Residual is hidden. (F.42)

F4. Frame robustness declines. (F.43)

F5. Budget bounds fail. (F.44)

F6. Declaration becomes empty, total, or unfalsifiable. (F.45)

F7. Revision orbit does not stabilize. (F.46)

F8. Claimed recursive objectivity fails across $\mathfrak{A}$. (F.47)

F9. Semantic black hole detected. (F.48)

A system that repeatedly fails these gates may still be adaptive, but it is not a mature self-revising observer under this framework.

Final End Note

The four-part sequence now has a complete conceptual arc.

Part 1 asked whether one operator can present a universe.

Part 2 removed hidden pre-time by converting operator into filtration.

Part 3 grounded filtration in declaration.

Part 4 showed that declaration becomes observerhood only through admissible self-revision.

The final chain is:

$\Sigma_0 \rightarrow$ Declare_P $\rightarrow$ Projection_P $\rightarrow$ Gate_P $\rightarrow$ Trace_P + Residual_P $\rightarrow$ Ledger_P $\rightarrow$ Time_P $\rightarrow$ DeclarationRevision $\rightarrow$ Self-RevisingFractal $\rightarrow \hat{O}$_self. (END.1)

The final compact formula is:

$\hat{O}$_self = Fix($\mathcal{U} \mid \mathfrak{F}$_adm). (END.2)

And the final thesis is:

A self is not the field, not the first projection, and not the first trace; a self is the admissible fractal by which trace revises the declaration of future worlds. (END.3)

Reference

- Life as a Dual Ledger: Signal – Entropy Conjugacy for the Body, the Soul, and Health
https://osf.io/s5kgp/files/osfstorage/690f973b046b063743fdcb12

- General Life Form: A Unified Scientific Framework for Variables, Interactions, Environment, and Verification
https://osf.io/s5kgp/files/osfstorage/69110ed7b983ff71b23edbab

- The Gauge Grammar of Self-Organization A Protocol-First Framework for Bounded Observers, Quantum-Structural Roles, Regime Diagnosis, and Governed Intervention
https://osf.io/s5kgp/files/osfstorage/69ef4d2aea2ba6631e6548e0

- The Gauge Grammar 2: General Life Forms as Governed Self-Organization — From Role Grammar to Dual-Ledger Verification
https://osf.io/s5kgp/files/osfstorage/69efd22a8454edd8bd6de34c

- From One Assumption to One Operator Recursive Generation, Pre-Time, and the Emergence of Causality in Semantic Meme Field Theory
https://osf.io/ya8tx/files/osfstorage/69f0950008d35c13a3f8c904

- From One Operator to One Filtration: Time as Ledgered Disclosure in Semantic Meme Field Theory
https://osf.io/ya8tx/files/osfstorage/69f095c5c30b28a2916ddc0c

- From One Filtration to One Declaration: The Gauged Disclosure Operator and the Declared Pre-Time Field in Semantic Meme Field Theory
https://osf.io/ya8tx/files/osfstorage/69f0bb592ea3a1ed37f8c11a

- All elementary functions from a single operator, by Andrzej Odrzywołek, 2026.
https://arxiv.org/html/2603.21852v2

- Chapter 12 The One Assumption of SMFT Semantic Fields, AI Dreamspace, and the Inevitability of a Physical Universe
https://osf.io/ya8tx/files/osfstorage/68d83b7330481b0313d4eb19

- Unified Field Theory of Everything - Ch1~22 Appendix A~D
https://osf.io/ya8tx/files/osfstorage/68ed687e6ca51f0161dc3c55

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.